

Объектно-ориентированное программирование

Подготовка к экзамену

4 семестр

2025 г.

СОДЕРЖАНИЕ

1. Структура программы на языках С и С++. Функции С и С++. Перегрузка функций в С++. Параметры функций по умолчанию.	9
1.1. Структура программы	9
1.2. Перегрузка функций	10
1.3. Параметры функций по умолчанию.	11
2. Ссылки. lvalue и rvalue ссылки. Передача параметров в функции по ссылке. Автоматическое выведение типа.	13
2.1. Ссылки	13
2.2. lvalue и rvalue ссылки	13
2.3. Передача параметров в функции по ссылке	14
2.4. Автоматическое выведение типа	15
3. Классы и объекты в С++. Определение класса с помощью class, struct, union. Ограничение доступа к членам класса в С++. Члены класса и объекта. Методы класса и объекта. Константные члены класса. Схемы наследования.	17
3.1. Классы и объекты в С++	17
3.2. Определение класса с помощью class, struct, union	17
3.3. Ограничение доступа к членам класса в С++	18
3.4. Члены класса и объекта. Константные члены класса.	19
3.5. Методы класса и объекта	20
3.6. Операторы, используемые для работы с объектами	21
3.7. Схема наследования	21
4. Создание и уничтожение объектов в С++. Конструкторы и деструкторы. Раздел инициализации конструкторов. Способы создания объектов. Явный и неявный вызов конструкторов. Приведение типа.	24
4.1. Создание и уничтожение объектов в С++	24
4.2. Конструкторы	24
4.3. Деструкторы	27
4.4. Способы создания объектов	29
4.5. Явный и неявный вызов конструкторов в С++	30
5. Конструкторы копирования и переноса. Модификатор explicit. Удаление конструктора и default конструктор. Делегирующие и унаследованные конструкторы.	32
5.1. Конструкторы копирования и переноса	32
5.2. Модификатор explicit	33
5.3. Удаление конструктора и default конструктор	34
5.4. Делегирующие и унаследованные конструкторы	36
6. Наследование в С++. Построение иерархии классов. Выделение общей части группы классов. Расщепление классов.	38
6.1. Наследование в С++	38

6.2.	Построение иерархии классов	39
6.3.	Расщепление классов	40
7.	Множественное наследование. Прямая и косвенная базы. Виртуальное наследование. Понятие доминирования. Порядок создания и уничтожения объектов. Проблемы множественного наследования. Неоднозначности при множественном наследовании.	42
7.1.	Множественное наследование	42
7.2.	Прямая и косвенная базы	42
7.3.	Виртуальное наследование	42
7.4.	Понятие доминирования	43
7.5.	Порядок создания и уничтожения объектов	43
7.6.	Проблемы множественного наследования	44
7.7.	Неоднозначности при множественном наследовании	45
8.	Полиморфизм в C++. Виртуальные методы. Чисто виртуальные методы. Виртуальные и чисто виртуальные деструкторы. Понятие абстрактного класса. Ошибки, возникающие при работе с указателем или ссылкой на базовый класс. Дружественные связи.	48
8.1.	Полиморфизм в C++	48
8.2.	Виртуальные методы	49
8.3.	Чисто виртуальные методы	51
8.4.	Виртуальный деструктор	51
8.5.	Чисто виртуальный деструктор	52
8.6.	Понятие абстрактного класса	53
8.7.	Ошибки, возникающие при работе с указателем или ссылкой на базовый класс	53
8.8.	Дружественные связи	54
9.	Обработка исключительных ситуаций в C++. Решение проблем структурного программирования. Блоки try и catch. Блоки try и catch методов и конструкторов. Безопасный код относительно исключений. Обёртывание исключения в exception_ptr. Задачи которые может решать исключение. Проблемы с динамической памятью при обработке исключительных ситуаций.	57
9.1.	Проблемы структурного программирования	57
9.2.	Обработка исключительных ситуаций в C++. Блоки try и catch	57
9.3.	Блоки try и catch методов и конструкторов	59
9.4.	Безопасный код относительно исключений	59
9.5.	Обёртывание исключения в exception_ptr	60
9.6.	Задачи, которые может решать исключение	62
9.7.	Проблемы с динамической памятью при обработке исключительных ситуаций	62

10. Перегрузка операторов в C++. Операторы .*, ->*. Правила перегрузки операторов. Перегрузка операторов =, () и []. Перегрузка операторов ->, * и ->*.	65
10.1. Перегрузка операторов в C++	65
10.2. Операторы .*, ->*	66
10.3. Правила перегрузки операторов	66
10.4. Перегрузка операторов =, (), []	67
10.5. Перегрузка операторов ->, *, ->*	69
11. Перегрузка унарных и бинарных операторов. Проблемы с перегрузкой операторов &&, , ,, &. Перегрузка операторов ++, --. Перегрузка операторов приведения типов. Тривалентный оператор spaceship.	72
11.1. Перегрузка унарных и бинарных операторов	72
11.2. Проблемы с перегрузкой операторов &&, , ,, &	73
11.3. Перегрузка операторов ++, --	74
11.4. Перегрузка операторов приведения типов	75
11.5. Тривалентный оператор spaceship	76
12. Шаблоны функций, методов классов и классов в C++. Недостатки шаблонов. Параметры шаблонов. Параметры типы и параметры значения. Шаблоны функций и методов классов. Подстановка параметров в шаблон. Выведение типов параметров шаблона. Явное указание значений типов параметров шаблона при вызове функции. Срезание ссылок и модификатора const.	80
12.1. Шаблоны функций, методов класса и классов в C++	80
12.2. Недостатки шаблонов	80
12.3. Параметры шаблонов. Параметры типы и параметры значения.	81
12.4. Шаблоны функций и методов классов	83
12.5. Подстановка параметров в шаблон	85
12.6. Выведение типов параметров шаблона	86
12.7. Пример явного указания типа параметра шаблона	87
13. Неявные шаблоны. Протаскивание типа передаваемого параметра через шаблон (шаблон std::forward). Определение типа с помощью decltype. decltype(auto). Специализация шаблонов функций.	88
13.1. Неявные шаблоны	88
13.2. Протаскивание типа передаваемого параметра через шаблон	88
13.3. Определение типа с помощью decltype. decltype(auto)	90
13.4. Специализация шаблонов функций	91
14. Шаблоны типов. Шаблоны классов. Полная или частичная специализация шаблонов классов. Параметры шаблонов задаваемых по умолчанию. Шаблоны с переменным числом параметров. Пространства имен.	94
14.1. Шаблоны типов и шаблоны классов	94
14.2. Полная и частичная специализация шаблонов классов	95
14.3. Параметры шаблонов по умолчанию	97

14.4. Шаблоны с переменным числом параметров	97
14.5. Пространства имён	99
15. Ограничения накладываемые на шаблоны. Требования к шаблонам (requires). Концепты. Типы ограничений. Варианты определения шаблонов функций и классов с концептами.	102
15.1. Ограничения накладываемые на шаблоны	102
15.2. Требования к шаблонам (requires)	102
15.3. Концепты	103
15.4. Типы ограничений (requires-выражений)	104
15.5. Варианты определения шаблонов функций и классов с концептами	105
16. Проблемы с динамическим выделением и освобождением памяти. Шаблон Holder. «Умные указатели» в C++: unique_ptr, shared_ptr, weak_ptr. Связь между shared_ptr и weak_ptr.	108
16.1. Проблемы с динамическим выделением и освобождением памяти	108
16.2. Шаблон Holder	109
16.3. Умные указатели в C++	110
16.4. Связь между shared_ptr и weak_ptr	113
17. Приведение типа в C++: static_cast, dynamic_cast, const_cast, reinterpret_cast. Контейнерные классы и итераторы. Требования к контейнерам и итераторам. Категории итераторов. Операции над итераторами. Цикл for для работы с контейнерными объектами.	115
17.1. Приведение типа в C++	115
17.2. Контейнерные классы и итераторы. Требования к контейнерам и итераторам	118
17.3. Категории итераторов	119
17.4. Цикл for для работы с контейнерными объектами	120
18. Структурное программирование: нисходящая разработка, использование базовых логических структур, сквозной структурный контроль.	122
18.1. Этапы разработки ПО	122
18.2. Использование базовых логических структур	125
18.3. Сквозной структурный контроль	125
19. Преимущества и недостатки структурного программирования. Идеи Энтони Хоара. Преимущества и недостатки объектно-ориентированного программирования	127
19.1. Преимущества структурного программирования	127
19.2. Недостатки структурного программирования	128
19.3. Идеи Энтони Хоара	128
19.4. Преимущества объектно-ориентированного программирования	129
19.5. Недостатки объектно-ориентированного программирования	130

20. Основные понятия ООП: инкапсуляция, наследование, полиморфизм. Понятие объекта. Категории объектов. Отношения между объектами. Понятие класса. Отношения между классами. Понятие домена.	132
20.1. Основные понятия ООП	132
20.2. Объект	132
20.3. Класс	133
20.4. Понятие домена	134
21. Цикл разработки ПО с использованием ООП: анализ, проектирование, эволюция, модификация. Рабочие продукты объектно-ориентированного анализа.	135
21.1. Цикл разработки ПО с использованием ООП	135
21.2. Рабочие продукты объектно-ориентированного анализа	137
22. Концепции информационного моделирования. Понятие атрибута. Типы атрибутов. Правила атрибутов. Понятие связи. Типы связей. Формализация связей. Композиция связей. Подтипы и супертипы. Диаграмма сущность-связь.	139
22.1. Концепции информационного моделирования	139
22.2. Атрибуты	139
22.3. Связи	140
22.4. Подтипы и супертипы	142
23. Модель поведения объектов. Жизненный цикл и диаграмма перехода в состояния (ДПС). Виды состояний. События, данные событий. Действия состояний. Таблица перехода в состояния (ТПС). Правила переходов.	143
23.1. Модель поведения объектов. Жизненный цикл	143
23.2. Диаграмма перехода в состояния (ДПС)	143
23.3. Таблица перехода в состояния (ТПС)	145
24. Модель взаимодействия объектов (МВО). Диаграмма взаимодействия объектов в подсистеме. Типы событий. Схемы управления. Имитирование. Каналы управления.	146
24.1. Модель взаимодействия объектов (МВО). Диаграмма взаимодействия объектов в подсистеме.	146
24.2. Типы событий	147
24.3. Схемы управления	147
24.4. Имитирование. Каналы управления	148
25. Диаграмма потоков данных действий (ДПДД). Типы процессов: аксессоры, генераторы событий, преобразования, проверки. Таблица процессов (ТП). Модель доступа к объектам (МДО).	149
25.1. Диаграмма потоков данных действий (ДПДД)	149
25.2. Типы процессов	150
25.3. Таблица процессов	151
25.4. Модель доступа к объектам (МДО)	151

26. Домены. Модели доменного уровня. Типы доменов. Мосты, клиенты, сервера.	153
26.1. Домены	153
26.2. Типы доменов	153
26.3. Мосты, клиенты, сервера	153
26.4. Модели доменного уровня	154
27. Объектно-ориентированное проектирование. Диаграмма класса. Структура класса. Диаграмма зависимостей. Диаграмма наследования.	155
27.1. Объектно-ориентированное проектирование.	155
27.2. Диаграмма класса	155
27.3. Структура класса	156
27.4. Диаграмма зависимостей	157
27.5. Диаграмма наследования	157
28. Порождающие паттерны: фабричный метод (Factory Method), абстрактная фабрика (Abstract Factory), строитель (Builder). Их преимущества и недостатки.	159
28.1. Порождающие паттерны	159
28.2. Фабричный метод (Factory Method)	159
28.3. Абстрактная фабрика (Abstract Factory)	162
28.4. Строитель (Builder)	164
29. Порождающие паттерны: одиночка (Singleton), прототип (Prototype), пул объектов (Object Pool). Их преимущества и недостатки.	169
29.1. Порождающие паттерны	169
29.2. Одиночка (Singleton)	169
29.3. Прототип (Prototype)	170
29.4. Пул объектов (Object Pool)	173
30. Структурные паттерны: адаптер (Adapter), декоратор (Decorator), компоновщик (Composite), мост (Bridge), заместитель (Proxy), фасад (Facade). Их преимущества и недостатки.	177
30.1. Структурные паттерны	177
30.2. Адаптер (Adapter)	177
30.3. Декоратор (Decorator)	179
30.4. Компоновщик (Composite)	182
30.5. Мост (Bridge)	186
30.6. Заместитель (Proxy)	189
30.7. Фасад (Facade)	192
31. Паттерны поведения: стратегия (Strategy), команда (Command), цепочка обязанностей (Chain of Responsibility), подписчик-издатель (Publish-Subscribe), посредник (Mediator). Их преимущества и недостатки.	196
31.1. Паттерны поведения	196
31.2. Стратегия (Strategy)	196

31.3. Команда (Command)	198
31.4. Цепочка обязанностей (Chain of Responsibility)	201
31.5. Подписчик-издатель (Publish-Subscribe)	205
31.6. Посредник (Mediator)	209
32. Паттерны поведения: посетитель (Visitor), опекун (Memento), шаблонный метод (Template Method), хранитель (Holder), итератор (Iterator), свойство (Property). Их преимущества и недостатки	213
32.1. Паттерны поведения	213
32.2. Посетитель (Visitor)	213
32.3. Опекун (Memento)	216
32.4. Шаблонный метод (Template Method)	219
32.5. Хранитель (Holder)	221
32.6. Итератор (Iterator)	222
32.7. Свойство (Property)	222

1. Структура программы на языках С и С++. Функции С и С++. Перегрузка функций в С++. Параметры функций по умолчанию.

1.1. Структура программы

1.1.1. На языке Си

- Подключение заголовочных файлов с помощью директивы `#include`.
- Определение констант, использование препроцессорных директив, таких как `#define`, для определения макросов.
- Объявление функций, чтобы уведомить компилятор о их существовании.
- Функция `main` — точка входа в программу.
 - Точка входа обязательно должна присутствовать.
 - `main` не может вызывать саму себя.
- Определение функций, где реализуется их логика.

1.1.2. На языке С++ (отличительные особенности)

- Существуют пространства имен, которые позволяют организовать код и избегать конфликтов имен.
- Существуют классы и объекты, которые являются основой парадигмы ООП.
- Существует возможность перегрузки функций, что позволяет создавать несколько функций с одинаковым именем, но разными параметрами.
- Существуют шаблоны функций и классов, которые позволяют создавать универсальный код, работающий с различными типами данных.
- Существуют исключения, которые позволяют обрабатывать ошибки и исключительные ситуации в программе.

1.1.3. Общие моменты

1) Объявление и определение функций

- Объявлений функций может быть несколько, но определение только одно.
- Функции могут быть объявлены в любом месте программы, но определены должны быть до их вызова.
- Функции не могут быть вложенными друг в друга (кроме лямбда-функций в С++).

2) Компиляция и связывание

- Программа может состоять из нескольких файлов, которые компилируются отдельно и затем связываются в единый исполняемый файл.
- На этапе компиляции каждый файл компилируется в объектный код.
- На этапе связывания объектные файлы объединяются в исполняемый файл с помощью компоновщика. На этом этапе могут подключаться библиотеки и другие зависимости.

3) Заголовочные файлы

- Заголовочные файлы содержат определения типов, констант времени компиляции, объявления функций, структур, классов и других элементов, которые могут использоваться в разных частях программы.
- Они включаются в исходные файлы с помощью директивы `#include`.
 - Для защиты от многократного включения заголовочных файлов используются условные компиляции с помощью директив `#ifndef`, `#define` и `#endif` или `#pragma once`.
- В заголовочных файлах **нельзя**:
 - Определять функции (только объявлять).
 - Определять глобальные переменные (только объявлять с ключевым словом `extern`).
 - Если определить переменную или функцию, то при каждом включении заголовочного файла будет происходить повторное определение, что приведет к ошибке компиляции.

1.2. Перегрузка функций

Перегрузка функций — Это возможность создавать несколько функций с одинаковым именем, но с разными параметрами (разные по количеству, типам или квалификаторам).

1.2.1. На перегрузку влияют

- Количество параметров
- Типы параметров
- Квалификаторы параметров (например, `const`, `volatile`), относящиеся к содержимому

```
1 int f(int &);           // 1
2 int f(const int &);    // 2 (const на то, на что указывает ссылка)
3 int f(int &&);          // 3
```

- Квалификаторы функций/методов

```
1 class A
2 {
3     void f();           // метод для неконстантных объектов
4     void f() const;     // перегрузка для константных объектов
5 };
```

- Константные методы гарантируют, что не будут изменять состояние объекта или вызывать неконстантные методы.

1.2.2. На перегрузку не влияют

- Возвращаемый тип функции
- Имена параметров
- Квалификаторы параметров, относящиеся к самому объекту

```

1 | int f(const int a); // (const на значение)
2 | int f(int* const a); // (const на указатель)

```

— Параметры по умолчанию

```

1 | void f(int a);
2 | void f(int a = 0); // ошибка: перегрузка не различается

```

1.2.3. Выбор перегрузки при вызове

Если «точной» перегрузки нет, компилятор всё равно пытается подобрать лучшую через неявные преобразования.

Возможны три исхода:

- 1) Нашёлся единственный лучший кандидат $\Rightarrow$ он вызывается
- 2) Подходящих несколько и ни один не «лучше» другого $\Rightarrow$ ошибка
- 3) Ни один не подходит (по числу, типам аргументов и т.п.) $\Rightarrow$ ошибка

1.3. Параметры функций по умолчанию.

Параметры функций по умолчанию — Это возможность задавать значения параметров функции, которые будут использоваться, если при вызове функции не переданы соответствующие аргументы.

```

1 | void f(int a = 0, int b = 1, int c = 2)
2 | {
3 |     // ...
4 | }

```

1.3.1. Где можно задавать

- В объявлении функции или в определении функции, но не в обоих местах одновременно.
- Обычно задаются в заголовочном файле, чтобы все вызовы функции могли использовать эти значения по умолчанию.

1.3.2. Правила

- Параметры по умолчанию должны быть справа от остальных параметров.

```

1 | void f(int a, int b = 0, int c = 1);
2 | void f(int a = 0, int b, int c = 1); // ошибка: после параметра по
   |   умолчанию -- обязательный

```

- При вызове функции можно пропускать только хвостовые параметры.

```

1 | void f(int a, int b = 2, int c = 3);
2 |
3 | f(1);           // a = 1, b = 2, c = 3
4 | f(1, 4);        // a = 1, b = 4, c = 3

```

```
5 f(1, 4, 5); // a = 1, b = 4, c = 5
6 // f(1, , 5); // Нельзя пропускать середину
```

- Можно указывать почти любое выражение, подходящее типу параметра, в т.ч. вызовы функций, `new`, константы и т.п. Не обязано быть `constexpr`.
- Выражение параметра по умолчанию вычисляется каждый раз при вызове, как обычное выражение (может иметь побочные эффекты).
- Можно добавлять дефолты при повторных объявлениях в той же области; при вызове берется объединение всех видимых дефолтов.
- Нельзя переопределить уже видимый дефолт (даже тем же значением).
- Значение параметра по умолчанию выбирается на этапе компиляции по статическому типу выражения, а виртуальный вызов выбирает только тело функции на этапе выполнения.

```
1 struct B { virtual void f(int x = 1); };
2 struct D : B { void f(int x = 2) override; };
3
4 D derived;
5 B& base = derived; // статический тип B, динамический D
6 base.f(); // вызовется D::f, но с x == 1
```

- На месте вызова `b.f()` компилятор видит тип `B&` $\Rightarrow$ подставляет `1` $\Rightarrow$ в рантайме виртуальный механизм выбирает `D::f`, но аргумент уже вычислен.

2. Ссылки. lvalue и rvalue ссылки. Передача параметров в функции по ссылке. Автоматическое выведение типа.

Ссылка — это ещё одно имя для **существующего** данного.

2.1. Ссылки

- Ссылка не является типом данных
- Ссылка обязательно должна быть инициализирована при создании
 - Ссылка не может быть переназначена
- Под ссылку не выделяется память
- Ссылки используются для передачи данных в функции для избежания копирования
- Ссылки не нужно контролировать, они не могут быть нулевыми
 - Но на низком уровне ссылки — это те же указатели
- Возвращать ссылки на локальные данные **нельзя!**
 - Исключение: возврат ссылки в подобных ситуациях:

```
1 maxElement(list) = 42; // изменяет значение максимального элемента в
   | списке
```

2.2. lvalue и rvalue ссылки

```
1 int i = 10; // Обычная переменная — lvalue
2 const int c = 0; // Константная переменная — тоже lvalue
```

2.2.1. lvalue

lvalue-ссылка — ссылка на lvalue-объект, у которого есть имя и адрес в памяти.

```
1 int& lv1 = i; // OK: обычная ссылка на изменяемое lvalue
2 const int& clv1 = c; // OK: ссылка на const lvalue
3
4 const int x = 5;
5 int& r = x; // Ошибка — нельзя убрать const
6 const int& cr = x; // OK
```

- `const T&` максимально гибкая ссылка:
 - Принимает как lvalue, так и rvalue
 - Не позволяет изменять значение

2.2.2. rvalue

rvalue-ссылка — ссылка на rvalue-объект (временный), который не имеет имени и адреса в памяти.

```
1 int&& rv1 = i;
2 // Ошибка: i — это lvalue, нельзя привязать его к rvalue-ссылке
```

```

3
4 int&& rv2 = rv1;
5 // Ошибка: rv1 – это переменная (имеет имя), значит она – lvalue, даже
  несмотря на тип int&&
6
7 int&& rv3 = 5;
8 // ОК: 5 – временное значение (rvalue), можно привязать к rvalue-ссылке
9
10 int&& rv4 = i + 0;
11 // ОК: выражение i + 0 – результат временного вычисления → rvalue
12
13 int&& rv5 = (int)i;
14 // ОК: приведение типа создаёт временный объект → rvalue
15
16 int&& rv6 = std::move(i);
17 // ОК: std::move превращает i (lvalue) во временный объект (rvalue), теперь
  можно привязать

```

- rvalue-ссылки позволяют захватывать ресурсы у временных объектов
- rvalue-ссылки «схлопываются» в lvalue-ссылки, если им дать имя

```

1 int&& x = 5;
2 f(x); // x – lvalue → вызовется версия с T&
3 f(std::move(x)); // вызовется версия с T&&

```

2.2.3. Универсальные ссылки (forwarding references)

В шаблонах `T&&` может быть универсальной ссылкой, которая подстраивается под аргумент:

```

1 template <typename T>
2 void func(T&& x);

```

- Если передано lvalue → `T = int&`, `T&& = int& &&` → сворачивается в `int&`
- Если передано rvalue → `T = int`, `T&& = int&&`
- Такая ссылка сохраняет категорию значения, и для правильной передачи используется `std::forward<T>(x)`.

2.3. Передача параметров в функции по ссылке

2.3.1. По lvalue-ссылке

```

1 void increment(int& x) { ++x; }
2
3 int a = 5;           // ОК
4 increment(a);        // a становится 6
5
6 increment(5);        // ошибка: 5 – rvalue

```

```

7 | const int b = 7;
8 | increment(b);    // ошибка: b – const

```

- Ссылка позволяет изменять оригинал
- Эффективно для больших объектов, так как не создаёт копию

2.3.2. По константной ссылке

```

1 | void print(const std::string& s) {
2 |     std::cout << s << '\n';
3 | }

```

- Подходит для любых объектов: и lvalue, и rvalue
- Безопасно, так как не изменяет оригинал
- Эффективно для больших объектов, так как не создаёт копию, но при этом и не позволяет захватить ресурсы

2.3.3. По rvalue-ссылке

```

1 | void take(std::string&& s) {
2 |     // можно "захватить" данные из s
3 |     std::string local = std::move(s);
4 | }

```

- Принимает только временные объекты
- Часто используется для перемещения ресурсов (move semantics), то есть без копирования
- Внутри функции обычно используют `std::move(s)`, чтобы получить владение ресурсами

2.3.4. По шаблонной универсальной ссылке

```

1 | template <typename T>
2 | void forwarder(T&& x) {
3 |     doSomething(std::forward<T>(x));
4 | }

```

- Позволяет принимать как lvalue, так и rvalue в зависимости от аргумента
- Без `forward` мы бы всегда передавали lvalue и теряли перемещение.

2.4. Автоматическое выведение типа

2.4.1. Ключевое слово `auto`

Ключевое слово `auto` позволяет компилятору автоматически определить тип переменной при её инициализации:

```

1 | auto x = 5;    // x : int
2 | auto y = 3.14; // y : double

```

— Ссылки не сохраняются автоматически

```
1 int i = 42;
2 int& r = i;
3
4 auto a = r; // a : int (ссылка теряется!)
5 auto& b = r; // b : int& (сохраняется как ссылка)
```

— `const` не сохраняется автоматически

```
1 const int i = 42;
2 auto a = i; // a : int (const теряется!)
3 const auto b = i; // b : const int
```

— Можно использовать универсальную ссылку

```
1 const int ci = 42;
2 auto&& x = ci;
```

- `auto` ведёт себя как параметр шаблона с выводением типа
- $U == \text{const int} \Rightarrow T = \text{const int\&} \Rightarrow T\&\& = \text{const int\& \&\&} \Rightarrow T\&\& = \text{const int\&}$

— Можно использовать `decltype(auto)` для полного сохранения типа:

```
1 int i = 42;
2 int& r = i;
3
4 auto a = r; // int (ссылка теряется)
5 decltype(auto) b = r; // int& (точно как r)
6
7 int& get();
8 auto a = get(); // int
9 decltype(auto) b = get(); // int&
```

2.4.2. Неявные шаблоны

В стандарте C++20 для функций и методов можно определить неявные шаблоны:

```
1 // эквивалентно template<typename T> void f(T x);
2 void f(auto x);
3 // эквивалентно template<typename T> void g(const T& x);
4 void g(const auto& x);
```

3. Классы и объекты в C++. Определение класса с помощью `class`, `struct`, `union`. Ограничение доступа к членам класса в C++. Члены класса и объекта. Методы класса и объекта. Константные члены класса. Схемы наследования.

3.1. Классы и объекты в C++

Класс — абстракция множества предметов реального мира такая, что все предметы в нём имеют одни и те же характеристики и общее поведение.

Объект класса — это конкретный представитель абстракции, заданной классом. Он воплощает в себе все характеристики и поведение, описанные в классе, но со своими конкретными значениями.

3.2. Определение класса с помощью `class`, `struct`, `union`

3.2.1. С помощью `class`

Это стандартный способ описания классов в C++. Используется, когда важна инкапсуляция и ООП-подход.

```
1 class MyClass
2 {
3     private:
4         int x_;
5
6     public:
7         void set(int value) { x_ = value; }
8         int get() const { return x_; }
9 };
```

- По умолчанию: члены `private`
- Поддерживает все возможности ООП: инкапсуляцию, наследование, полиморфизм

3.2.2. С помощью `struct`

Исторически пришла из языка C, где описывала просто набор полей. В C++ это полноценный класс с единственным отличием:

```
1 struct MyStruct
2 {
3     int x;
4
5     void set(int value) { x = value; }
6 };
```

- По умолчанию: члены `public`

- Можно использовать для ООП, но чаще — для «простых» структур, где нет скрытых данных

3.2.3. С помощью **union**

union — это специальный тип, в котором все члены занимают одну и ту же область памяти, то есть в каждый момент времени можно использовать только один член.

```
1 union MyUnion
2 {
3     int i;
4     float f;
5     char c;
6 };
```

- По умолчанию все члены **public**
- Все поля перекрываются в памяти
- Обычно используется для:
 - экономии памяти,
 - работы с низкоуровневыми представлениями данных (битовые поля, типы данных разных форматов)
- В C++ **union** может содержать конструкторы, деструкторы, методы и даже нестандартные типы (начиная с C++11)

3.3. Ограничение доступа к членам класса в C++

3.3.1. **private**

- Это значение по умолчанию в **class**.
- Члены доступны только изнутри самого класса или **friend**-функциям/классам.

```
1 class A
2 {
3     private:
4         int x; // доступен только внутри A
5     public:
6         void set(int val) { x = val; } // доступен извне
7 };
```

3.3.2. **protected**

- Доступ из:
 - самого класса,
 - **friend**,
 - классов-наследников.

```
1 class Base
2 {
3     protected:
```

```

4     int value;
5 };
6
7 class Derived : public Base
8 {
9     public:
10    void f()
11    {
12        value = 42; // можно
13    }
14 };

```

Но:

```

1 Base b;
2 b.value = 10; // Ошибка: protected не даёт доступ снаружи

```

3.3.3. public

— Доступ открыт всем, в том числе извне.

```

1 class Open
2 {
3     public:
4         int x;
5 };
6
7 Open o;
8 o.x = 5; // Разрешено

```

3.3.4. Принцип инкапсуляции

- Поля (данные) делают private или protected.
- Методы доступа (get/set) делают public.

3.4. Члены класса и объекта. Константные члены класса.

Вид	Описание
Обычные поля	Привязаны к каждому объекту. У каждого экземпляра своё значение.
<code>static</code> поля	Общие для всех объектов. Хранятся на уровне класса, а не объекта.
<code>const</code> поля	Неизменяемы после инициализации. Инициализируются в списке инициализации конструктора.
<code>mutable</code> поля	Могут изменяться даже в <code>const</code> -методах. Используются, когда нужно модифицировать «кэши», счётчики и т.п.
Ссылки как члены	Не могут быть проинициализированы по умолчанию, обязаны инициализироваться в списке инициализации конструктора.

3.5. Методы класса и объекта

Вид	Описание
Обычные методы	Работают с экземпляром класса.
<code>const</code> методы	Не изменяют объект. Обязательны при вызове у <code>const</code> объектов.
<code>static</code> методы	Не имеют доступа к <code>this</code> , не могут обращаться к нестатическим членам напрямую. Работает на уровне класса, общий для всех объектов. Может вызываться без создания объекта.
<code>virtual</code> методы	Поддерживают полиморфизм. Могут быть переопределены в наследниках.
Чисто виртуальные	Делают класс абстрактным. Не имеют реализации.
<code>inline</code> методы	Могут быть определены прямо в теле класса. Компилятор может встроить их.
<code>friend</code> функции	Не являются методами, но имеют доступ к <code>private</code> / <code>protected</code> членам.
Шаблонные методы	Параметризуются типами, задаются внутри <code>template<class T></code> и т.п.
Конструкторы/деструкторы	Специальные методы создания и разрушения объектов.

```
1 class MyClass
2 {
3     public:
4         static int counter;
5
6         static void increment() { ++counter; } // метод класса
7 };
```

Вызов:

```
1 MyClass::increment();
2 obj.increment(); // (разрешено, но не рекомендуется)
```

3.5.1. Определение методов, которые будут вызываться для определённых объектов

`&`, `&&`, `const &` позволяют перегружать методы в зависимости от типа объекта, для которого они вызываются:

Объявление метода	Что означает
<code>int f()</code>	Метод без ограничений — можно вызывать для любого объекта
<code>int f() &</code>	Метод вызывается только для lvalue-ссылок (<code>obj.f()</code>)
<code>int f() const &</code>	Только для <code>const</code> lvalue

Объявление метода	Что означает
<code>int f() &&</code>	Только для rvalue объектов (<code>std::move(obj).f()</code> или <code>Demo().f()</code>)
<code>int f() const &&</code>	Только для <code>const</code> rvalue, редко используется

3.6. Операторы, используемые для работы с объектами

Оператор	Синтаксис	Назначение	Перегружается	Пример кода
,	<code>expr1, expr2</code>	Вычисляет оба выражения слева направо, возвращает значение второго	Да	<code>(a.f(), a.g());</code>
->	<code>ptr->member</code>	Разыменование указателя и доступ к члену	Да	<code>ptr->method();</code>
::	<code>A::member</code>	Разрешение области видимости: доступ к члену класса/пространства имён	Нет	<code>A::g();</code> (вызов статического метода)
.*	<code>obj.*pm</code>	Доступ к методу или полю по указателю на член класса (через объект)	Да	<code>(a.*pm)();</code> где <code>pm</code> — указатель на метод
->*	<code>ptr->*pm</code>	Доступ к члену по указателю через указатель на объект	Да	<code>(ptr->*pm)();</code> где <code>pm</code> — указатель на метод

3.7. Схема наследования

Схема наследования — это структура, которая описывает, как классы наследуются друг от друга, какие члены и методы передаются, и как они переопределяются. В C++ поддерживаются различные виды наследования, а также их комбинации.

Наследование может быть по схемам

- `public`
 - По умолчанию для `struct`
 - Такая схема приводит к объединению интерфейсов
- `protected`

- Интерфейс базового класса остаётся доступным для классов, которые будут производными о производного класса (D передаёт интерфейс класса B своим производным классам)
 - При этом создаётся новый интерфейс
- private
- По умолчанию для **class**
 - Интерфейс базового класса недоступен и полностью заменяется

Наследование	Было в Base	Стало в Derived
public наследование	public	public
	protected	protected
	private	недоступно
protected наследование	public	protected
	protected	protected
	private	недоступно
private наследование	public	private
	protected	private
	private	недоступно

3.7.1. Возвращение уровня доступа базового поля

Если базовый класс содержит перегруженные функции, они не становятся автоматически доступными в производном классе, если:

- базовый класс унаследован по private или protected;
- в производном классе определена функция с тем же именем, даже с другой сигнатурой.

В таких случаях перегрузки «скрываются», и чтобы снова сделать их доступными — используется **using**.

```

1 class A
2 {
3     public:
4         void f(int);
5         void f(double);
6 };
7
8 class B : private A
9 {
10     public:
11         using A::f;    // возвращаем доступ к f(int) и f(double)
12         void f(char); // перегрузка, специфичная для B
13 };

```

Если не написать **using A::f;**, то **f(int)** и **f(double)** будут недоступны, и при вызове **b.f(3)** будет вызван только **f(char)** (если он подходит), иначе — ошибка.

3.7.2. Неявное приведение

При public-наследовании преобразование $D^* \rightarrow \text{Base}^* / D\& \rightarrow \text{Base}\&$ публично доступно — любой внешний код может его делать (неявно или `static_cast`).

При protected/private-наследовании такое преобразование существует, но оно не доступно снаружи. В обычном внешнем коде компилятор не имеет права использовать это преобразование (и неявно, и через `static_cast`). Оно разрешено только в «привилегированных» местах:

- protected: внутри D, его друзей и внутри производных от D;
- private: внутри D и его друзей (но не в производных от D).

```
1 struct Base {};  
2 struct Pub : Base {};           // public  
3 struct Prot : protected Base {}; // protected  
4 struct Priv : private Base {};  // private  
5  
6 void use(Base*);  
7  
8 void outside() {  
9     Pub p; use(&p); // OK: public наследование  
10    Prot pr; use(&pr); // ОШИБКА: преобразование скрыто снаружи  
11    Priv pv; use(&pv); // ОШИБКА: скрыто снаружи  
12 }  
13  
14 struct E1 : Prot {  
15     void m() { use(this); } // OK: мы внутри наследника Prot  
16 };  
17  
18 struct E2 : Priv {  
19     void m() { use(this); } // ОШИБКА: приватная база недоступна наследнику  
20 }
```

4. Создание и уничтожение объектов в C++. Конструкторы и деструкторы. Раздел инициализации конструкторов. Способы создания объектов. Явный и неявный вызов конструкторов. Приведение типа.

4.1. Создание и уничтожение объектов в C++

В C++ объект — это экземпляр класса. Процесс его создания и уничтожения управляется механизмами, встроенными в язык.

Жизненный цикл объекта:

- Выделение памяти под объект.
- Вызов конструктора — инициализация объекта.
- Работа с объектом (в пределах области видимости).
- Вызов деструктора — завершение жизни объекта.
- Освобождение памяти.

4.2. Конструкторы

Конструктор — это специальный метод класса, который вызывается автоматически при создании объекта и служит для инициализации его состояния.

- Конструктор совпадает с именем класса.
- Не имеет возвращаемого типа (даже `void`).
- Может быть перегружен по числу и типу параметров.

4.2.1. Порядок вызова конструктора

- При наследовании сначала вызывается конструктор виртуального базового класса, потом просто базового класса
- Вызывается раздел инициализации (`: a(x), b(42)`).
- Затем выполняется тело конструктора.
- Только после этого объект считается полностью созданным.

4.2.2. Виды конструкторов

По умолчанию (без параметров)

- `MyClass();`
- Создаётся компилятором, если не определён ни один другой конструктор.
- Обнуляет поля (если `= {}`).

С параметрами

- `MyClass(int x);`
- Никогда не создаётся автоматически.
- Может вызываться неявно, если не `explicit`.

Копирования

- `MyClass(const MyClass&);`
- Создаётся компилятором, если нет пользовательских конструкторов копирования и перемещения.
 - При наследовании базовый класс и его члены должны быть копируемыми.
- Поверхностное копирование по умолчанию.

Перемещения

- `MyClass(MyClass&&);`
- Создаётся компилятором, если нет пользовательских конструкторов копирования, перемещения и деструктора.
 - При наследовании базовый класс и его члены должны быть перемещаемыми.
- Используется при `return` и `std::move`.

Делегирующий

- `MyClass() : MyClass(0) {}`
- Создаётся только вручную.
- Делегирует вызов другому конструктору.

Для списка инициализации

- `MyClass(std::initializer_list<int>)`
- Не создаётся компилятором.
- Позволяет использовать синтаксис `MyClass x{1,2,3}`.

4.2.3. Правило пяти

Если класс владеет ресурсом (указатель, файл, дескриптор и т.п.), то мы обязаны определить:

```
1 Array(const Array&);           // Копирования
2 Array(Array&&);                // Перемещения
3 ~Array();                     // Деструктор
4 Array& operator=(const Array&); // Присваивание-копия
5 Array& operator=(Array&&);     // Присваивание-перенос
```

4.2.4. Статический конструктор

В C++ нет настоящих статических конструкторов, но есть инициализация статических членов класса.

«Статическим конструктором» в C++ считается код, который инициализирует статические поля класса — он выполняется один раз, до первого использования этого поля.

```
1 class A
2 {
3     static int counter; // объявление
4 };
```

```
5  
6 int A::counter = 42; // инициализация в .cpp-файле
```

Особенности:

- Выполняется до `main()`, если объект глобальный/статический.
- В теле класса нельзя инициализировать статическое поле значением:

```
1 class A  
2 {  
3     static int x = 5;           // ошибка до C++17  
4     static inline int x = 5;   // возможно с C++17  
5 };
```

4.2.5. Раздел инициализации конструктора

```
1 struct S {  
2     const int id;  
3     std::string name;  
4     S(int id, std::string n) : id(id), name(std::move(n)) {}  
5 };
```

- Только в нём инициализируются `const`-поля, ссылки, базовые классы и поля, не имеющие конструктора по умолчанию.
- Поля инициализируются в порядке объявления в классе, а не в порядке перечисления в списке инициализации.
- Более эффективен, так как нет создания пустого объекта и последующего присваивания.

4.2.6. Наследование конструкторов

- Конструкторы не наследуются автоматически.
 - Если бы конструктор просто наследовался, новые поля производного класса остались бы неинициализированными — именно поэтому по умолчанию конструкторы не наследуются.
- Производный класс должен явно вызвать конструктор базового класса через `: Base(...)`.
- C++11 и новее: можно писать `using Base::Base;` для наследования конструкторов.

4.2.7. Что запрещено для конструктора

- `static` — потому что вызывается для конкретного объекта.
- `const` — потому что он создаёт объект, а не работает с уже существующим.
- `virtual`, `override`, `final` — потому что таблица виртуальных функций (vtable) ещё не построена.
- `volatile`, `&`, `&&` — не допускаются у конструкторов.

4.2.8. Что может быть у конструктора

- `explicit` — запрещает неявные преобразования при инициализации.

- `constexpr` — разрешает вызов конструктора в константных выражениях (при соблюдении требований).
- `= default` — просим компилятор сгенерировать конструктор.
- `= delete` — запрещаем вызов определённого конструктора.
- `inline` допустим (как у любых функций), но редко нужен.

4.2.9. Виртуальные вызовы внутри конструктора

```

1 struct Base {
2     virtual void f();
3     Base(){ f(); } // <-- вызов здесь, в Base()
4 };
5 struct Der : Base {
6     void f() override;
7 };
8 Der d; // создаём Der

```

При входе в `Base()`:

- активен базовый под-объект,
- `this` указывает на базовую часть,
- `vptr` настроен на таблицу `Base`,
- поэтому `f()` → `Base::f` (а не `Der::f`).

Вывод:

- Производный класс сначала вызывает конструкторы базового класса.
- Это приводит к вызову виртуального метода базы, а не производного класса (как ожидалось).
- В деструкторе аналогично — тоже виртуальный метод базы, так как производный уже разрушен.

Поэтому не рекомендуется вызывать виртуальные методы в конструкторах и деструкторах.

4.3. Деструкторы

Деструктор — это специальный метод класса, вызываемый автоматически при уничтожении объекта. Его задача — освободить ресурсы, захваченные в процессе жизни объекта.

Свойство	Есть/Нет	Комментарий
Имя	<code>~ИмяКласса()</code>	Всегда начинается с <code>~</code>
Возвращаемый тип	Нет	Не должен быть указан, даже <code>void</code>
Параметры	Нет	Не принимает аргументы
Перегрузка	Нет	Только один деструктор
Наследование	Да	Наследуется и может быть виртуальным

Свойство	Есть/Нет	Комментарий
Может быть <code>virtual</code> , <code>override</code> , <code>final</code>	Да	Особенно важно при удалении через указатель на базовый класс
Может быть <code>= default</code> / <code>= delete</code>	Да	C++11
Может быть <code>constexpr</code>	Да	Начиная с C++20
Не может быть <code>static</code> , <code>const</code> , <code>volatile</code> , <code>&</code> , <code>&&</code>	Нет	Как и конструктор

4.3.1. Когда вызывается деструктор

- Автоматические объекты (на стеке): при выходе из области видимости.
- Глобальные/статические объекты: после завершения `main()` (или при завершении программы).
- Динамические объекты (в куче): только при `delete`.
 - Если забыть `delete`, деструктор не вызовется → утечка памяти.
- Основной механизм вызова: неявный
- Может вызываться явно, но это крайне не рекомендуется.

```
1 MyClass obj;
2 obj.~MyClass(); // крайне не рекомендуется – вручную!
```

- Деструктор лишь отпустит ресурсы, которыми владеет объект, но не освободит из-под него память
- При выходе из области видимости компилятор всё равно попытается вызвать деструктор ещё раз ⇒ получится «вторичное разрушение» несуществующего объекта ⇒ UB (неопределённое поведение).

4.3.2. Порядок вызова деструкторов

- Для составных объектов (имеющих поля других классов), деструкторы вызываются в обратном порядке:
 - Сначала уничтожается тело класса (в т.ч. поля),
 - Затем вызывается деструктор базового класса.

4.3.3. Виртуальный деструктор

Нужен только если объект будут удалять через указатель/умный указатель на базу (полиморфное удаление). Иначе при `delete basePtr`; с динамическим типом `Derived`:

- если `~Base()` не виртуальный ⇒ поведение неопределено: вызовется только `~Base()`, часть `Derived` не разрушится (ресурсы утекут, инварианты сломаются);
- если `~Base()` виртуальный ⇒ правильно вызовется цепочка `~Derived()` → `~Base()`.

```
1 struct Base {
2     ~Base() {} // не виртуальный
3 };
4 struct Derived : Base {
```

```

5   ~Derived() { /* Освобождение ресурсов */}
6 };
7
8 Base* p = new Derived;
9 delete p; // UB: ~Derived не вызовется (или хуже)

```

4.4. Способы создания объектов

4.4.1. Автоматическое создание (на стеке)

Создание переменной внутри области видимости (обычно — функция, блок, main):

```

1 MyClass a;           // вызывает конструктор по умолчанию
2 MyClass b(10);       // вызывает конструктор с параметром
3 MyClass c{1, 2};     // список инициализации (C++11)

```

Особенности:

- Память выделяется автоматически на стеке.
- Объект уничтожается при выходе из области видимости.
- Деструктор вызывается автоматически

4.4.2. Динамическое создание (в куче) — через **new**

```

1 MyClass* ptr1 = new MyClass();           // по умолчанию
2 MyClass* ptr2 = new MyClass(42);        // с параметром
3 MyClass* ptr3 = new MyClass{1, 2, 3}    // список инициализации

```

Особенности:

- Память выделяется в куче
- Возвращается указатель на объект.
- Объект существует пока не вызван **delete**.
 - Деструктор не вызовется, если забыть **delete**.

Для массивов:

```

1 MyClass* arr = new MyClass[10]; // вызывает 10 конструкторов
2 delete[] arr;                  // вызывает 10 деструкторов

```

4.4.3. Создание статических объектов

```

1 static MyClass s1; // внутри функции
2 MyClass s2;        // глобальный объект

```

Особенности:

- **static**-объекты живут весь срок жизни программы.
- Инициализируются при первом входе в область видимости (гарантировано один раз).
- Деструктор вызывается после завершения main.

4.4.4. Создание через инициализацию копией / перемещением

```
1 MyClass a(10);  
2 MyClass b = a; // копирование (если `MyClass(const MyClass&)`)  
3 MyClass c = std::move(a); // перемещение (если `MyClass(MyClass&&)`)
```

4.4.5. Создание временного объекта (анонимного)

```
1 MyClass(42); // временный объект, живёт до конца полного выражения
```

Используется, например, при:

```
1 func(MyClass(5)); // временный объект передаётся по значению
```

4.4.6. Создание через return из функции

```
1 MyClass make() {  
2     MyClass x(10);  
3     return x; // вызывает перемещающий или копирующий конструктор  
4 }
```

C++ может применить Return Value Optimization (RVO) и не вызвать вообще никаких конструкторов.

4.4.7. Создание с помощью умных указателей (RAII)

```
1 std::unique_ptr<MyClass> p1 = std::make_unique<MyClass>(10);  
2 std::shared_ptr<MyClass> p2 = std::make_shared<MyClass>(20);
```

Преимущества:

- Безопасно управляют временем жизни объекта.
- Не требуют вручную вызывать `delete`.

4.4.8. Особое внимание

- Не забывать `delete` для `new` — иначе утечки
- При использовании `delete` для массива, всегда писать `delete[]`.
- Не вызывать `delete` для объектов, созданных не через `new`
- Не использовать `new` и `delete` вместе с `shared_ptr` напрямую — лучше `make_shared`.

4.5. Явный и неявный вызов конструкторов в C++

4.5.1. Явный вызов конструктора

Это прямое указание, что именно нужно вызвать, с передачей аргументов:

```
1 MyClass a(10); // круглые скобки  
2 MyClass b{10}; // фигурные скобки (uniform initialization, C++11)  
3 MyClass c = MyClass(10); // временный объект + копирование/перемещение  
4 MyClass(42); // временный объект (анонимный)
```

```

5 MyClass* p1 = new MyClass(10); // динамически через new
6 MyClass* p2 = new MyClass{10}; // тоже динамически (список инициализации)
7 std::make_shared<MyClass>(10); // умный указатель
8 std::make_unique<MyClass>(10);
9 func(MyClass(10));           // передача временного объекта
10 return MyClass(10);         // возврат временного объекта

```

4.5.2. Неявный вызов конструктора

Конструктор вызывается как результат преобразования типа. Особенно опасно с одним аргументом:

```

1 MyClass a = 10;           // неявно вызовется MyClass(int)
2 MyClass b = {10};         // тоже неявно
3 MyClass c = MyClass{10}; // конструкция копирующего инициализатора
4 void func(MyClass);
5 func(10);                 // неявный вызов конструктора MyClass(int)
6 MyClass f()
7 {
8     return 10;             // возврат приводит int → MyClass(int)
9 }

```

4.5.3. Проблемы с неявным вызовом

Может произойти непреднамеренное преобразование типа, что приведёт к ошибкам.

```

1 class String
2 {
3     public:
4         String(int size); // выделяет строку длины size
5 };
6
7 void print(String s);
8
9 print(5); // неожиданно: создаётся String(5)!

```

- Ключевое слово **explicit** говорит компилятору: запрещаю неявные преобразования.
- Также можно удалить некоторые перегруженные конструкторы, чтобы предотвратить неявные вызовы.

5. Конструкторы копирования и переноса. Модификатор `explicit`. Удаление конструктора и default конструктор. Делегирующие и унаследованные конструкторы.

5.1. Конструкторы копирования и переноса

5.1.1. Конструктор копирования

```
1 MyClass(const MyClass& other);
```

Создаёт новый объект как копию существующего (по ссылке).

Когда вызывается

— При передаче объекта по значению:

```
1 void f(MyClass obj); // копирование
2 MyClass a;
3 f(a);                // вызов copy ctor
```

— При инициализации копией:

```
1 MyClass a;
2 MyClass b = a; // вызов copy ctor
3 MyClass c(a);  // вызов copy ctor
```

— При возврате из функции (если нет RVO и перемещающий ctor не применим):

```
1 MyClass make()
2 {
3     MyClass x;
4     return x; // может быть копирование
5 }
```

5.1.2. Конструктор переноса

```
1 MyClass(MyClass&& other);
```

Использует ресурсы временного (rvalue) объекта, избегая копирования.

Когда вызывается

— При `std::move`:

```
1 MyClass a;
2 MyClass b = std::move(a); // вызов move ctor
```

— При возврате временного объекта из функции:

```

1 MyClass make()
2 {
3     return MyClass(); // move
4 }

```

— При инициализации от временного:

```

1 MyClass b = MyClass(); // move

```

5.1.3. Поведение по умолчанию

Если не определить копирующий/перемещающий конструктор, компилятор может сгенерировать их автоматически:

— Копирование

- Если нет пользовательских конструкторов копирования и перемещения.
- При наследовании базовый класс и его члены должны быть копируемыми.

— Перемещение

- Если нет пользовательских конструкторов копирования, перемещения и деструктора.
- При наследовании базовый класс и его члены должны быть перемещаемыми.

5.1.4. Проблемы с автоматическим конструктором копирования

- Поверхностное копирование (shallow copy) — просто копирует указатели, не данные по адресам
- Копирование одинаковых полей дважды (при множественном наследовании)
- Неправильное копирование объектов с указателями (риск двойного освобождения памяти)

5.2. Модификатор `explicit`

Модификатор `explicit` — ключевое слово, применяемое к конструкторам, которое запрещает неявные преобразования типов и копирующую инициализацию.

Если у конструктора **один параметр** (или ещё параметры со значениями по умолчанию), он может использоваться для преобразования типов, даже если мы этого не ожидаем:

```

1 class A
2 {
3     public:
4         A(int x) { std::cout << "A(int)\n"; }
5 };
6
7 void func(A a) { /* ... */ }
8
9 int main()
10 {

```

```

11     func(42); // автоматически создаётся A(42)
12 }

```

С помощью **explicit** мы можем запретить неявные преобразования:

```

1 class A
2 {
3     public:
4         explicit A(int x) { std::cout << "A(int)\n"; }
5 };
6
7 void func(A a) { /* ... */ }
8
9 int main()
10 {
11     func(42); // ошибка: неявное преобразование запрещено
12     func(A(42)); // ОК: явное создание временного объекта
13 }

```

5.2.1. Где **explicit** работает

Контекст	Без explicit	С explicit
A a = 5;	Да	Нет — ошибка
A a = {5};	Да	Нет — ошибка
A a(5);	Да	Да
A a{5};	Да	Да
func(5); // void func(A)	Да	Нет — ошибка
func(A(5));	Да	Да
return 5; // из функции A f()	Да	Нет — ошибка

5.3. Удаление конструктора и default конструктор

5.3.1. Удаление конструкторов (= **delete**)

Ключевое слово = delete — оно сообщает компилятору: не генерируй и не разрешай вызов этой функции (в т.ч. конструктора). Это способ запретить использование определённых функций.

```

1 class A
2 {
3     public:
4         A() = delete; // запрет конструктора по умолчанию
5         A(const A&) = delete; // запрет копирования
6         A(int x) {} // разрешён только такой
7 };
8
9 A a; // ошибка: A() удалён

```

Где применяют = delete:

Можно удалить?	Пример
Конструктор	<code>MyClass() = delete;</code>
Метод класса	<code>void f(double) = delete;</code>
Обычная функция	<code>void f(int) = delete;</code>
Шаблонная функция/метод	<pre> 1 struct MyInt { /*...*/ }; 2 3 // Обобщённая функция сравнения — нельзя 4 template<typename T, typename U> 5 bool operator==(const T&, const U&) = delete; 6 7 // Но конкретные сравнения для MyInt мы реализуем 8 bool operator==(MyInt const&, MyInt const&); </pre>
Оператор	<code>operator=(const MyClass&) = delete;</code>

5.3.2. Default конструктор (= default)

Default конструктор — конструктор без параметров или с параметрами по умолчанию для всех аргументов. С помощью `= default` можно попросить компилятор сгенерировать такой конструктор.

```

1 class Simple
2 {
3     public:
4         // Явно запрашиваем у компилятора создание конструктора по умолчанию
5         Simple() = default;
6
7         // Другие конструкторы
8         Simple(int x) : value(x) {}
9
10    private:
11        int value = 0;
12 };

```

Особенности

- Гарантирует точную семантику, определённую стандартом
- Может применяться **только** к любой из 6 специальных методов (конструктор по умолчанию, копирования, переноса, деструктор, операторы присваивания)
- Часто используется при наличии других пользовательских конструкторов

5.4. Делегирующие и унаследованные конструкторы

5.4.1. Делегирующие конструкторы

Делегирующий конструктор — конструктор одного класса вызывает другой конструктор того же класса в списке инициализации.

```
1 class Vec2 {
2     float x_, y_;
3
4     public:
5     Vec2() : Vec2(0.f, 0.f) {} // делегирование к «главному»
6     explicit Vec2(float v) : Vec2(v, v) {}
7     Vec2(float x, float y) : x_(x), y_(y) {} // главный ctor
8 };
```

Особенности

- Позволяет избежать дублирования кода при наличии множества конструкторов
- В списке инициализации может быть только сам вызов целевого конструктора, члены класса инициализировать нельзя
- Нельзя создавать цикл в делегировании (это приведёт к ошибке компиляции)
- Весь код в теле делегирующего конструктора выполняется после завершения делегированного конструктора
- Сначала выполняется целевой конструктор, затем тело делегирующего

5.4.2. Унаследованные конструкторы

Унаследованные конструкторы — механизм, позволяющий производному классу наследовать конструкторы базового класса без их явного переопределения.

```
1 class Base
2 {
3     public:
4     Base(int x) { std::cout << "Base(int)\n"; }
5 };
6
7 class Derived : public Base
8 {
9     public:
10    using Base::Base; // унаследовать конструкторы
11 };
12
13 Derived d(5); // вызовет Base(int)
```

Особенности

- Включаются с помощью `using Base::Base;` в теле производного класса
- Наследуются все конструкторы, кроме `private`

- Может возникнуть неоднозначность:

```
1 struct A { A(int); };
2 struct B { B(int); };
3
4 struct C : A, B {
5     using A::A;
6     using B::B;
7 };
8
9 C c(1); // Ошибка: двусмысленно (A(int) или B(int))
10
11 struct C2 : A, B {
12     using A::A; // только A-конструкторы
13     // или явно мост:
14     // C2(int x) : A(x), B() {}
15 };
```

- Наследуемые конструкторы **не инициализируют члены производного класса**
- Параметры конструкторов по умолчанию не наследуются
- Собственные члены производного класса инициализируются значениями по умолчанию
- Доступ у унаследованных конструкторов — как у места `using` (можно «поднять» `protected` → `public`)
- Если определить свой конструктор с той же сигнатурой, что и унаследованный, то он перекроет унаследованный
- Унаследованные конструкторы считаются пользовательскими конструкторами

6. Наследование в C++. Построение иерархии классов. Выделение общей части группы классов. Расщепление классов.

6.1. Наследование в C++

Наследование — Это механизм ООП, позволяющий создавать новый класс (производный) на основе существующего (базового), с возможностью переиспользовать его функциональность и добавлять новое поведение.

6.1.1. Синтаксис

```
1 class Base
2 {
3     protected:
4         int value;
5 };
6
7 class Derived : public Base
8 {
9     public:
10         void set(int v) { value = v; }
11 };
```

Модификатор `final` позволяет запретить дальнейшее наследование от класса:

```
1 class Base final {}; // нельзя создать class Derived : public Base
```

Это используется, когда класс задуман как завершённый и не предполагает дальнейшей иерархии.

6.1.2. Схемы наследования

— `public`

- По умолчанию для `struct`
- Такая схема приводит к объединению интерфейсов

— `protected`

- Интерфейс базового класса остаётся доступным для классов, которые будут производными о производного класса (D передаёт интерфейс класса B своим производным классам)
- При этом создаётся новый интерфейс

— `private`

- По умолчанию для `class`
- Интерфейс базового класса недоступен и полностью заменяется

Наследование	Было в Base	Стало в Derived
public наследование	public	public
	protected	protected
	private	недоступно
protected наследование	public	protected
	protected	protected
	private	недоступно
private наследование	public	private
	protected	private
	private	недоступно

6.1.3. Особенности

- Конструкторы и операторы присваивания не наследуются по умолчанию. Но можно «протянуть» конструкторы:

```

1 class D : Base
2 {
3     public:
4         using Base::Base;
5 };

```

- Деструктор базы не «наследуется» как член; у производного будет свой.
- Друзья не наследуются.
- Имена могут скрываться: если в Derived есть `void pub(int);`, он скрывает все перегрузки `pub` из базы
 - Их можно вернуть `using Base::pub;`

6.2. Построение иерархии классов

Цель: Упростить архитектуру программы, сократить дублирование кода, выделить абстракции.

```

1 class Shape
2 {
3     public:
4         virtual void draw() = 0;
5         virtual ~Shape() = default;
6 };
7
8 class Circle : public Shape
9 {
10     public:
11         void draw() override { /* реализация */ }
12 };
13
14 class Rectangle : public Shape
15 {

```

```
16 public:
17     void draw() override { /* реализация */ }
18 };
```

- Shape — абстрактный базовый класс (интерфейс)
- Circle, Rectangle — конкретные реализации

6.2.1. Когда нужно выделять базовую абстракцию

- Если предполагается, что поведение класса в будущем будет расширяться или изменяться — удобно создать базовый интерфейс и наследников.
- Если разные классы имеют одинаковые действия — общие методы можно вынести в базовый класс.
- Если один и тот же класс используется в разных частях программы — возможно, удобнее разбить его на абстрактную базу и разные реализации.
- Если при обсуждении проекта несколько классов упоминаются вместе — возможно, у них есть общая абстракция.

6.3. Расщепление классов

Расщепление класса (class decomposition) — это обратный процесс: когда один класс делает слишком много, его делят на более простые специализированные классы.

6.3.1. Признаки:

- Класс нарушает принцип единственной ответственности
- Много методов, слабо связанных между собой (одни методы работают с одной группой полей, другие с другой)
- Часто вносятся изменения в одни и те же части класса при разных правках

Пример до

```
1 class Report
2 {
3 public:
4     void generate();
5     void exportToPDF();
6     void sendEmail();
7 };
```

Пример после

```
1 class ReportGenerator
2 {
3 public:
4     void generate();
5 };
6
7 class PDFExporter
```

```
8 {  
9     public:  
10         void exportToPDF(const ReportGenerator&);  
11 };  
12  
13 class EmailSender  
14 {  
15     public:  
16         void sendEmail(const ReportGenerator&);  
17 };
```

6.3.2. Когда нужно расщеплять класс

- Если один объект играет несколько ролей — разумно расщепить его на несколько компонентов.
- Если на одну сущность возлагается слишком много обязанностей — надо расщеплять, чтобы упростить сопровождение.

7. Множественное наследование. Прямая и косвенная базы. Виртуальное наследование. Понятие доминирования. Порядок создания и уничтожения объектов. Проблемы множественного наследования. Неоднозначности при множественном наследовании.

7.1. Множественное наследование

Множественное наследование — это способность класса наследоваться сразу от нескольких базовых классов.

```
1 class A { ... };
2 class B { ... };
3 class C : public A, public B { ... };
```

Это позволяет повторно использовать интерфейсы и функциональность из разных источников. Например, класс `WindowWithScrollBar` может унаследовать `Window` и `ScrollBar`.

7.2. Прямая и косвенная базы

Прямая база — класс, указанный непосредственно при наследовании.

```
1 class A {};
```

```
2 class B {};
```

```
3 class C : public A, public B {}; // A и B — прямые базы C
```

— Прямая база может входить в класс только один раз.

Косвенная база — класс, находящийся в иерархии выше (база базы).

```
1 class A {};
```

```
2 class B : public A {};
```

```
3 class C : public B {}; // A — косвенная база для C
```

— Косвенная база может входить в класс несколько раз.

7.3. Виртуальное наследование

Виртуальное наследование — наследование, при котором косвенная база входит один раз. Используется для решения проблемы ромбовидного наследования.

Проблема ромбовидного наследования:

```
1 class A {};
```

```
2 class B : public A {};
```

```
3 class C : public A {};
```

```
4 class D : public B, public C {}; // D содержит 2 копии A!
```

Решение — виртуальное наследование:

```
1 class A {};  
2 class B : virtual public A {};  
3 class C : virtual public A {};  
4 class D : public B, public C {}; // D содержит 1 копию A
```

7.4. Понятие доминирования

Доминирование — когда один и тот же метод приходит в класс по нескольким путям, более «нижестоящий» (ближе к текущему классу) метод доминирует (подменяет метод базового класса) и снимает неоднозначность.

7.4.1. Примеры

Ромб с одним переопределением

```
1 class A { public: virtual void f(); };  
2 class B : public virtual A { };  
3 class C : public virtual A { public: void f() override; };  
4  
5 class D : public B, public C  
6 {  
7     public:  
8         void test() { f(); } // Однозначно: вызывается C::f()  
9 };
```

— Здесь `C::f` доминирует над `A::f`, пришедшей через `B`, поэтому неоднозначности нет.

Ромб с двумя переопределениями

```
1 class A { public: virtual void f(); };  
2 class B : public virtual A { public: void f() override; };  
3 class C : public virtual A { public: void f() override; };  
4  
5 class D : public B, public C  
6 {  
7     public:  
8         // Без явного выбора — неоднозначно (два равноправных overrider'а)  
9         void f() override { B::f(); } // Явно выбираем ветку  
10        // либо: void f() override { C::f(); }  
11 };
```

7.5. Порядок создания и уничтожения объектов

7.5.1. Создание

— Сначала создаются базовые классы

- Виртуальные базы — сначала и один раз в порядке объявления (слева направо)
- Потом прямые неvirtуальные базы в порядке объявления
- Затем создаются поля текущего класса (в порядке объявления)
- Потом выполняется тело конструктора

Особенности

- Инициализирует виртуальную базу всегда **самый производный класс** (тот, чей конструктор сейчас реально строит «полный объект»)
 - Инициализации `A(...)` в конструкторах промежуточных В/С игнорируются, когда строится D
 - Поэтому в классе D обязательно должна быть инициализация A
 - Если не вызвать конструктор A, то вызовется конструктор A по умолчанию
 - Если конструктор по умолчанию не определён, то в D нужно явно вызвать конструктор виртуальной базы

7.5.2. Уничтожение

- Происходит в обратном порядке

7.6. Проблемы множественного наследования

- Дублирование базовых классов (без `virtual`)
- Неоднозначности при доступе к членам
- Сложность конструкции, трудность сопровождения
- Неоднозначный порядок вызова (при множественном виртуальном наследовании)

7.6.1. Проблема множественного вызова метода

В ромбе $A \leftarrow B/C \leftarrow D$ часто пишут так (пример с лекции про метод `draw`):

- `A::f()` делает «базовую часть» работы.
- `B::f()` и `C::f()` каждый вызывает `A::f()` и добавляет своё.
- `D::f()` вызывает и `B::f()`, и `C::f()`.

Итог: `A::f()` отрабатывает дважды — по обеим веткам. Виртуальное наследование здесь не спасает: оно гарантирует один объект A, но не предотвращает двойной вызов кода, который мы сами дважды позвали.

Решение

Разделим метод на две части: защищённую и публичную

- В A сделаем защищённый `_f()` (только своя часть), а публичный `f()` — лишь делегирует на `_f()`.
- В B и C тоже завести `_f()` для своей части; их публичные `f()` могут вызывать `A::_f()` и свой `_f()`, когда ветка вызывается отдельно.
- В D: `f()` не вызывать `B::f()/C::f()` (иначе снова потянем «сверху вниз»), а вручную исполнить ровно по одному защищённому с каждого уровня:

```
1 A::_f();
2 B::_f();
```

```
3 C::_f();
4 this->_f();
```

Так `A::_f()` сработает один раз.

7.7. Неоднозначности при множественном наследовании

Неоднозначность возникает, когда компилятор не может однозначно определить, какой из членов (функций или полей) следует использовать, потому что они унаследованы из нескольких базовых классов с одинаковыми именами.

7.7.1. Неоднозначность методов с одинаковым именем

```
1 class A { public: void hello() {} };
2 class B { public: void hello() {} };
3 class C : public A, public B {};
4
5 int main()
6 {
7     C c;
8     c.hello();    // ошибка: неоднозначность
9     c.A::hello(); // явно указали путь
10 }
```

- Компилятор видит `A::hello()` и `B::hello()` и не знает, какой из них выбрать.
- Решение: указать явно: `c.A::hello()`; или `c.B::hello()`;

7.7.2. Одинаковые поля

```
1 class A { public: int value = 1; };
2 class B { public: int value = 2; };
3 class C : public A, public B {};
4
5 int main()
6 {
7     C c;
8     cout << c.value;    // ошибка: неоднозначность
9     cout << c.A::value; // 1
10    cout << c.B::value; // 2
11 }
```

7.7.3. Неоднозначность даже при разном уровне доступа

```
1 class A { private: void f() {} };
2 class B { public: void f() {} };
3 class C : public A, public B
4 {
5     public:
6     void call()
7     {
```

```

8      f(); // ошибка: неоднозначность, даже несмотря на private
9  }
10 };

```

Важно

- Компилятор сначала проверяет наличие неоднозначности, и только потом — доступность.
- То есть `A::f` и `B::f` конфликтуют по имени, даже если одна из них `private`.

7.7.4. Устранение неоднозначности через `using`

```

1  class A
2  {
3      public:
4          void f(int);
5          void f(double);
6  };
7
8  class B : public A
9  {
10     public:
11         using A::f; // возвращает все перегрузки
12         void f(char);
13 };
14
15 int main()
16 {
17     B b;
18     b.f(42); // вызывает A::f(int)
19     b.f(3.14); // вызывает A::f(double)
20     b.f('c'); // вызывает B::f(char)
21 }

```

Без `using A::f` — перегрузки `f(int)` и `f(double)` будут скрыты, и доступна будет только `B::f(char)`.

7.7.5. Неоднозначность при ромбовидном наследовании

```

1  class A { public: void f() {} };
2  class B : public A {};
3  class C : public A {};
4  class D : public B, public C {};
5
6  int main()
7  {
8      D d;
9      d.f(); // ошибка: две копии A — неоднозначность
10 }

```

Решение: виртуальное наследование

```
1 class A { public: void f() {} };
2 class B : virtual public A {};
3 class C : virtual public A {};
4 class D : public B, public C {};
5
6 int main()
7 {
8     D d;
9     d.f(); // ОК, одна копия A
10 }
```

8. Полиморфизм в C++. Виртуальные методы. Чисто виртуальные методы. Виртуальные и чисто виртуальные деструкторы. Понятие абстрактного класса. Ошибки, возникающие при работе с указателем или ссылкой на базовый класс. Дружественные связи.

8.1. Полиморфизм в C++

Полиморфизм — это способность одного интерфейса работать с разными типами данных. В контексте ООП — это возможность обрабатывать объекты производных классов через указатель или ссылку на базовый класс, при этом вызывая переопределённые методы.

Другое определение: это возможность подменять одно другим, не изменяя написанный код.

8.1.1. Виды полиморфизма

Статический (compile-time)

Перегрузка/шаблоны — выбор реализации делает компилятор по статическому типу выражения.

```
1 int area(int w, int h); // прямоугольник
2 double area(double r); // круг
3
4 auto s1 = area(3, 4); // выбрана версия (int,int) на этапе компиляции
5 auto s2 = area(2.0); // выбрана версия (double)
```

Динамический (run-time)

Виртуальные функции — выбор делается во время выполнения программы по динамическому типу объекта через vtable.

```
1 class Shape
2 {
3     public:
4         virtual double area() const = 0;
5         virtual ~Shape() = default;
6 };
7
8 class Circle : public Shape
9 {
10     double r;
11     public:
12         explicit Circle(double rr) : r(rr) {}
13         double area() const override { return 3.14 * r * r; }
14 };
```

```

15
16 void printArea(const Shape& s)
17 {
18     std::cout << s.area(); // вызов решается в run-time
19 }

```

8.2. Виртуальные методы

Виртуальные методы — это нестатические методы, помеченные `virtual` в базе. Вызов через указатель/ссылку на базу выбирает реализацию по динамическому типу объекта.

8.2.1. Зачем нужны виртуальные методы

Они обеспечивают динамический полиморфизм — ключевую возможность ООП, позволяющую:

- обращаться к объектам производных классов через интерфейс базового класса,
- не менять код при добавлении новых реализаций,
- использовать общую архитектуру для обработки разных объектов.

8.2.2. Синтаксис

```

1 class Base
2 {
3     public:
4         virtual void f();
5 };

```

- Ключевое слово `virtual` пишется в базовом классе.
- В производном классе рекомендуется писать `override`.

```

1 class Derived : public Base
2 {
3     public:
4         void f() override;
5 };

```

- С помощью `final` можно запретить дальнейшее переопределение метода (`virtual ... final`) или наследование класса (`class C final`).

8.2.3. Механизм работы

Если в классе определён хотя бы один виртуальный метод:

- Компилятор создаёт `vtable` — таблицу указателей на виртуальные функции (одна на класс).
- В каждый объект этого класса добавляется скрытое поле — указатель на `vtable` (обычно называется `vptr`).
- При вызове виртуального метода `p->f()` происходит не прямой вызов, а переход по таблице.

8.2.4. Пример

```
1 class Base
2 {
3     public:
4         virtual void f() { cout << "Base\n"; }
5 };
6
7 class Derived : public Base
8 {
9     public:
10        void f() override { cout << "Derived\n"; }
11 };
12
13 int main()
14 {
15     Base* ptr = new Derived;
16     ptr->f(); // Вызовется Derived::f()
17     delete ptr;
18 }
```

8.2.5. Особенности

- Метод, помеченный **virtual**, может быть переопределён.
- Если метод не переопределён в производном классе, вызовется версия из базового класса.
- Виртуальный метод может быть перегружен.
- Виртуальный метод не может быть:
 - **static**
 - **constexpr** (до C++20)
 - **inline** — не имеет смысла
- Для явного переопределения следует использовать **override**.

8.2.6. Зачем писать **override**

Если забыть **override**, можно случайно не переопределить метод, а скрыть его.

```
1 class A
2 {
3     public:
4         virtual void f(int);
5 };
6
7 class B : public A
8 {
9     public:
10        void f(double); // ошибка: не override, это новый метод!
11 };
```

8.2.7. Когда класс становится полиморфным

- Если в классе есть хотя бы один виртуальный метод — он полиморфный.
- Все производные от него классы тоже становятся полиморфными.

8.2.8. Почему важно удалять объекты через базовый указатель корректно

Если удалять объект производного класса через указатель на базу:

```
1 Base* p = new Derived;  
2 delete p; // должен быть виртуальный деструктор!
```

то без виртуального деструктора вызовется только `Base::~~Base()`, а `Derived::~~Derived()` — нет.

8.2.9. Примечания

- `vtable` — одна на класс.
- `vptr` — одно поле в каждом объекте.
- Сколько классов — столько таблиц.
- Не путать с перегрузкой функций и шаблонным полиморфизмом.

8.3. Чисто виртуальные методы

Чисто виртуальный метод — это метод, который не имеет реализации в базовом классе и обязан быть реализован в производном классе. Он задаёт интерфейс, но не поведение.

8.3.1. Синтаксис

```
1 class A  
2 {  
3     public:  
4         virtual void f() = 0;  
5 };
```

- `= 0` указывает, что метод чисто виртуальный.

8.4. Виртуальный деструктор

8.4.1. Зачем он нужен?

Если мы удаляем объект через указатель на базовый класс, важно, чтобы вызвался деструктор производного класса. Без виртуального деструктора это может не произойти — вызовется только деструктор базового класса, что приведёт к утечке ресурсов.

8.4.2. Пример:

```
1 class A  
2 {  
3     public:  
4         ~A() {};
```

```

5 };
6
7 class B : public A
8 {
9     public:
10     ~B() {};
11 };
12
13 int main()
14 {
15     A* p = new B;
16     delete p; // Вызовется только A::~~A() — утечка
17 }

```

Добавим **virtual**:

```

1 class A
2 {
3     public:
4     virtual ~A() = default;
5 };

```

Теперь **delete** p вызовет сначала B::~~B(), затем A::~~A() — корректное разрушение.

8.5. Чисто виртуальный деструктор

8.5.1. Когда нужен?

Он делает класс абстрактным, даже если больше нет чисто виртуальных методов. Удобно, когда нужно запретить создание базового, не создавая чисто виртуальных методов.

8.5.2. Пример объявления:

```

1 class A
2 {
3     public:
4     virtual ~A() = 0; // чисто виртуальный деструктор
5 };
6
7 A::~~A() = default; // всё равно нужно определить!

```

8.5.3. Почему нужно определение?

Даже если деструктор «чисто виртуальный», его всё равно вызывает компилятор при удалении объекта — он не может остаться неопределённым.

8.6. Понятие абстрактного класса

Абстрактный класс — это класс, в котором есть хотя бы одна чисто виртуальная функция. Такие классы нельзя инстанцировать напрямую, они служат для определения интерфейса и создания иерархий наследования.

```
1 class Shape
2 {
3     public:
4         virtual void draw() = 0; // чисто виртуальная функция
5 };
```

8.6.1. Зачем нужны абстрактные классы?

- Задают общий интерфейс для всех производных классов
- Позволяют использовать полиморфизм
- Помогают писать обобщённый код, не зависящий от конкретных реализаций

8.6.2. Особенности

- Нельзя создавать объекты такого класса, иначе — ошибка компиляции. Указатели/ссылки можно.
- Если сигнатура в наследнике не совпала (например, потеряли `const`, `&`, `noexcept`), то чистая функция не переопределена $\Rightarrow$ наследник остаётся абстрактным.

8.7. Ошибки, возникающие при работе с указателем или ссылкой на базовый класс

8.7.1. Сравнение указателей на базу и на производный класс

```
1 A* pa = nullptr;
2 B* pb = nullptr;
3 if (pa == pb)
```

Проблема

Если B унаследован от A, то B* и A* могут указывать на разные адреса (подобъект A может быть внутри B, но не в начале объекта).

Итог

pa == pb может быть ложью даже для одного и того же объекта. Это связано с внутренним размещением подобъекта в памяти.

8.7.2. Случайное копирование при передаче по значению

```
1 void f(A obj); // копирование!
2
3 B obj;
4 A& alias = obj;
5 f(alias); // копируется A — вызовется конструктор копирования!
```

Проблема

Мы вызываем `f`, которая принимает `A` по значению, но передаём объект класса `B`. Это приведёт к отсечению производной части (object slicing) и вызову конструктора копирования класса `A`.

Решение

- Передавать по ссылке: `void f(const A& obj)`
- Запретить копирование в `A`:

```
1 A(const A&) = delete;
```

- Или сделать конструктор `explicit`, чтобы ограничить случайные преобразования

8.7.3. Нельзя работать с массивами производных через указатель на базу

```
1 A& index(A* arr, int i)
2 {
3     return arr[i];
4 }
5
6 B* ar = new B[10];
7 A obj = index(ar, 2); // slicing!
```

Проблема

- `A obj = index(...)` — опять копирование → отсечение производной части
- Если `sizeof(A) != sizeof(B)`, то арифметика `arr[i]` становится невалидной
- Даже `A* ptr = ar; ptr[i]` — небезопасно, потому что:
 - Компилятор считает, что `ptr + i` — это `i`-ый объект типа `A`, а он лежит в памяти совсем по другой сетке смещений, чем `B`

8.8. Дружественные связи

Дружественная связь — это способ дать функции, методу или другому классу доступ к `private` и `protected` членам класса, минуя обычные ограничения инкапсуляции.

```
1 class C
2 {
3     int x;
4
5     public:
6     C(int val) : x(val) {}
7     friend void print(C& obj); // функция-друг
8 };
9
10 void print(C& obj)
11 {
12     std::cout << obj.x; // доступ к private
13 }
```

8.8.1. Кто может быть другом?

```
1 class C
2 {
3     friend void f(C&);           // функция
4     friend void B::g();          // метод другого класса
5     friend class A;              // весь класс
6 };
```

8.8.2. Виртуальный метод как друг

```
1 class Base
2 {
3     protected:
4         int secret = 42;
5
6     public:
7         virtual void expose() = 0;
8 };
9
10 class Friend
11 {
12     public:
13         void call(Base& b) { b.expose(); } // вызывает виртуальный метод
14 };
```

Теперь в производном классе:

```
1 class Derived : public Base
2 {
3     public:
4         void expose() override
5         {
6             std::cout << secret; // доступ есть – это поле базового класса
7         }
8
9         friend void Friend::call(Base&); // метод `Friend::call` – друг
10 };
```

Что здесь происходит?

- Метод `Friend::call` является другом класса `Derived`
- При этом он не друг `Base`
- Однако `call()` вызывает виртуальный метод `expose()`, который у `Derived` переопределён
- Поскольку метод `call()` — друг `Derived`, то он имеет право вызывать `Derived::expose()`, а та уже имеет доступ к `secret` через `protected` доступ

- То есть `call()` не видит `secret` напрямую, но он «переходит через виртуальность», и фактически через доверенного «посредника» — виртуальный метод — получает доступ к закрытым данным объекта

8.8.3. Важные свойства дружбы

Свойство	Комментарий
Не наследуется	Если А — друг В, то <code>DerivedA</code> не будет другом В
Не транзитивна	Если А — друг В, а В — друг С, это не значит, что А — друг С
Декларируется только в том классе, где разрешён доступ	Не в «другом», а именно в «приглашающем»
Работает на уровне функций	Даже если <code>friend</code> — метод класса, сам класс не становится другом
Доступ даётся ко всем членам	Нет ограничения только к конкретному полю — даётся ко всему <code>private</code> и <code>protected</code>

8.8.4. Когда использовать?

Желательно избегать, но...

- Иногда оправдано, если:
 - Нужно реализовать внешние операторы (`operator<<`, `operator==`)
 - Есть вспомогательный класс, который тесно связан с основным
- Иногда удобно для взаимодействия двух классов, если один — обёртка, другой — ядро логики

9. Обработка исключительных ситуаций в C++. Решение проблем структурного программирования. Блоки `try` и `catch`. Блоки `try` и `catch` методов и конструкторов. Безопасный код относительно исключений. Обертывание исключения в `exception_ptr`. Задачи которые может решать исключение. Проблемы с динамической памятью при обработке исключительных ситуаций.

9.1. Проблемы структурного программирования

1) Протаскивание ошибок

При возникновении ошибки мы вынуждены протаскивать её (`int rc/bool ok`) через уровни абстракции до тех пор, пока не сможем обработать её.

2) Смещение нормальной логики и обработки сбоев.

Читаемость падает, растёт количество ветвлений. Весь код насыщен проверками на ошибки.

3) Потеря ошибок.

Легко забыть проверить код возврата.

4) Сложный откат (`cleanup`).

При раннем выходе нужно вручную закрывать уже открытые ресурсы $\Rightarrow$ дублирование кода или `goto cleanup`.

5) Слабая типизация ошибок.

`-1` — это что? Недоступен файл? Плохие права? Нехватка памяти?

9.1.1. Что меняют исключения

— Единый канал распространения ошибки.

`throw` поднимает ошибку вверх к ближайшему подходящему `catch` без протаскивания кодов.

— Автоматическая очистка через развёртывание стека.

Деструкторы локальных объектов вызываются гарантированно $\Rightarrow$ RAII работает и при ошибках.

— Разделение положительных и негативных путей.

Основная логика — прямолинейная; обработка — в `catch` или выше.

— Богатая типизация.

Имена типов исключений несут семантику (`std::out_of_range`, `std::bad_alloc`, свои типы).

9.2. Обработка исключительных ситуаций в C++. Блоки `try` и `catch`

— Реализуется при помощи операторов `throw`, `try` и `catch`

9.2.1. Механизм работы

— Оператор `try` обозначает участок кода, в котором может быть выброшено исключение

- За блоком `try` может следовать один или несколько блоков `catch` для разных типов исключений
- `throw` `expr`; создаёт объект исключения (копия/перемещение из `expr`) и запускает развёртывание стека: локальные объекты по пути уничтожаются их деструкторами.
- Идёт поиск ближайшего подходящего `catch`. Подходящим считается:
 - точное совпадение типа,
 - либо базовый класс по ссылке/указателю,
 - либо `catch(...)` (ловит любые типы, но используем осторожно).
- Порядок `catch` важен: от более конкретного к более общему (иначе «съешь» производные типы базовым обработчиком).
- Ловим по ссылке: `catch (const MyError& e)`. Ловля по значению может «порезать» (slicing) производные поля, а ещё дороже по копиям.
- Повторный бросок:
 - `throw`; (внутри `catch`) — перекинуть ровно то же исключение, без копии;
 - `throw e`; — создаст новую копию (и может привести к slicing, если `e` — базовый тип).
- Если подходящего обработчика нет — вызывается `std::terminate`

9.2.2. Пример использования

```

1 try {
2     // потенциально опасные операции
3     if (!openFile(filename)) {
4         throw std::runtime_error("Не удалось открыть файл");
5     }
6     processData();
7 } catch (const std::runtime_error& e) {
8     std::cerr << "Ошибка времени выполнения: " << e.what() << std::endl;
9 } catch (const std::exception& e) {
10    std::cerr << "Стандартное исключение: " << e.what() << std::endl;
11 } catch (...) {
12    std::cerr << "Неизвестная ошибка" << std::endl;
13 }
```

9.2.3. Иерархия стандартных исключений

Класс	Наследник	Назначение
<code>std::exception</code>	<code>std::logic_error</code>	ошибки логики программы
<code>std::exception</code>	<code>std::runtime_error</code>	ошибки времени выполнения
<code>std::logic_error</code>	<code>std::invalid_argument</code>	некорректный аргумент функции
<code>std::runtime_error</code>	<code>std::out_of_range</code>	выход за границы диапазона

9.2.4. Пользовательские исключения

- Рекомендуется наследовать от `std::exception` или его потомков
- Рекомендуется переопределить метод `what()`, чтобы вернуть понятное описание ошибки
- Рекомендуется для каждой исключительной ситуации создавать свой класс

9.3. Блоки `try` и `catch` методов и конструкторов

- `function-try-block`: позволяет обрабатывать исключения, выброшенные в теле метода или при инициализации членов/базовых классов
- Синтаксис: после сигнатуры метода или конструктора ставится `try`, затем тело и блоки `catch`
- При выбросе исключения сначала разрушаются уже сконструированные члены, затем выполняется `catch`
- В блоке `catch` можно выполнить дополнительную очистку и затем перебросить исключение через `throw`;

9.3.1. Пример `function-try-block` для метода

```
1 void MyClass::doWork() try
2 {
3     performTask(); // потенциально выбрасывающий код
4 }
5 catch (const std::exception& e)
6 {
7     logError(e.what()); // логирование и очистка
8     throw;              // при необходимости проброс дальше
9 }
```

9.3.2. Пример `function-try-block` для конструктора

```
1 MyClass::MyClass(int n) try
2 : size(n), data(new int[n])
3 {
4     if (n <= 0) {
5         throw std::invalid_argument("n должно быть > 0");
6     }
7 }
8 catch (const std::exception& e)
9 {
10    // освобождение ресурсов, выделенных до исключения
11    delete[] data;
12    std::cerr << "Ошибка в конструкторе: " << e.what() << std::endl;
13    // объект не считается сконструированным
14    throw;
15 }
```

9.4. Безопасный код относительно исключений

- Цель: обеспечить предсказуемое поведение и отсутствие утечек при выбросе исключений
- Основывается на трёх уровнях гарантии: базовая, строгая и отсутствие исключений

9.4.1. Basic гарантия

Если метод бросит исключение, то объект останется валидным, но может частично измениться.

```
1 class Bag
2 {
3     std::vector<int> v;
4     public:
5     void add_all_basic(const std::vector<int>& more) {
6         // если бросит — часть уже добавлена
7         for (int x : more) v.push_back(x);
8     }
9 };
```

Если `push_back` кинет (например, `bad_alloc`), инварианты `v` сохранены, но элементы могли добавиться частично.

9.4.2. Strong гарантия

Либо полный успех, либо откат всех изменений.

```
1 class Bag2
2 {
3     std::vector<int> v;
4     public:
5     void add_all_strong(const std::vector<int>& more) {
6         // если копия бросит — v не тронут
7         std::vector<int> tmp = v;
8         // заранее просим память
9         tmp.reserve(v.size() + more.size());
10        // если бросит — v не тронут
11        for (int x : more) tmp.push_back(x);
12        // commit: noexcept
13        v.swap(tmp);
14    }
15 };
```

Все рискованные шаги — на «черновике» `tmp`. Если успех — безопасный `swap`.

9.4.3. **noexcept** гарантия

- Функции с **гарантией отсутствия исключений** никогда не бросают исключений (например, все операции над встроенными типами)
- Нарушение этой гарантии приводит к вызову `std::terminate`
- Пустая спецификация **noexcept** не всегда гарантирует отсутствие выброса — это лишь обещание, нарушение которого завершает программу

9.5. Обёртывание исключения в `exception_ptr`

- `std::exception_ptr` хранит ссылку на текущий объект исключения

- Позволяет сохранить исключение для последующего повторного выброса в более подходящем месте

9.5.1. Получение `exception_ptr`

```
1 std::exception_ptr captureException()
2 {
3     try {
4         riskyOperation();
5     } catch (...) {
6         return std::current_exception(); // сохраняем текущее исключение
7     }
8     return nullptr;
9 }
```

9.5.2. Повторный выброс

```
1 void wrapper()
2 {
3     auto ex = captureException();
4     if (ex) {
5         std::rethrow_exception(ex); // кидаем сохранённое исключение
6     }
7 }
```

9.5.3. Особенности

- `std::current_exception()` не копирует стек, а создаёт обёртку
- Управление временем жизни исключения возлагается на стандартную библиотеку, утечек не возникает

9.5.4. Обёртывание исключения в `exception_ptr` с разделением метода на части

- На методы класса накладывается важная ответственность: они не должны после своего выполнения нарушить целостность объекта
- Метод разбивается на три логические части, чтобы обеспечить безопасность при изменении состояния объекта и отложенный выброс исключения

Часть	Описание	Разрешено выбрасывать
Начало	Объект ещё не изменён; допускается выброс исключений	Да
Модификация	Изменение состояния объекта; выбрасывать исключения нельзя	Нет
Завершение	После модификации; можно пробросить сохранённое исключение	Да

Пример реализации

```
1 void MyClass::update()
```

```

2 {
3     std::exception_ptr ex;
4
5     // Часть 1: начало – ещё не изменяем состояние
6     try {
7         prepare(); // может выбросить
8     } catch (...) {
9         throw; // пробрасываем сразу, если ошибка на подготовке
10    }
11
12    // Часть 2: модификация – состояние объекта меняется
13    try {
14        modifyState(); // изменяет поля, не должен выбрасывать
15    } catch (...) {
16        // сохраняем исключение, не выбрасываем
17        ex = std::current_exception();
18    }
19
20    // Часть 3: завершение – после изменений можно выбросить сохранённое
21    if (ex) {
22        // повторно кидаем обёрнутое исключение
23        std::rethrow_exception(ex);
24    }
25
26    finalize(); // завершающие операции, тоже могут выбрасывать
27 }

```

9.6. Задачи, которые может решать исключение

- Протаскивание ошибок на уровень, где их удобно обрабатывать
- Отделение логики обработки ошибок от основной бизнес-логики
- Автоматический откат или освобождение ресурсов через развёртывание стека (RAII)
- Перенаправление ошибок (например, сбор нескольких ошибок перед броском)
- Сигнализация о критических ситуациях при инициализации или разрушении объектов
- Упрощение кода при работе с вложенными вызовами и множественными уровнями функций
- Централизованное логирование и анализ ошибок в едином обработчике

9.7. Проблемы с динамической памятью при обработке исключительных ситуаций

- Вручную управляемая память подвержена утечкам при неожиданном `throw`
- Риск двойного освобождения при очистке в `catch` и затем в деструкторе
- Исключения в конструкторах могут прервать инициализацию, оставив ресурсы не освобождёнными

9.7.1. Пример утечки при исключении

```
1 int* p = new int[100];
2 process(p); // может выбросить
3 delete[] p; // недостижимо при исключении → утечка памяти
```

Решение через RAII

```
1 auto p = std::make_unique<int[]>(100);
2 process(p.get()); // при исключении память автоматически освободится
```

9.7.2. Исключение в разделе инициализации конструктора

- Если в списке инициализации возникает исключение, тело конструктора не выполняется и без `function-try-block` поймать его нельзя
- Применяют `function-try-block` вокруг всего конструктора (включая список инициализации)

```
1 Array::Array(size_t n) try
2 : data(new int[n]), size(n)
3 {
4     if (n == 0) {
5         throw std::invalid_argument("n не должно быть нулевым");
6     }
7 }
8 catch (const std::exception& e)
9 {
10     delete[] data; // очистить ресурсы, выделенные в списке инициализации
11     std::cerr << "Ошибка при инициализации: " << e.what() << std::endl;
12     throw;        // объект не считается сконструированным
13 }
```

9.7.3. Исключения в деструкторе

- Если во время развёртывания стека уже летит одно исключение, и деструктор бросит второе, произойдёт двойное исключение $\Rightarrow$ `std::terminate()`.
- По стандарту деструкторы по умолчанию `noexcept` (условно — если их подчасти не бросают). Если из них «утечёт» исключение $\Rightarrow$ `std::terminate()`.
- Практика: ловить всё внутри деструктора, не выбрасывать наружу. Если нужно сообщить об ошибке — лог/флаг.

```
1 MyClass::~~MyClass()
2 {
3     try {
4         cleanup(); // может бросить
5     } catch (const std::exception& e) {
6         std::cerr << "Ошибка в деструкторе: " << e.what() << std::endl;
7         // не пробрасываем, чтобы не вызывать terminate()
8     }
```


10. Перегрузка операторов в C++. Операторы .*, ->*. Правило перегрузки операторов. Перегрузка операторов =, () и []. Перегрузка операторов ->, * и ->*.

10.1. Перегрузка операторов в C++

Перегрузка операторов в C++ — это механизм, позволяющий дать объектам пользовательских классов поведение, аналогичное встроенным типам, при работе со стандартными операторами. Это делает код более выразительным и интуитивно понятным.

10.1.1. Какие операторы можно перегружать:

- 1) Арифметические: `+` `-` `*` `/` `%`
- 2) Инкремент и декремент: `++` `--`
- 3) Побитовые: `&` `|` `^` `<<` `>>`
- 4) Логические: `!` (унарный), `&&`, `||`
- 5) Сравнения: `==` `!=` `<` `>` `<=` `>=`
- 6) Присваивания: `=` `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=`
- 7) Другие: `[]`, `()`, `->`, `*` (разыменование), `->*`, `,`, `new`, `delete` и др.

10.1.2. Нельзя перегрузить

Оператор	Краткое описание
<code>::</code>	Разрешение области видимости
<code>.</code>	Прямой доступ к члену объекта
<code>.*</code>	Доступ к члену объекта через указатель на член
<code>?:</code>	Тернарный условный оператор, выбирающий значение в зависимости от условия
<code>sizeof</code>	Определяет размер объекта или типа в байтах
<code>typeid</code>	Оператор для получения информации о типе во время выполнения

10.1.3. Способы объявления

Как метод-член класса

```
1 class Vector
2 {
3     public:
4         Vector operator+(const Vector& other) const
5         {
6             // реализация сложения компонент
7         }
8 };
```

— При таком объявлении левый операнд — `*this`, правый — параметр метода.

Как свободная функция

```
1 Vector operator+(const Vector& lhs, const Vector& rhs)
2 {
3     // реализация сложения компонент
4 }
```

— Оба операнда передаются явно в параметры функции.

Особенности:

- Методы-члены неявно получают доступ к приватным полям класса.
- Свободные функции рекомендуется объявлять `friend`, если им нужен доступ к приватным членам.
- Возвращаемый тип обычно — новый объект; в редких случаях может быть ссылка или rvalue.

10.2. Операторы `.*`, `->*`

```
1 class S
2 {
3     public:
4         int x = 0;
5         int f(int a) { return x + a; }
6         int g(int a) const { return a - x; }
7 };
8
9 int S::* pX = &S::x;
10 int (S::* pF)(int) = &S::f;
11 int (S::* pG)(int) const = &S::g;
12
13 S s;   S* ps = &s;
14
15 s.*pX = 7;           // доступ к полю через .*
16 (ps->*pX) = 9;       // доступ к полю через ->*
17
18 int r1 = (s.*pF)(3); // вызов метода через .*
19 int r2 = (ps->*pF)(4);
20 int r3 = (s.*pG)(5); // у const-метода тип указателя другой (с const)
```

10.3. Правила перегрузки операторов

1) Нельзя менять приоритет, ассоциативность и число операндов

- Перегружается только существующий оператор с его стандартным знаком и арностью

2) Нельзя вводить новые операторы

- Только те, что определены в языке

3) Место объявления и сигнатура

- `operator=`, `operator[]`, `operator()`, `operator->` — только методы-члены

- Унарные операторы-члены — без параметров
 - Бинарные операторы-члены — один параметр
 - Свободные функции (не-члены) — два параметра, оба перечисляются явно
- 4) **Оператор-член не может быть `static`**
- 5) **Наличие пользовательского типа**
- Хотя бы один операнд должен быть типом класса или перечислением, иначе перегрузка не применяется
- 6) **Возвращаемые типы**
- `operator=` возвращает `Class&`
 - Префиксные `++/--` возвращают `Class&`; постфиксные — `Class` (с фиктивным `int` в параметрах)
 - Операторы сравнения возвращают `bool`
 - Арифметические и др. обычно возвращают новый объект (значение) или ссылку, если требуется
- 7) **Семантические рекомендации**
- Поведение должно быть логичным и соответствовать исходному смыслу оператора
 - Избегать неожиданных побочных эффектов и неочевидных преобразований

10.4. Перегрузка операторов `=`, `()`, `[]`

10.4.1. `operator=`

- Объявляется как метод-член: `Class& operator=(const Class& other)`
- Копирующий: `T& operator=(const T& rhs);`
- Перемещающий: `T& operator=(T&& rhs) noexcept(/*если все поля noexcept*/);`

Правила

- 1) Всегда объявляется как член класса
- 2) Обязательно обрабатывать самоприсваивание:

```
1 if (this == &other) return *this;
```

- 3) Может быть объявлен как `= default` или `= delete`

Пример

```
1 class Buf
2 {
3     std::size_t size_ = 0;
4     char* ptr_ = nullptr;
5 public:
6     Buf() = default;
7     explicit Buf(std::size_t m): size_(m), ptr_(m ? new char[m] : nullptr)
8 {}
9
10    ~Buf() { delete[] p; }
11
12    // Копирующий конструктор
13    Buf(const Buf& other): Buf(other.size_)
```

```

12     {
13         std::copy(other.ptr_, other.ptr_ + size_, ptr_);
14     }
15
16     // Копирующее присваивание с проверкой
17     Buf& operator=(const Buf& rhs)
18     {
19         if (this == &rhs) return *this; // защита от a = a
20         delete[] ptr_;
21         size_ = rhs.size_;
22         ptr_ = size_ ? new char[size_] : nullptr;
23         std::copy(rhs.ptr_, rhs.ptr_ + size_, ptr_);
24         return *this;
25     }
26 };

```

- Нужно `if (this == &rhs)`, иначе при `a = a` мы бы удалили собственные данные и скопировали из освобождённой памяти.

10.4.2. `operator()`

- Объявляется как член класса с произвольной сигнатурой:
 - `ReturnType operator()(ParamType1, ParamType2, ...)`
- Назначение: превращение объекта в функциональный объект (функтор)
- Можно перегружать с разным числом и типами параметров

```

1 class Multiplier
2 {
3     int factor;
4     public:
5     Multiplier(int f) : factor(f) {}
6     int operator()(int x) const { return factor * x; }
7 };

```

10.4.3. `operator[]`

- Объявляется как член класса в двух вариантах:
 - 1) `T& operator[](size_t index)` (для `b[i] = 42;`)
 - 2) `const T& operator[](size_t index) const`
- Назначение: доступ к элементу по индексу, возвращает ссылку для чтения и записи

Особенности

- 1) Ожидается, что `operator[]` не выполняет проверку диапазона (для этого используют метод `at()`)
- 2) Возвращаемая ссылка позволяет менять значение элемента

Пример

```

1 class Vec

```

```

2 {
3     int* data;
4     size_t size;
5
6     public:
7     Vec(size_t n) : data(new int[n]), size(n) {}
8     ~Vec() { delete[] data; }
9     int& operator[](size_t i) { return data[i]; }
10    const int& operator[](size_t i) const { return data[i]; }
11 };

```

10.5. Перегрузка операторов `->`, `*`, `->*`

10.5.1. `operator->`

- Объявляется как член класса `ReturnType* operator->()`
- Назначение: перенаправить доступ к членам внутреннего объекта
- Возвращает либо «сырой» указатель `T*`, либо объект, у которого снова есть `operator->` (цепочка продолжается, пока не получится указатель)
- Широко используется в умных указателях и прокси-классах

Пример

```

1 struct Obj
2 {
3     void foo() { }
4 };
5 class Smart
6 {
7     Obj* ptr;
8     public:
9     explicit Smart(Obj* p) : ptr(p) {}
10    Obj* operator->() { return ptr; }
11    const Obj* operator->() const { return ptr; }
12 };
13
14 int main()
15 {
16     Obj o;
17     Smart s{&o};
18     s->foo(); // эквивалентно o.foo()
19 }

```

10.5.2. `operator*`

- Объявляется как член класса `ReturnType& operator*()`
- Назначение: имитировать разыменование указателя, возвращая ссылку на объект

Пример

```
1 struct Obj
2 {
3     void foo() { }
4 };
5 class Smart
6 {
7     Obj* ptr;
8     public:
9     explicit Smart(Obj* p) : ptr(p) {}
10    Obj& operator*() { return *ptr; }
11    const Obj& operator*() const { return *ptr; }
12 };
13
14 int main()
15 {
16     Obj o;
17     Smart s{&o};
18     (*s).foo(); // вызывает Obj::foo()
19 }
```

10.5.3. operator->*

- Объявляется как член класса MemberResult `operator->*(MemberPtr mp)`
- `->*` применяет указатель на член класса к указателю на объект. Это «стрелочная версия» для `.*`, который работает с самим объектом
- Типы вида `M T::*` — не обычные `M*` и не взаимозаменяемы

Пример

```
1 struct Person
2 {
3     int age = 0;
4     std::string name = "Anon";
5 };
6
7 // Указателеподобная обёртка
8 class PersonPtr
9 {
10    Person* p;
11    public:
12    explicit PersonPtr(Person* q) : p(q) {}
13
14    // Перегрузки под разные указатели-на-поле
15    int& operator->*(int Person::* pm) noexcept
16    {
17        return p->*pm;
18    }
```

```

19     const int& operator->*(int Person::* pm) const noexcept
20     {
21         return p->*pm;
22     }
23
24     std::string& operator->*(std::string Person::* pm) noexcept
25     {
26         return p->*pm;
27     }
28     const std::string& operator->*(std::string Person::* pm) const noexcept
29     {
30         return p->*pm;
31     }
32 };
33
34 int main()
35 {
36     Person ivan{30, "Ivan"};
37     PersonPtr h(&ivan);
38
39     int Person::* pAge = &Person::age;
40     std::string Person::* pName = &Person::name;
41
42     h->*pAge = 31; // int& → присваиваем
43     (h->*pName).append("Petrov"); // std::string& → вызываем методы
44
45     // то же самое напрямую:
46     ivan.*pAge = 32;
47 }

```

11. Перегрузка унарных и бинарных операторов. Проблемы с перегрузкой операторов `&&`, `||`, `,` и `&`. Перегрузка операторов `++`, `--`. Перегрузка операторов приведения типов. Тривалентный оператор `spaceship`.

11.1. Перегрузка унарных и бинарных операторов

11.1.1. Что такое унарные и бинарные операторы?

- Унарные операторы принимают один операнд.
 - Примеры: `+`, `-`, `!`, `~`, `++`, `--`, `*` (разыменование), `&` (взятие адреса).
 - Обычно удобнее как члены — они меняют/читают состояние объекта через `*this`
- Бинарные операторы принимают два операнда.
 - Примеры: `+`, `-`, `*`, `/`, `==`, `<`, `&&`, `,` и т.д.
 - Бинарные «симметричные» операторы — чаще свободные (иногда `friend`) функции
 - Так будут работать неявные преобразования не только справа, но и слева
 - Операции будут коммутативными: $a + b = b + a$

11.1.2. Основные принципы перегрузки

- Перегрузка операторов в C++ позволяет задать поведение операторов для пользовательских типов (например, для класса `Vector`, `Complex`, `Matrix` и т.п.)
- Перегрузка не должна менять:
 - Приоритет оператора
 - Ассоциативность
 - Арность (унарный оператор не станет бинарным)
- Хотя бы один операнд должен быть объектом пользовательского типа
- Перегружаются только существующие операторы, новые вводить нельзя

11.1.3. Как задаётся перегрузка

Через ключевое слово `operator`

```
1 class A
2 {
3     int x;
4     public:
5     A(int val) : x(val) {}
6
7     // Унарный плюс
8     A operator+() const { return *this; }
9
10    // Унарный минус
11    A operator-() const { return A(-x); }
12
13    // Бинарный плюс
14    A operator+(const A& other) const { return A(x + other.x); }
```

```
15 };
```

11.2. Проблемы с перегрузкой операторов `&&`, `||`, `,` и `&`

Некоторые операторы в C++ обладают особым поведением, которое теряется при перегрузке

11.2.1. `&&` и `||`

— Обычное поведение:

```
1 if (f() && g()) // g() вызовется только если f() == true
```

— При перегрузке:

```
1 A a, b;  
2 a && b; // вызовется operator&&(A const&, A const&)
```

Проблемы

- Это уже вызов функции, а не встроенный оператор
- Оба операнда будут вычислены до вызова, их порядок не определён
- Нет возможности реализовать «ленивую» логику.
- Это может нарушить семантику и привести к побочным эффектам, которые программист не ожидает.

Вывод

Перегружать эти операторы следует только в крайних случаях.

11.2.2. `,` (запятая)

— Обычное поведение:

- Выражения вычисляются строго слева направо
- Возвращается значение правого операнда

```
1 int a = (f(), g()); // возвращает g(), выполняет f() и g() слева направо
```

— При перегрузке:

```
1 A a1, a2;  
2 A a3 = a1, a2; // вызовется operator,(A const&, A const&)
```

Проблемы

- Снова вызов функции, а не встроенный оператор
- Оба операнда будут вычислены до вызова, их порядок не определён
- При перегрузке лучше возвращать правый операнд, если хотим сохранить «похожую» семантику.

11.2.3. `&` (взятие адреса)

— Обычное поведение:

```
1 A a;  
2 A* p = &a; // Получили адрес объекта
```

- Так как работать с адресами объектов опасно, в C++ позволили перегружать этот оператор.
- Этот оператор у класса можно удалить, таким образом запретив получение адреса.

Проблема

- Нарушает базовую семантику языка.
- Может помешать взять реальный адрес объекта.
- Вызовет непредсказуемое поведение при работе с `&obj`.

Решение

- Если перегрузили `operator&`, но нужен настоящий адрес — используем `std::addressof(obj)`.

11.3. Перегрузка операторов `++`, `--`

- Это унарные операторы, работают с одним операндом
- Бывают префиксные и постфиксные формы
 - `++a` (префикс)
 - `a++` (постфикс)

11.3.1. Отличие в семантике

- Префиксный `++a` увеличивает значение, возвращает новое значение
- Постфиксный `a++` увеличивает значение, возвращает старое значение (для этого сохраняется копия старого)
 - Поэтому префиксный `++` обычно эффективнее

11.3.2. Перегрузка в классе

```
1 class Counter  
2 {  
3     int value;  
4  
5     public:  
6     Counter(int v) : value(v) {}  
7  
8     // Префиксный ++  
9     Counter& operator++()  
10    {  
11        ++value;  
12        return *this;  
13    }  
14  
15    // Постфиксный ++  
16    Counter operator++(int)  
17    {
```

```

18     Counter temp = *this;
19     ++value;
20     return temp;
21 }
22 };

```

11.3.3. Тоже самое для --

```

1 Counter& operator--()    // префикс
2 Counter  operator--(int) // постфикс

```

11.3.4. Пояснения к постфиксной форме

- В постфиксной форме `x++` компилятор ищет перегрузку `operator++(int)`.
- Фиктивный `int` — просто метка, чтобы отличить постфикс от префикса на уровне перегрузок.
- Аргумент никто не передаёт вручную: компилятор сам подставляет `0` при разборе `x++`. Значение не используется.
- Тип именно `int` — так предписано стандартом для сигнатуры постфиксной формы. Менять на другой тип — дурной тон и может вести к странному выбору перегрузок.
- Параметр обычно безымянный или помечается `[[maybe_unused]]`, чтобы не было предупреждений от компилятора.

11.4. Перегрузка операторов приведения типов

11.4.1. Суть

В C++ можно определить, как объект пользовательского типа будет преобразовываться к другому типу, в том числе — встроенному (`int`, `double`, `bool`, `std::string`, и др.)

Для этого используется перегрузка специального оператора как член класса:

```

1 explicit operator тип() const;

```

Пример

```

1 class A
2 {
3     int value;
4
5     public:
6     A(int v) : value(v) {}
7
8     // Приведение к int
9     operator int() const { return value; }
10 };
11
12 int main()
13 {

```

```

14     A obj(42);
15     int x = obj; // вызовется operator int()
16 }

```

11.4.2. Проблема: неявные преобразования

- Такой оператор можно вызывать неявно, что может привести к неожиданным ситуациям.

```

1 A a(1);
2 if (a) { ... } // неявно вызывает operator int() => может вести к
  ошибкам

```

- Компилятор не найдёт `operator bool()`, поэтому будет использовать `operator int()`, чтобы потом неявно привести `int` → `bool`.
- `int(a)` — такая запись похожа на вызов конструктора, можно получить неоднозначность.

Решение

```

1 class A
2 {
3     int value;
4
5     public:
6         explicit operator int() const { return value; }
7 };
8
9 int x = static_cast<int>(a); // OK
10 int y = a; // Ошибка: требуется явное преобразование

```

11.5. Тривалентный оператор spaceship

- Оператор `<=>` — трёхсторонний оператор сравнения, он появился в C++20.
- Его задача — автоматически реализовать все сравнения (`<`, `<=`, `>`, `>=`)
 - Для `==`/`!=` всё ещё нужен `operator==` (обычно = `default`), иначе равенство не появится.
 - Например, регистронезависимый порядок может сказать, что `"Ab"` и `"ab"` эквивалентны (`weak_ordering`), но `==` при этом — `false`. Плюс равенство может быть дорогим/нестандартным, поэтому стандарт не синтезирует `==` из `<=>`.

11.5.1. Виды порядков (тип возвращаемого результата)

Компилятор сам выводит «наиболее слабую достаточную» категорию из того, что сравниваем.

`std::strong_ordering`, строгий порядок

- Трёхчленность: ровно одно из $\{a < b, a = b, a > b\}$ истинно. Никаких «эквивалентны, но не равны».
- Пример типов: `int`, `char`, `std::string`.

— Поведение:

```
1 4 < 7 // true
2 "ab" < "ac" // true
3 Point{1,2} < Point{1,3} // true (при default-сравнении по полям int)
```

— Эквивалентности нет: если «не меньше и не больше», то равны, и `==` (если его определили) обязан это подтверждать.

`std::weak_ordering`, слабый порядок

— Есть «классы эквивалентности»: `a` и `b` могут считаться эквивалентными (ни меньше, ни больше), но при этом `a == b` может быть `false` (если мы так определили равенство).

```
1 Ci{"Ab"} <=> Ci{"ab"} // weak_ordering::equivalent
2 Ci{"Ab"} < Ci{"ab"} // false
3 Ci{"Ab"} > Ci{"ab"} // false
4 // а вот равенство — как мы реализуем operator==:
5 Ci{"Ab"} == Ci{"ab"} // может быть false (если == регистрозависимый)
```

`std::partial_ordering`, частичный порядок

Есть пары, которые не сравнимы (`unordered`). Тогда все `<`, `<=`, `>`, `>=` — `false`.

Тип порядка	Значение	Смысл	Пример сравниваемых данных
<code>std::partial_ordering</code>	<code>equivalent</code>	Равны, если сравнимы	<code>3.14 <=> 3.14</code> → <code>equivalent</code>
<code>std::partial_ordering</code>	<code>less</code>	Меньше	<code>2.0 <=> 4.5</code> → <code>less</code>
<code>std::partial_ordering</code>	<code>greater</code>	Больше	<code>9.81 <=> 3.0</code> → <code>greater</code>
<code>std::partial_ordering</code>	<code>unordered</code>	Не поддаются сравнению	<code>NaN <=> 1.0</code> → <code>unordered</code>

11.5.2. Поведение по умолчанию

— Порядок: сначала сравниваются базовые классы (в порядке перечисления в объявлении), затем поля в порядке объявления.

11.5.3. Примеры интерпретации выражений

```
1 a < b
```

преобразуется в:

```
1 (a <=> b) < 0
```

```
1 a < 2; // → (a <=> 2) < 0
```

```

2 a == 2; // → (a <=> 2) == 0
3 a >= 2; // → (a <=> 2) >= 0

```

11.5.4. Способы определения

С помощью `= default`

```

1 #include <compare> // нужно для типов результата: strong_ordering и др.
2
3 class Point
4 {
5     int x, y;
6 public:
7     // членам и базам тоже нужен ==
8     bool operator==(const Point&) const = default;
9     // вернёт std::strong_ordering
10    auto operator<=>(const Point&) const = default;
11 };
12
13
14 class P
15 {
16     int i;
17     double d;
18 public:
19     bool operator==(const P&) const = default;
20     // partial_ordering из-за double
21     auto operator<=>(const P&) const = default;
22 };

```

Вручную

```

1 #include <compare>
2
3 struct Pair
4 {
5     int x, y;
6
7     bool operator==(const Pair& r) const noexcept
8     {
9         return x == r.x && y == r.y;
10    }
11
12    std::strong_ordering operator<=>(const Pair& r) const noexcept
13    {
14        if (x < r.x) return std::strong_ordering::less;
15        if (x > r.x) return std::strong_ordering::greater;
16        if (y < r.y) return std::strong_ordering::less;
17        if (y > r.y) return std::strong_ordering::greater;

```

```
18         return std::strong_ordering::equal; // оба поля равны
19     }
20 };
```

12. Шаблоны функций, методов классов и классов в C++. Недостатки шаблонов. Параметры шаблонов. Параметры типы и параметры значения. Шаблоны функций и методов классов. Подстановка параметров в шаблон. Выведение типов параметров шаблона. Явное указание значений типов параметров шаблона при вызове функции. Срезание ссылок и модификатора `const`.

12.1. Шаблоны функций, методов класса и классов в C++

Шаблон — это способ описания универсального кода, который может работать с разными типами данных. Компилятор на основании конкретных типов, указанных при вызове шаблона, создаёт конкретные реализации функций или классов (процесс называется **инстанцированием**).

12.1.1. Отличия от макросов

- Макросы выполняются препроцессором (до компиляции), шаблоны — компилятором.
- Шаблоны типобезопасны, макросы — нет.
- Шаблоны умеют проверять типы, генерировать разные варианты кода, быть перегруженными, специализироваться.

12.2. Недостатки шаблонов

12.2.1. Ошибки выявляются только при инстанцировании

Шаблон сам по себе не компилируется, если не используется. Проверка на корректность типов и выражений происходит только при конкретной подстановке типов. Это приводит к:

- большим спискам ошибок, которые сложно интерпретировать;
- невозможности заранее проверить корректность шаблона;
- неочевидным ошибкам (например, отсутствие нужного оператора у типа).

```
1 template <typename T>
2 void f(T x)
3 {
4     x + x; // если T не поддерживает operator+, будет ошибка
5 }
```

12.2.2. Увеличение времени компиляции

Каждая подстановка типа в шаблон генерирует новую версию функции или класса, что:

- увеличивает время компиляции;
- увеличивает размер бинарного файла.

12.2.3. Отсутствие изоляции в интерфейсе

Шаблон не может «заранее» сказать, что ему нужен тип с определённым интерфейсом. До C++20 не было официального способа задать ограничения (требования к типам) — только SFINAE или `static_assert`.

SFINAE (Substitution Failure Is Not An Error) — механизм, при котором ошибки, возникшие при подстановке типов в шаблон, не считаются фатальными, и компилятор просто исключает неподходящий шаблон из выбора, продолжая искать другие подходящие перегрузки.

12.2.4. Необходимость размещать реализацию в заголовочном файле

Из-за отложенной компиляции шаблонов, реализация должна быть в `.h/.hpp` файле. Иначе другая единица трансляции не сможет его инстанцировать.

12.2.5. Сложность диагностики ошибок

Ошибки при подстановке типов в шаблоны могут быть громоздкими и запутанными, особенно при использовании вложенных шаблонов или SFINAE.

12.2.6. Нет ограничения на допустимые типы (до C++20)

До введения `concepts` в C++20, шаблоны были полностью открыты: можно было подставить любой тип, даже не поддерживающий нужные операции.

12.3. Параметры шаблонов. Параметры типы и параметры значения.

12.3.1. Общая форма шаблона

```
1 template <параметры>
```

12.3.2. Типовые параметры (type template parameters)

Используются для подстановки типов: `int`, `double`, пользовательские типы и т.д.

```
1 template <typename T> // или: template <class T>
2 T max(T a, T b);
```

— `typename` и `class` — синонимы, но `typename` более предпочтительно (семантически точнее).

— Может быть несколько типовых параметров:

```
1 template <typename T, typename U>
2 auto sum(T a, U b) -> decltype(a + b);
```

12.3.3. Параметры-значения (non-type template parameters)

Используются для подстановки значений, известных на этапе компиляции:

```
1 template <int N>
```

```

2 int pow2() { return 1 << N; }
3
4 cout << pow2<4>(); // 16
5
6 template <int N>
7 struct Array { char data[N]; };
8
9 Array<10> a; // массив из 10 элементов

```

Допустимые типы (C++17 и раньше):

- Целые типы: `int`, `char`, `bool`, `enum`
- Указатели и ссылки: на объекты с внешним связыванием (например, глобальные переменные или функции)

С C++20 список расширен до:

- `float`, `double`
- Начиная с C++20, можно передавать объекты `constexpr`-типов в шаблоны:

```

1 template <Point P>
2 struct Widget
3 {
4     static constexpr int value = P.getX();
5 };
6
7 constexpr Point pt{5, 7};
8 Widget<pt> w; // шаблон параметризован объектом!

```

- Раньше (до C++20) это было невозможно: параметрами могли быть только простые значения (`int`, `char`, указатели).
- Теперь можно параметризовать целыми структурами, если они соответствуют требованиям литеральности.

12.3.4. Шаблоны как параметры шаблона (template template parameters)

Позволяют принимать шаблон в качестве параметра:

```

1 template <typename T>
2 class Box
3 {
4     public:
5         void show() { std::cout << "Box of " << typeid(T).name() << '\n'; }
6 };
7
8 template <template <typename> class Container>
9 class Wrapper
10 {
11     public:
12         Container<int> data;

```

```

13     void call() { data.show(); }
14 };
15
16 int main()
17 {
18     Wrapper<Box> w; // в шаблон передаётся сам шаблон Box<T>
19     w.call();      // вызовется Box<int>::show()
20 }

```

Объяснение:

- Box — это шаблон от одного типа T.
- Wrapper — шаблон, принимающий другой шаблон от одного типа.
- Мы передали Box в Wrapper, и внутри Wrapper он применяется к `int`, т.е. создаётся `Box<int>`.

12.3.5. Параметры по умолчанию

Параметры шаблона могут иметь значения по умолчанию:

```

1 template <typename T = int>
2 class A { T value; };

```

Правила

- Параметры со значениями по умолчанию должны быть последними.
- Как и с функциями, можно опускать при вызове, если они идут справа.

12.4. Шаблоны функций и методов классов

12.4.1. Шаблоны функций

Функции можно обобщить с помощью шаблонов. Это самый простой и распространённый случай.

Пример

```

1 template <typename T>
2 T max(T a, T b)
3 {
4     return (a > b) ? a : b;
5 }
6
7 int main()
8 {
9     max(3, 5);      // T = int
10    max(2.5, 4.1);  // T = double
11 }

```

Особенности

- Компилятор сам выводит тип T на основе аргументов.

- Можно явно указать тип: `max<int>(3, 5);`
- Не происходит неявного приведения типов:

```
1 | max(3, 2.5); // ошибка: нет общего T
```

12.4.2. Шаблоны методов классов

Метод внутри обычного (не шаблонного) класса может быть шаблонным.

Пример

```
1 | class Printer
2 | {
3 |     public:
4 |         template <typename T>
5 |         void print(const T& value)
6 |         {
7 |             std::cout << value << '\n';
8 |         }
9 | };
10 |
11 | Printer p;
12 | p.print(42);           // T = int
13 | p.print("Hello");      // T = const char*
```

12.4.3. Шаблонные методы в шаблонных классах

Обычно методы наследуют шаблонные параметры класса. Но они могут иметь и собственные:

```
1 | template <typename T>
2 | class Box
3 | {
4 |     T value;
5 |
6 |     public:
7 |         void print() { std::cout << value << '\n'; }
8 |
9 |         template <typename U>
10 |        void compare(const U& other)
11 |        {
12 |            std::cout << (value == other ? "equal\n" : "not equal\n");
13 |        }
14 | };
15 |
16 | Box<int> b{42};
17 | b.print();           // 42
18 | b.compare(42);        // equal
19 | b.compare(3.14);      // not equal
```

12.4.4. Шаблонные конструкторы и операторы

Можно шаблонизировать конструктор или оператор, чтобы, например, позволить копировать из другого типа:

```
1  template <typename T>
2  class Container
3  {
4      T value;
5
6      public:
7          template <typename U>
8          Container(const U& other) : value(other) {}
9  };
10
11 Container<int> a = 42;
12 Container<double> b = a; // работает благодаря шаблонному конструктору
```

12.5. Подстановка параметров в шаблон

- 1) Компилятор собирает всех кандидатов с таким именем, которые видит рядом (обычные функции и шаблоны).
- 2) Для каждого шаблона он пытается «угадать» аргументы шаблона по тем значениям, что мы передали, — подставить такие T, чтобы сигнатура совпала.
- 3) Подставляет угаданные типы обратно и проверяет: всё ли корректно и подходят ли ограничения. Неподходящие варианты просто отбрасываются, без ошибки.
- 4) Из оставшихся выбирается лучший: сначала предпочитают обычные функции, среди шаблонов — тот, который наиболее «конкретный».
- 5) Для выбранного варианта компилятор реально «разворачивает» тело: подставляет типы и компилирует именно эту версию. Всё внутри шаблона тоже инстанцируется по мере реальной надобности.
- 6) Сгенерированный вариант попадает в итоговый код; одинаковые экземпляры из разных файлов склеиваются на линковке.

```
1  template <typename T>
2  void f(T x) { std::cout << "generic\n"; }
3
4  template <>
5  void f<int>(int x) { std::cout << "int\n"; }
6  void f(int x) { std::cout << "int\n"; } // то же самое, компилятор сам
   выводит, что это f<int>
7
8  void f(double x) { std::cout << "non-template\n"; }
9
10 f(42); // вызывает специализацию шаблона: f<int>
11 f(3.14); // вызывает обычную функцию: f(double)
12 f("hi"); // вызывает f<const char*> → инстанцирует шаблон
```

12.6. Выведение типов параметров шаблона

Автоматическое выведение типа — это возможность компилятора автоматически определить тип переменной или шаблонного параметра из контекста.

```
1 int i;  
2 int& a = i;  
3 int&& c = 0; // ячейка памяти имеет имя — это lvalue  
4 const int& b = 0;
```

12.6.1. Параметр T (и **auto**)

- 1) Массивы и функции преобразуются к указателю
- 2) С аргумента «снимаются» внешняя ссылка и верхний **const/volatile**
 - Верхний = у указателя и ссылки, а не у того, на что они указывают

```
1 int* const cp = &v; // const — верхний: константен САМ указатель  
2 const int* pc = &v; // const — нижний: константен ОБЪЕКТ, на который  
   указываем
```

— У ссылки cv-квалификаторов нет

```
1 template<class T>  
2 void f(T v);  
3  
4 f(i);           // T = int  
5 f(a);           // T = int   (ссылка снята)  
6 f(c);           // T = int   (по имени c — lvalue)  
7 f(b);           // T = int   (внешний const снят)  
8 f(std::move(a)); // T = int   (rvalue → копия по значению)
```

12.6.2. Параметр T& (и **auto&**)

- Должен быть lvalue
- Сохраняет **const** аргумента
- rvalue не привяжется

```
1 template<class T>  
2 void f(T& v);  
3  
4 f(i); // T = int  
5 f(a); // T = int  
6 f(c); // T = int   (c — lvalue)  
7  
8 f(b); // T = const int  
9 f(std::move(a)); // ошибка: rvalue к T& не привязывается
```

12.6.3. Параметр `const T&` (и `const auto&`)

- Привязывается и к lvalue, и к rvalue
- T выводится без внешнего `const`

```
1 template<class T>
2 void f(const T& v);
3
4 f(i);           // T = int
5 f(a);           // T = int
6 f(c);           // T = int
7 f(b);           // T = int   (const «сидит» в самой ссылке)
8 f(std::move(a)); // T = int   (rvalue допустим)
```

12.6.4. Параметр `T&&` (и `auto&&`)

`T&&` — универсальная ссылка (forwarding reference), которая подстраивается под тип аргумента.

- Если аргумент — lvalue U, то T выводится как `U&`, а тип параметра схлопывается до `U&`.
- Если аргумент — rvalue U, то `T = U`, параметр — `U&&`.
- Схлопывание — это ситуации, когда через подстановки получается «ссылка на ссылку», и это сводится к одной ссылке по правилу:

$$\left\{ \begin{array}{l} \& + \& \implies \& \\ \& + \&\& \implies \& \\ \&\& + \& \implies \& \\ \&\& + \&\& \implies \&\& \end{array} \right.$$

```
1 template<class T>
2 void f(T&& v); // forwarding
3
4 f(i);   // T = int&      → параметр: int&
5 f(a);   // T = int&      → параметр: int&
6 f(c);   // T = int&      → параметр: int& (c — lvalue)
7 f(b);   // T = const int& → параметр: const int&
8 f(std::move(a)); // T = int → параметр: int&&
```

12.7. Пример явного указания типа параметра шаблона

```
1 template <typename T>
2 void f(T a, T b);
3
4 f(1, 2);   // OK: T = int
5 f(1, 2.0); // ошибка: не может вывести общий T
6
7 f<double>(1, 2.0); // OK: явно указали T = double
```

13. Неявные шаблоны. Протаскивание типа передаваемого параметра через шаблон (шаблон `std::forward`). Определение типа с помощью `decltype`. `decltype(auto)`. Специализация шаблонов функций.

13.1. Неявные шаблоны

В C++20 появилась возможность использовать неявные шаблоны функций, где вместо написания шаблона с `template<typename T, typename U>` можно сразу использовать `auto`:

```
1 // Обычно
2 template<typename T, typename U>
3 auto sum(T a, U b) { return a + b; }
4
5 // C++20: неявный шаблон
6 auto sum(auto a, auto b) { return a + b; }
```

13.1.1. Особенности

- Это синтаксический сахар — под капотом всё равно создаются шаблоны.
- Такие функции/методы не могут быть виртуальными (виртуальные — не шаблоны).
- Таким способом нельзя создавать шаблоны для классов и структур.
- Каждый `auto` — свой скрытый параметр шаблона. Типы выводятся независимо и могут отличаться.

```
1 auto sum(auto a, auto b) { return a + b; } // T и U выводятся отдельно
2 sum(2, 3.5); // a:int, b:double
```

- Если нужны одинаковые типы — накладываем ограничения:

```
1 auto sum(auto a, auto b)
2     requires std::same_as<decltype(a), decltype(b)>
3 {
4     return a + b;
5 }
```

- Для `auto&&` каждый параметр — forwarding ссылка сам по себе: категория значения выводится отдельно для каждого аргумента.
- Нельзя задать «типовой параметр по умолчанию» (как `typename T = int`) — только обычные значения по умолчанию у самих параметров.

13.2. Протаскивание типа передаваемого параметра через шаблон

- Пишем параметр как forwarding-ссылку: `T&& x` в шаблоне (или `auto&& x` в неявном).
- Передаём дальше как `std::forward<T>(x)` (или `std::forward<decltype(x)>(x)` в неявном), чтобы сохранить категорию значения: `lvalue` остаётся `lvalue`, `rvalue` остаётся `rvalue`.

Задача: сохранить точный тип переданного выражения, включая:

- `const` / `volatile`,
- ссылку (`&` или `&&`),
- возможность перемещения временного объекта.

`T&&` — универсальная ссылка (forwarding reference), которая подстраивается под тип аргумента.

- Если аргумент — lvalue `U`, то `T` выводится как `U&`, а тип параметра схлопывается до `U&`.
- Если аргумент — rvalue `U`, то `T = U`, параметр — `U&&`.
- Схлопывание — это ситуации, когда через подстановки получается «ссылка на ссылку», и это сводится к одной ссылке по правилу:

$$\left\{ \begin{array}{l} \& + \& \Rightarrow \& \\ \& + \&\& \Rightarrow \& \\ \&\& + \& \Rightarrow \& \\ \&\& + \&\& \Rightarrow \&\& \end{array} \right.$$

13.2.1. Классическая проблема

```
1 template <typename T>
2 void wrapper(T&& arg)
3 {
4     doSomething(arg); // теряем rvalue!
5 }
```

- Если передали временный объект (rvalue), `arg` — это всё равно lvalue, потому что имеет имя.
- `doSomething(arg)` вызовет копирование, а не перемещение!

Решение: `std::forward<T>(arg)`

```
1 template <typename T>
2 void wrapper(T&& arg)
3 {
4     // сохраняем категорию
5     doSomething(std::forward<T>(arg));
6 }
7
8 void wrapper(auto arg)
9 {
10    // для неявных шаблонов
11    doSomething(std::forward<decltype(arg)>(arg));
12 }
```

Почему работает?

`std::forward<T>` использует универсальную ссылку и `static_cast<T&&>`, чтобы «восстановить» исходную категорию:

```

1 template <typename T>
2 T&& forward(typename std::remove_reference<T>::type& t)
3 {
4     return static_cast<T&&>(t);
5 }

```

13.3. Определение типа с помощью decltype. decltype(auto)

13.3.1. decltype

Если внутри `decltype(...)` стоит имя переменной без скобок (id-выражение), то результат — ровно объявленный тип этой переменной, со всеми `&/&&` и `const/volatile`.

```

1 int x = 42;
2 decltype(x) y = x; // y: int

```

Главное отличие от auto

`auto` — выводит тип, но срезаёт `const`, `volatile` и ссылки, в отличие от `decltype(x)`.

Примеры

```

1 int i = 42;
2 int& r = i;
3 const int ci = 0;
4
5 decltype(i) a; // int
6 decltype(r) b = i; // int& — важно!
7 decltype(ci) c; // const int

```

13.3.2. Как decltype работает с выражениями

Выражение	decltype(...)	Комментарий
x	T& (если x lvalue)	Даёт ссылку
(x)	T&	Даже скобки сохраняют ссылочность
std::move(x)	T&&	rvalue
func()	тип возвращаемого значения	int, int&, const int& и т.д.

Пример с определением возвращаемого типа

```

1 auto sum(const auto& a, const auto& b) -> decltype(a + b)
2 {
3     return a + b; // возвращает тип выражения a + b
4 }
5

```

13.3.3. decltype(auto)

Это гибрид `auto` и `decltype`: позволяет вызывать `decltype(...)`, но автоматически:

```

1 int x = 10;
2 int& r = x;
3
4 auto a = r;           // a: int (ссылка теряется)
5 decltype(auto) b = r; // b: int& (точный тип сохранён)

```

Он позволяет сохранить точный тип при возвращении из шаблона:

```

1 template <typename T, typename U>
2 decltype(auto) sum(T&& a, U&& b)
3 {
4     return std::forward<T>(a) + std::forward<U>(b);
5 }

```

Если мы бы написали просто `auto`, то:

- возвращаемая `const int&` → станет `int`
- возвращаемая `int&&` → станет `int`

Пример с функцией

```

1 int& get_ref();
2 int get_val();
3
4 decltype(auto) a = get_ref(); // a: int&
5 decltype(auto) b = get_val(); // b: int

```

13.4. Специализация шаблонов функций

13.4.1. Что такое специализация?

Специализация шаблона функции — это создание отдельной реализации шаблона для конкретных типов параметров.

Она позволяет:

- Изменить поведение шаблона при определённых типах.
- Реализовать оптимизированный код для конкретных случаев.

13.4.2. Полная специализация

Полная специализация — когда подставлены все параметры шаблона.

```

1 template <typename T>
2 void print(const T& x)
3 {
4     std::cout << "Generic: " << x << '\n';
5 }
6
7 // Полная специализация для типа int

```

```

8 template <>
9 void print<int>(const int& x)
10 {
11     std::cout << "Integer: " << x << '\n';
12 }

```

```

1 print(3.14); // Generic
2 print(42);   // Integer

```

Правила полной специализации функций

- Функции должны быть в том же пространстве имён, где объявлен первичный шаблон.
- Объявляются после первичного шаблона (иначе не на что «специализироваться»).
- В перегрузке могут конкурировать с обычными функциями
 - Выбирается «лучшая» по обычным правилам перегрузки (часто нешаблонная версия выигрывает).

13.4.3. Частичная специализация шаблонов (только для классов)

Частичная специализация — когда подставлены не все параметры шаблона, а только часть, или сделаны ограничения на форму.

Для функций частичная специализация не поддерживается, только полная.

Как обойти частичную специализацию?

Используем перегрузку вместо специализации:

```

1 template <typename T>
2 void f(T* ptr)
3 {
4     std::cout << "Pointer\n";
5 }
6
7 void f(int* ptr)
8 {
9     std::cout << "Pointer to int\n";
10 }

```

Это обычная перегрузка, но выглядит как специализация.

13.4.4. Что делает компилятор?

Когда вызывается `f(...)`, компилятор:

- 1) Ищет перегруженные функции с подходящей сигнатурой.
- 2) Если не находит, ищет полные специализации шаблонов.
- 3) Если и их нет — выводит типы и инстанцирует универсальный шаблон.

```

1 template <typename T>
2 void f(T) { std::cout << "Generic\n"; }

```

```
3
4 template <>
5 void f<int>(int) { std::cout << "int\n"; }
6
7 void f(double) { std::cout << "double overload\n"; }
8
9 f(1);      // int специализация
10 f(3.14);   // double перегрузка
11 f("abc"); // Шаблонная функция
```

14. Шаблоны типов. Шаблоны классов. Полная или частичная специализация шаблонов классов. Параметры шаблонов задаваемых по умолчанию. Шаблоны с переменным числом параметров. Пространства имен.

14.1. Шаблоны типов и шаблоны классов

Шаблон — это способ описания универсального кода, который может работать с разными типами данных. Компилятор на основании конкретных типов, указанных при вызове шаблона, создаёт конкретные реализации функций или классов (процесс называется **инстанцированием**).

14.1.1. Шаблоны типов (alias-template)

Используется для задания типа на основе шаблона:

```
1 // 1) Упростили длинное имя
2 template<typename T>
3 using Vec = std::vector<T>;
4 // Vec<int> == std::vector<int>
5
6 // 2) Мета-алиас как в стандартной библиотеке
7 template<typename T>
8 using remove_cvref_t = std::remove_cv_t<std::remove_reference_t<T>>;
9
10 // 3) Упрощение для указателей на функции
11 template <typename T>
12 using comparator_t = int (*)(const T&, const T&);
13
14 int compare(const int& a, const int& b) { return a - b; }
15 comparator_t<int> cmp = compare; // Обязательно указываем `<int>`
```

- Не создаёт новый тип, а даёт синоним существующему.
- В отличие от `typedef`, может быть шаблонным.
- Нельзя перегружать и нельзя специализировать ни полностью, ни частично
- Нет вывода параметров.
 - У функций типы шаблона выводятся из аргументов вызова.
 - У `alias` все параметры должны быть указаны явно или иметь значения по умолчанию.

14.1.2. Шаблоны классов

```
1 template <typename T>
2 class MyContainer
3 {
4     T value;
5 }
```

```

6 public:
7     void set(const T& val) { value = val; }
8     T get() const { return value; }
9 };

```

Создание конкретного класса:

```

1 MyContainer ci{9}; // T = int;
2 ci.set(10);
3
4 MyContainer<std::string> cs;
5 cs.set("hello");

```

14.2. Полная и частичная специализация шаблонов классов

- Специализации шаблонов используются, когда требуется изменить поведение шаблона для определённых типов.
- Можно специализировать и отдельные члены шаблонного класса.

Полная специализация — когда подставлены все параметры шаблона.

- Полная специализация срабатывает только при точном совпадении аргументов (`bool` $\neq$ `const bool` на уровне параметров шаблона).

Частичная специализация — когда подставлены не все параметры шаблона, а только часть, или сделаны ограничения на форму.

14.2.1. Примеры

```

1 template<typename T, typename U>
2 class Duo
3 {
4     T first{};
5     U second{};
6
7     public:
8     void set(T a, U b)
9     {
10         first = std::move(a);
11         second = std::move(b);
12     }
13     const T& get_first() const { return first; }
14     const U& get_second() const { return second; }
15 };
16
17
18 // Полная специализация для конкретной пары типов
19 template<>

```

```

20 class Duo<bool, int>
21 {
22     bool flag{};
23     int value{};
24 public:
25     void set(bool f, int v)
26     {
27         flag = f;
28         value = v;
29     }
30     bool get_first() const { return flag; }
31     int get_second() const { return value; }
32 };
33 // Полная специализация побеждает всегда при точном совпадении
34 Duo<bool, int> c;
35
36 // Более узкий шаблон: «если U – указатель»
37 template<typename T, typename U>
38 class Duo<T, U*>
39 {
40     T first{};
41     U* ptr{};
42
43 public:
44     void set(T a, U* p) {
45         first = std::move(a);
46         ptr = p;
47     }
48     const T& get_first() const { return first; }
49     U* get_ptr() const { return ptr; }
50 };
51 Duo<int, double> a; // Первичный шаблон
52 Duo<int, double*> b; // Частичная специализация

```

14.2.2. Возможен конфликт специализаций

```

1 template <typename T>
2 class A<int, T> {}; // подходит, если первый аргумент – int
3 template <typename T>
4 class A<T, T> {}; // подходит, если оба аргумента одинаковые
5
6 A<int, int> a; // неоднозначность: подходит и та, и другая

```

— Наличие первичного шаблона не убирает неоднозначность

- Как только хоть одна частичная специализация подходит, первичный шаблон больше не кандидат «по умолчанию».
- Любая подходящая частичная специализация считается строже, чем первичный.

- Если таких специализаций две и они не сравнимы по строгости, получается неоднозначность, и компилятор не имеет права откатываться к первичному.

14.3. Параметры шаблонов по умолчанию

14.3.1. Синтаксис

```
1 template<typename T, typename U = int>
2 class Duo { /* ... */ };
3
4 Duo<double> x;           // Duo<double, int>
5 Duo<double, char> y;     // переопределили второй параметр
```

14.3.2. Правила

- Значения по умолчанию указываются только справа:

```
1 template <typename T, typename U = double> // OK
2 template <typename T = int, typename U>    // Ошибка
```

- После первого параметра по умолчанию все последующие также должны иметь значения по умолчанию.
- При использовании:
 - Можно пропускать только хвостовые параметры шаблона.
 - Если явно указан первый, но не указан второй — он берётся по умолчанию.
- Значения по умолчанию указываются только в первичном шаблоне, а не в специализациях.

14.3.3. Пример

```
1 template<typename T, std::size_t N = 16>
2 struct Arr { T data[N]; };
3
4 Arr<char> a;      // N = 16
5 Arr<int, 32> b;   // N = 32
```

14.4. Шаблоны с переменным числом параметров

14.4.1. Основная идея

Variadic templates используют, когда нужно обрабатывать множество аргументов, но заранее неизвестно их количество.

```
1 template <typename... Params>
2 void f(Params... params);
```

- Params... — пакет параметров шаблона.
- params... — пакет параметров функции.
- Пакет может быть пустым.

14.4.2. Универсальные ссылки + расширение пакета

Часто используют универсальные ссылки, чтобы передавать и lvalue, и rvalue:

```
1 template <typename... Params>
2 void f(Params&&... params)
3 {
4     g(params...); // расширили пакет параметров
5 }
```

Если `f(1, 2, 3)`, то будет вызов: `g(1, 2, 3)`

14.4.3. Расширения в сложных выражениях

```
1 g(h(params)...);
2
3 g(
4     h(p1), h(p2), ..., h(pN)
5 );
```

```
1 g(h(params, params...)...);
2
3 g(
4     h(p1, p1, p2, ..., pN),
5     h(p2, p1, p2, ..., pN),
6     h(p3, p1, p2, ..., pN)
7     ...
8     h(pN, p1, p2, ..., pN)
9 );
```

```
1 g((h(params) + h(params...))...);
2
3 g(
4     h(p1) + h(p1, p2, ..., pN),
5     h(p2) + h(p1, p2, ..., pN),
6     ...
7     h(pN) + h(p1, p2, ..., pN)
8 );
```

14.4.4. Бинарные операции над пакетами

```
1 params = (v1, v2, v3, v4)
```

- @ — любой бинарный оператор
- params — пакет параметров

Раскрытие:

```
1 params @ ...
```

```
2 → v1 @ (v2 @ (v3 @ v4)) // Справа налево (правоассоциативное)
```

```
1 ... @ params  
2 → ((v1 @ v2) @ v3) @ v4 // Слева направо (левоассоциативное)
```

```
1 params @ ... @ value  
2 → v1 @ (v2 @ (v3 @ (v4 @ value)))
```

```
1 value @ ... @ params  
2 → (((value @ v1) @ v2) @ v3) @ v4
```

14.4.5. Реальные примеры: суммы, свёртки, цепочки

Сумма элементов (левоассоциативная)

```
1 template <typename... Args>  
2 auto sum(Args... args)  
3 {  
4     return (... + args); // ((a + b) + c) + d  
5 }
```

Умножение с начальными значением

```
1 template <typename... Args>  
2 auto product(Args... args)  
3 {  
4     return (1 * ... * args); // (1 * a * b * c)  
5 }
```

14.5. Пространства имён

14.5.1. Что такое пространство имён?

Пространство имён (namespace) — это способ организовать код и избежать конфликтов имён, особенно при использовании библиотек.

```
1 namespace имя  
2 {  
3     // код, типы, функции, классы  
4 }
```

14.5.2. Зачем нужно?

```
1 // Функция из первой библиотеки  
2 int max(int a, int b);  
3 // Функция из второй библиотеки  
4 float max(float a, float b);
```

Если подключить обе — будет конфликт имён. Чтобы этого избежать, каждую функцию можно «поместить» в пространство имён:

```
1 namespace lib1
2 {
3     int max(int a, int b);
4 }
5 namespace lib2
6 {
7     float max(float a, float b);
8 }
```

Теперь можно явно указать:

```
1 lib1::max(3, 5);
2 lib2::max(3.5f, 5.0f);
```

14.5.3. `using`

Точечный импорт имени (без засорения всего scope)

```
1 using std::string; // только string
```

Импорт всего пространства имён

```
1 using namespace math;
2
3 double area = pi * square(10);
```

```
1 // your.hpp
2 #include <print>
3 using namespace std; // подтянули std::print
4
5 // user.cpp
6 #include "your.hpp"
7 void print(const char*);
8 // теперь вызов print(...) может стать неоднозначным
```

- В `.hpp` — никогда не используем `using namespace`.
- В `.cpp` допускается локально в узкой области, лучше точечные `using std::....`

14.5.4. Анонимное (безымянное) пространство имён

```
1 namespace
2 {
3     int hidden = 42;
4 }
```

- Заменяет старый `static` на свободных функциях/переменных.

14.5.5. Вложенные пространства имён

```
1 namespace graphics
2 {
3     namespace color
4     {
5         void setRGB(int, int, int);
6     }
7 }
8
9 // С C++17 можно:
10 namespace graphics::color
11 {
12     void setRGB(int, int, int);
13 }
```

15. Ограничения накладываемые на шаблоны. Требования к шаблонам (requires). Концепты. Типы ограничений. Варианты определения шаблонов функций и классов с концептами.

15.1. Ограничения накладываемые на шаблоны

Ограничение — это логическое условие на параметры шаблона.

- Если условие истинно, шаблон участвует в перегрузке.
- Если ложно, этот шаблон просто не считается подходящим (без «жёсткой» ошибки), пока есть другие кандидаты.

С приходом C++20 появилась возможность явно задавать ограничения на параметры шаблона. Это позволяет:

- Проверять требования к типу до компиляции тела шаблона.
- Выдавать более осмысленные сообщения об ошибках.
- Улучшать читаемость и поддержку кода.

Выражения внутри `requires`-выражения проверяются «безболезненно»: если что-то не подходит, это превращается в «ограничение не выполнено», а не в лавину ошибок подстановки.

15.2. Требования к шаблонам (requires)

Механизм `requires` в C++20 позволяет:

- Ограничить применимость шаблона только к тем типам, которые удовлетворяют определённым условиям.
- Использовать предикаты (выражения, вычисляемые во время компиляции) для уточнения допустимости типа.

15.2.1. Синтаксис `requires`

```
1 // После списка параметров шаблона
2 template<typename T>
3     requires std::integral<T>
4 T inc(T x)
5 {
6     return x + 1;
7 }
8
9 // Или после заголовка функции (у классов только первый способ)
10 template<typename T>
11 T inc(T x) requires std::integral<T>
12 {
13     return x + 1;
14 }
```

15.2.2. Что можно использовать в `requires`?

После `requires` указывается одно **булево выражение**.

Имена концептов (`concept-id`)

```
1 requires std::integral<T>;
2 requires std::same_as<U, V>;
3 requires std::derived_from<T, Base>;
```

Булевы константные выражения, зависящие от параметров

```
1 requires (sizeof(T) <= 8);
2 requires std::is_trivially_copyable_v<T>;
3 requires noexcept(std::declval<T&>() = std::declval<const T&>());
```

Логические связки и скобки

```
1 requires (std::integral<T> && (sizeof(T) >= 2)) || std::floating_point<T>;
2 requires (!std::copy_constructible<T>);
```

`requires`-выражение как единый операнд

```
1 requires (requires (T a, T b) { a + b; });
```

15.2.3. Преимущества `requires`

- Проверка происходит до компиляции тела шаблона.
- Компилятор выдаёт краткие и понятные сообщения об ошибках.

15.2.4. Важная особенность

`requires` — это ограничение, но не гарантия. Даже если тип прошёл проверку в `requires`, это не значит, что шаблон с ним обязательно скомпилируется. Ошибки всё ещё могут возникать при глубокой инстанциии (например, в теле функции).

15.3. Концепты

Концепт — это именованное булево выражение времени компиляции о типах/значениях.

Введён в C++20, он:

- Определяет условия, которым должен соответствовать тип.
- Используется для ограничения параметров шаблона.

15.3.1. Синтаксис

```
1 template <typename T>
2 concept ConceptName = <логическое или requires-выражение>;
```

Пример

```
1 #include <concepts>
2 #include <type_traits>
3
4 template<typename T>
5 concept SmallSignedInt =
6     std::signed_integral<T> && (sizeof(T) <= 4);
7
8 template <typename T>
9 concept Incrementable = requires(T t)
10 {
11     {++t} -> same_as<T&>;
12     {t++} -> same_as<T>;
13 }
```

15.3.2. Зачем нужны концепты?

- Позволяют повторно использовать ограничения (не писать `requires` каждый раз).
- Делают код самодокументируемым: `template <Sortable T>` читается гораздо понятнее, чем `requires ....`

15.3.3. Концепты из стандартной библиотеки

Концепт	Что проверяет
<code>std::same_as<T, U></code>	Типы T и U идентичны
<code>std::integral<T></code>	Тип T — целочисленный
<code>std::floating_point<T></code>	Тип — с плавающей точкой
<code>std::convertible_to<T, U></code>	T можно привести к U
<code>std::derived_from<T, U></code>	T является производным от U
<code>std::invocable<F, Args...></code>	<code>F(args...)</code> допустим

15.4. Типы ограничений (`requires`-выражений)

Ограничения в шаблонах делятся на разные типы в зависимости от того, что именно мы проверяем.

15.4.1. Простые выражения

Проверяется, допустимо ли выражение.

```
1 requires(T a, T b) {
2     a + b; // просто «существует оператор +»
3 };
```

15.4.2. Составные выражения

Проверяется тип результата выражения.

```
1 requires(std::ostream& os, T t) {
```

```

2 { os << t } -> std::same_as<std::ostream&>; // тип результата
3 { t.swap(t) } noexcept; // не бросает исключений
4 };

```

15.4.3. Требование к типу

Проверяется наличие вложенных типов.

```

1 requires(T t) {
2     typename T::value_type; // у T должен быть вложенный тип value_type
3 };

```

15.4.4. Вложенные requires

Всё выше перечисленное может быть объединено:

```

1 // Как в функциях, имя параметра можно опустить, если оно не используется
2 requires(T)
3 {
4     requires std::copy_constructible<T>; // любое constraint-выражение
5     requires (sizeof(T) <= 64);
6 };

```

Комбинировать их можно свободно; итог — единое булево условие.

constraint-выражение — это логическое условие, накладываемое на параметры шаблона.

15.5. Варианты определения шаблонов функций и классов с концептами

15.5.1. Функции

```

1 // 1) Концепт в параметре шаблона
2 template<std::integral T>
3 T inc(T x) { return x + 1; }
4
5 // 2) requires (prefix или trailing)
6 template<typename T>
7     requires std::integral<T>
8 T dec(T x) { return x - 1; }
9
10 template<typename T>
11 T mul(T a, T b) requires std::integral<T>
12 {
13     return a * b;
14 }
15
16 // 3) «Сокращённая» форма с auto

```

```
17 auto add(std::integral auto a, std::integral auto b) { return a + b; }
```

15.5.2. Классы

```
1 // 1) Через концепт в списке параметров
2 template<std::regular T>
3 class Box { /*...*/ };
4
5 // 2) Через requires после параметров
6 template<typename T>
7     requires std::regular<T>
8 class Box { /*...*/ };
```

— Для класса «trailing requires» не бывает — только после параметров шаблона.

15.5.3. Ограничения членов класса

```
1 template<typename T>
2 class Box
3 {
4     public:
5         // Метод
6         void put(const T& v) requires std::copyable<T> { /*...*/ }
7
8         // Конструктор
9         Box(const T& v) requires std::movable<T> { /*...*/ }
10
11         // Оператор преобразования
12         template<class U>
13             requires std::convertible_to<T, U>
14             explicit operator U() const { return static_cast<U>(/*...*/); }
15
16         // Статический метод
17         template<class U>
18             static void log(const U&) requires std::ostreamable<U> { /*...*/ }
19 };
```

15.5.4. Пример: концепт с несколькими параметрами

Концепт принимает два типа:

```
1 template <typename T, typename U>
2 concept PairAddable = requires(T a, U b)
3 {
4     { a + b } -> std::same_as<T>;
5 };
```

- Тогда использовать его напрямую как `template <PairAddable T>` невозможно, потому что непонятно, чем заполнить `U` — компилятор подставит только первый аргумент (`T`), а `U` окажется неопределённым.
- Но можно явно указать второй параметр при использовании концепта:

```
1 template<class T>
2 requires PairAddable<T, int>
3 void f(T);
```

16. Проблемы с динамическим выделением и освобождением памяти. Шаблон Holder. «Умные указатели» в C++: `unique_ptr`, `shared_ptr`, `weak_ptr`. Связь между `shared_ptr` и `weak_ptr`.

16.1. Проблемы с динамическим выделением и освобождением памяти

В C++ динамическое выделение памяти осуществляется через оператор `new`, а освобождение — через `delete`. Основная проблема — необходимость вручную управлять временем жизни объектов.

16.1.1. Утечка памяти

```
1 A* p = new A{};
2 p->f(); // если f выбросит исключение, delete не будет вызван
3 delete p;
```

Объект в куче создаётся, но не удаляется, если программа покидает область действия до `delete` — например, из-за исключения.

16.1.2. Двойное освобождение памяти

```
1 A* p = new A{};
2 delete p;
3 delete p; // неопределённое поведение
```

16.1.3. Использование удалённой памяти (висячие указатели)

```
1 A* p = new A{};
2 delete p;
3 p->f(); // обращение к освобождённой памяти
```

16.1.4. Способы решения

- Использовать RAII — оборачивать `new` в объекты, которые автоматически удаляют ресурсы в деструкторе.
- Избегать `new` и `delete` напрямую. Использовать обёртки, шаблоны и умные указатели.
- В сложных системах: применять `make_shared`, `make_unique`, фабрики и шаблоны-владельцы (Holder и др.).

RAII (Resource Acquisition Is Initialization / Получение Ресурса Есть Инициализация) — это идиома, при которой «ресурс живёт ровно столько, сколько живёт владеющий им объект».

Объект в момент создания сразу «захватывает» какой-то ресурс (файл, память), а при уничтожении — автоматически освобождает.

16.2. Шаблон Holder

Holder — это концептуальный прототип умного указателя, разработанный для устранения проблем ручного управления памятью. Он реализует принцип RAII: объект `Holder<T>` хранит `T*`, а при разрушении сам вызывает `delete`.

16.2.1. Идея

- Указатель помещается в оболочку, которая:
 - располагается на стеке;
 - управляет временем жизни объекта;
 - при уничтожении удаляет объект.
- Интерфейс `Holder` — максимально прозрачный, с перегрузкой `->`, `*`, `bool` и др.
- Обязательные компоненты:
 - запрет на копирование (нельзя допустить двойное удаление);
 - разрешение перемещения (move-семантика);
 - передача владения через методы `get()`, `release()`, `reset()`.
- `Holder` — концепция, которая ближе к `unique_ptr`.

16.2.2. Пример использования

```
1 Holder<A> obj(new A{});
2 obj->f(); // безопасный вызов
```

Даже если `f()` выбросит исключение, `Holder` вызовет `delete` в своём деструкторе.

16.2.3. Пример реализации

```
1 template <typename T>
2 class Holder
3 {
4     T* ptr = nullptr;
5 public:
6     explicit Holder(T* p = nullptr) noexcept : ptr(p) {}
7     ~Holder() { delete ptr; }
8
9     Holder(const Holder&) = delete;
10    Holder& operator=(const Holder&) = delete;
11
12    Holder(Holder&& other) noexcept :
13        ptr(std::exchange(other.ptr, nullptr)) {}
14    Holder& operator=(Holder&& other) noexcept
15    {
16        if (this != &other) {
17            delete ptr;
18            ptr = std::exchange(other.ptr, nullptr);
19        }
20        return *this;
21    }
```

```

22
23     T* get()          const noexcept { return ptr; }
24     T& operator*()    const noexcept { return *ptr; }
25     T* operator->()   const noexcept { return ptr; }
26     operator bool()   const noexcept { return ptr != nullptr; }
27
28     T* release() noexcept // отдать владение
29     {
30         return std::exchange(ptr, nullptr);
31     }
32     void reset(T* p = nullptr) noexcept
33     {
34         if (ptr != p) {
35             delete ptr;
36             ptr = p;
37         }
38     }
39 };

```

16.2.4. auto_ptr

auto_ptr — устаревшая попытка реализации того же, но одним указателем решить все проблемы оказалось невозможно.

16.3. Умные указатели в C++

16.3.1. unique_ptr

Это умный указатель с уникальным владением. Только один unique_ptr может владеть объектом — никакого копирования.

```

1 std::unique_ptr<A> p = std::make_unique<A>();
2 p->f();

```

Особенности

- Перемещаемый, но не копируемый.
- Удаляет объект при уничтожении.
- Нет накладных расходов на счётчик ссылок.

Доступные операторы

- *, ->, bool, [], ==, !=

Доступные методы

- T* get() — получить «сырой» указатель, не отдавая владение.
- T* release() — отдать владение, ответственность за удаление переходит к вызывающему.
- void reset(T* ptr = nullptr) — удалить текущий объект (если есть) и принять новый указатель.

- `void swap(unique_ptr& other)` — обменять указатели между двумя `unique_ptr` без удаления объектов.

```
1 std::unique_ptr<A> p1 = std::make_unique<A>();
2 std::unique_ptr<A> p2;
3
4 p1.swap(p2); // теперь p2 владеет объектом, а p1 — nullptr
```

Проблема с массивами

Если работает с массивом (`new[]`), то указатель ничего не знает размер.

```
1 // Допустимо, но нельзя узнать размер
2 std::unique_ptr<int[]> arr(new int[10]);
```

16.3.2. shared_ptr

Умный указатель с разделяемым владением. Несколько `shared_ptr` могут владеть одним объектом.

```
1 std::shared_ptr<A> p1 = std::make_shared<A>();
2 std::shared_ptr<A> p2 = p1; // счётчик увеличился
```

Особенности

- Копируется (инкремент счётчика), перемещается (передача без инкремента).
- Объект удаляется, когда счётчик ссылок становится 0.
- Есть формы для массивов: `shared_ptr<T[]>`, `make_shared<T[]>(n)` (C++20).

Доступные операторы

- `*`, `->`, `bool`, `[]`, `==`, `!=`

Доступные методы

- `T* get()` — получить «сырой» указатель, не отдавая владение.
- `void reset(T* ptr = nullptr)` — перестать владеть текущим объектом (удаление объекта зависит от счётчика) и принять новый указатель.
- `void swap(shared_ptr& other)` — обменять указатели между двумя `shared_ptr` без удаления объектов.
- `long use_count()` — узнать, сколько указателей ссылается.
- `bool unique()` — проверить, остался ли только один.

Что хранится в контрольном блоке `shared_ptr`

1		
2	Контрольный блок	
3		
4	Счётчик <code>shared_ptr</code>	(strong count)
5	Счётчик <code>weak_ptr</code>	(weak count, включая <code>shared_ptr</code> + <code>weak_ptr</code>)
6	Указатель на объект	(T*)
7	Деструктор объекта	
8		

- Копирование `shared_ptr` — увеличение strong count
- Уничтожение `shared_ptr` — уменьшение strong count
- Копирование `weak_ptr` — увеличение weak count
- Уничтожение `weak_ptr` — уменьшение weak count

Жизненный цикл объекта

- Объект (T) удаляется, когда `strong count == 0`
- Контрольный блок удаляется, когда оба счётчика становятся нулями:

```
1 | strong count == 0 && weak count == 0
```

- Цикл `shared_ptr` $\leftrightarrow$ `shared_ptr` не сойдётся счётчиками до нуля $\Rightarrow$ утечка. Для обратной ссылки применяют `weak_ptr`.

Пример

```
1 | auto sp1 = std::make_shared<int>(42); // strong = 1, weak = 0
2 | auto wp = std::weak_ptr<int>(sp1);    // strong = 1, weak = 1
3 | {
4 |     auto sp2 = sp1;                    // strong = 2
5 | }
6 |                                     // sp2 уходит -> strong = 1
7 | sp1.reset();                          // strong = 0, объект удалён
8 | wp.expired(); // true                  // weak = 1
```

Контрольный блок удалится только после `wp`:

```
1 | wp.reset(); // weak = 0 → удаляется и контрольный блок
```

16.3.3. `weak_ptr`

Указатель, не владеющий объектом, управляемым `shared_ptr`. Используется, чтобы избежать циклических ссылок и висячих указателей.

```
1 | std::weak_ptr<A> w = p1; // p1 — shared_ptr
2 | if (auto s = w.lock()) {
3 |     s->f();
4 | }
```

Особенности

- Не имеет доступа к объекту напрямую: сначала `lock()` → временный `shared_ptr`.
- Увеличивает только `weak_count` контрольного блока.
- Может «протухнуть» (`expired()`), если все `shared_ptr` уничтожены.

Доступные методы

- `void reset()` — отвязаться от контрольного блока (`shared_ptr`).
- `void swap(weak_ptr& other)` — обменять указатели между двумя `weak_ptr`.
- `shared_ptr<T> lock()`
 - Если объект жив, возвращает ненулевой `shared_ptr` и увеличивает счётчик.
 - Если объект удалён, возвращает пустой `shared_ptr`.
- `bool expired()` — проверить, жив ли объект (`user_count == 0`).
- `long use_count()` — сколько `shared_ptr` владеют объектом.

16.4. Связь между `shared_ptr` и `weak_ptr`

16.4.1. Общая идея

- `shared_ptr` — владеет объектом, управляет его временем жизни.
- `weak_ptr` — не владеет, но может наблюдать за `shared_ptr`.

Это два взгляда на один и тот же ресурс, но с разной ответственностью.

16.4.2. Почему нужен `weak_ptr`

Если два объекта владеют друг другом через `shared_ptr`, возникает циклическая зависимость, и они никогда не удалятся:

```
1 struct A
2 {
3     std::shared_ptr<B> b;
4 };
5
6 struct B
7 {
8     std::shared_ptr<A> a;
9 };
```

Решение

Один из указателей должен быть `weak_ptr`, чтобы не увеличивать счётчик и не препятствовать удалению:

```
1 struct B
2 {
3     std::weak_ptr<A> a;
4 };
```

16.4.3. Механика связи

- `shared_ptr` хранит:
 - `strong count` — число владельцев (`shared`);
 - `weak count` — число наблюдателей (`weak_ptr`) + 1 (для управления блоком памяти).
- `weak_ptr` увеличивает только `weak count`.
- `lock()` — безопасный способ «временного» доступа: если объект всё ещё жив, можно с ним поработать.

16.4.4. Восстановление `shared_ptr` из `weak_ptr`

```
1 std::shared_ptr<A> p = std::make_shared<A>();
2 std::weak_ptr<A> w = p;
3
4 if (auto s = w.lock()) { // s — валидный shared_ptr
5     s->f();
6 } else {
7     // объект уже уничтожен
8 }
```

17. Приведение типа в C++: `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast`. Контейнерные классы и итераторы. Требования к контейнерам и итераторам. Категории итераторов. Операции над итераторами. Цикл `for` для работы с контейнерными объектами.

17.1. Приведение типа в C++

17.1.1. `static_cast`

Это приведение типа во время компиляции. Компилятор проверяет корректность типов и выполняет безопасные преобразования.

Когда используется

- Числовые преобразования: расширение/сужение.
- Перевод `enum` $\longleftrightarrow$ целые (если осознанно).
- В иерархиях классов:
 - Хорошо: вверх по иерархии (`Derived*` $\longrightarrow$ `Base*`).
 - Опасно: вниз (`Base*` $\longrightarrow$ `Derived*`) — без проверки.
- `void*` $\longleftrightarrow$ `T*`, если точно знаешь исходный тип.
- Снятие ссылочности/приведение ссылок.
- Явные преобразования, которые могли бы быть выполнены неявно.

Особенности

- Не снимает `const/volatile`.

Пример

```
1 // Явное сужение
2 double d = 3.14;
3 int i = static_cast<int>(d);
4
5 struct Base { virtual ~Base() = default; };
6 struct Derived : Base { void foo(); };
7
8 // БЕЗ проверки: разыменование ок, только если pb реально указывает на
   Derived
9 Base* pb = new Derived;
10 Derived* pd1 = static_cast<Derived*>(pb);
11
12 // Допустимо, если pv получен из Base*
13 void* pv = pb;
14 Base* pb2 = static_cast<Base*>(pv);
```

17.1.2. `dynamic_cast`

Это приведение типа во времени выполнения программы с проверкой на возможность этой операции.

Когда используется

- В деревьях иерархий, когда храним объекты «как Base», а иногда нужно узнать, не является ли конкретный объект `Derived`, `OtherDerived` и т.п.
- При множественном наследовании `dynamic_cast` сам корректирует адрес и позволяет «перейти» от одной базовой части объекта к другой без ручных «трюков» с указателями.

Требования

- Исходник — указатель/ссылка на полиморфный тип (обычно `Base*/Base&`).
- Целевой тип — класс в той же иерархии.

Поведение

- Если типы на этапе компиляции не состоят в одной иерархии наследования — ошибка компиляции.
- Выполняет проверку типов и возвращает:
 - `nullptr`, если указатель не может быть приведён
 - выбрасывает `std::bad_cast`, если используется со ссылкой
- Особый случай: `dynamic_cast<void*>(p)` даёт указатель на самый производный объект.

Примеры

```
1 struct Base { virtual ~Base() = default; };
2 struct Left : Base { void lf(); };
3 struct Right: Base { void rg(); };
4
5 void use(Base* pb) {
6     if (auto pl = dynamic_cast<Left*>(pb)) {
7         pl->lf(); // Ок, если pb действительно указывал на Left
8     } else if (auto pr = dynamic_cast<Right*>(pb)) {
9         pr->rg();
10    } // иначе — ни то, ни другое
11 }
```

```
1 // Ссылки: исключение при провале
2 void call(Base& b) {
3     try {
4         auto& r = dynamic_cast<Right&>(b);
5         r.rg();
6     } catch (const std::bad_cast&) {
7         // не Right
8     }
9 }
```

```

1 // Кросс-каст (множественное наследование)
2 struct A { virtual ~A() = default; };
3 struct B { virtual ~B() = default; };
4 struct C : A, B {};
5
6 void f(A* pa) {
7     if (auto pb = dynamic_cast<B*>(pa)) { // корректирует смещение
8         /* ... */
9     }
10 }

```

17.1.3. `const_cast`

Когда используется

- Меняет только квалификаторы `const/volatile` у объектов по указателю/ссылке (добавляет или снимает).
- Никаких преобразований «по смыслу» не делает.

```

1 // сняли const с *pc (ОПАСНО: менять можно только если исходный объект
  не-константный)
2 const int* pc = /* ... */;
3 int* p = const_cast<int*>(pc);
4
5 // добавили const (обычно каст не нужен: int* → const int* и так неявно)
6 int* pn = /* ... */;
7 const int* cpn = const_cast<const int*>(pn);

```

Особенности

- Если объект изначально неконстантный, но у нас есть `const`-указатель/ссылка на него, можно снять `const` и безопасно менять.
- Если объект объявлен как `const`, снимать `const` и менять — неопределённое поведение.
- `const` на содержимое: `const int* p` — нельзя менять `*p`. Вот это как раз можно снять с помощью `const_cast<int*>(p)`.
- `const` на сам указатель: `int* const p` — менять адрес `p` нельзя; `const_cast` тут не поможет, это уже не про квалификатор у объекта.

```

1 class S
2 {
3     // лучше `mutable int counter = 0;` если нужно менять в const-методе
4     int counter = 0;
5 public:
6     void touch() const
7     {
8         // плохо: лучше сделать поле mutable
9         const_cast<S*>(this)->counter++;
10    }

```

```
11 void touch() { counter++; }
12 };
```

17.1.4. reinterpret_cast

- Самый низкоуровневый каст.
- Грубая переинтерпретация бит/адреса без каких-либо проверок и без реального преобразования значения. Компилятор лишь меняет «как смотреть» на те же биты/тот же адрес.

```
1 int x = 12345;
2 char* p = reinterpret_cast<char*>(&x); // доступ к байтам int как к массиву char
```

Часто используется при низкоуровневом программировании, взаимодействии с оборудованием или внешними бинарными форматами.

17.2. Контейнерные классы и итераторы. Требования к контейнерам и итераторам

17.2.1. Контейнерные классы

Контейнер — это шаблонный класс, который владеет набором элементов одного типа и предоставляет единообразный интерфейс для их хранения и доступа к ним.

- Владение и время жизни элементов — на контейнере.
- Чтобы работать с данными (перебирать, искать, сортировать), нужно универсальное средство доступа — итератор.

Требования к контейнеру, чтобы с ним можно было работать через итераторы

- Метод `begin()` — возвращает итератор на первый элемент
- Метод `end()` — возвращает итератор за последним элементом (sentinel)
- Метод `size()` — возвращает количество элементов (для sized-контейнеров)
- Типовые алиасы:

```
1 using value_type = T;
2 using size_type = std::size_t;
3 using iterator = Iterator<T>;
4 using const_iterator = ConstIterator<T>;
```

17.2.2. Итераторы

Итератор — это объект, который «указывает» на элемент контейнера. Он обобщает работу с разными типами контейнеров (массив, список, дерево и т.п.).

Требования к итератору

- Типовые алиасы

```
1 using value_type = T;  
2 using reference = T&;  
3 using pointer = T*;  
4 using difference_type = std::ptrdiff_t;  
5 using iterator_category = ..._tag; // (см. ниже)
```

17.3. Категории итераторов

В стандарте C++20 для каждой категории итераторов появились специальные концепты (например, `std::input_iterator`).

17.3.1. `output_iterator`

Операции

- `*it = value;` (разыменование как `lvalue`)
- `++it, it++`

Особенности

- Итератор однократный
- Может не поддерживать `==/!=`

17.3.2. `input_iterator`

Операции

- Чтение `*it`
- `++it, it++`
- Сравнение `==/!=`
- `it->member`

Особенности

- Итератор однократный. Повторный проход по тому же контейнеру может дать другой результат (например, хеш)
- Чтение может давать временное значение

17.3.3. `forward_iterator`

Операции

- Всё из `input_iterator`

Особенности

- Многопроходность (копии независимы, можно итерироваться несколько раз)

17.3.4. `bidirectional_iterator`

Операции

- Всё из `forward_iterator`
- `--it, it--`

Особенности

- Можно ходить в обе стороны (списки, деревья)
- Дополнительно нужны `rbegin()`, `rend()` в контейнере

17.3.5. `random_access_iterator`

Операции

- Всё из `bidirectional_iterator`
- `it += n`, `it -= n`
- `it + n`, `n + it`, `it - n`
- `it2 - it1`
- `it[n]`,
- Сравнения `<`, `<=`, `>`, `>=`

Особенности

- Произвольный доступ за $O(1)$
- Как «указатель на массив»

17.3.6. `contiguous_iterator`

Операции

- Всё из `random_access_iterator`

Особенности

- Элементы лежат смежно в памяти (как у «сырых» указателей)
- Допустимо получать «сырой» адрес: `std::to_address(it)`
- Даёт права на побайтные операции (memcpy и т.п.) при соблюдении правил типов

17.3.7. Почему нельзя создать универсальный итератор?

Потому что разные контейнеры имеют разную логику доступа:

- У `vector` — прямой доступ
- У `list` — только последовательный

Поэтому выбирается минимально необходимая категория и реализуется соответствующее поведение.

17.4. Цикл `for` для работы с контейнерными объектами

17.4.1. Два подхода к итерации по контейнеру

Явный цикл с итераторами

```
1 for (auto it = v.begin(); it != v.end(); ++it) {
2     std::cout << *it << '\n';
3 }
```

Используется, когда нужен контроль:

- Можно модифицировать элементы через `*it`

- Можно работать с `it`, делать `++`, `--`, `+n`, `-n` и т.п.
- Можно сохранить `it` для будущего использования

Range-based for (диапазонный **for**)

```
1 for (auto elem : c) {  
2     // читаю, НО копирую каждый элемент  
3 }  
4 for (auto& elem : c) {  
5     // читаю и МЕНЯЮ элементы  
6 }  
7 for (const auto& elem : c) {  
8     // только читаю, без копий  
9 }
```

17.4.2. Требования к контейнеру для работы с range-based **for**

- Контейнер должен иметь `begin()` и `end()` (или `std::begin()` и `std::end()` для массива)
- Эти функции должны возвращать итераторы
- Итераторы должны быть `input_iterator` или выше

18. Структурное программирование: нисходящая разработка, использование базовых логических структур, сквозной структурный контроль.

Структурное программирование — метод разработки программ, в котором:

- код организуют в небольшие логически завершённые части (модули, функции);
- управляют потоком исполнения только с помощью ограниченного набора базовых конструкций: последовательность, ветвление, цикл;
- избегают произвольных переходов (**goto**);
- стараются упростить передачу данных между модулями и сделать каждый модуль «чёрным ящиком» с понятным интерфейсом.

Главная цель — улучшить читаемость, проверяемость и надёжность программы.

18.1. Этапы разработки ПО

18.1.1. Анализ

Цель

- Полная формализация задачи и подготовка технического задания: входы/выходы, ограничения, сценарии использования.

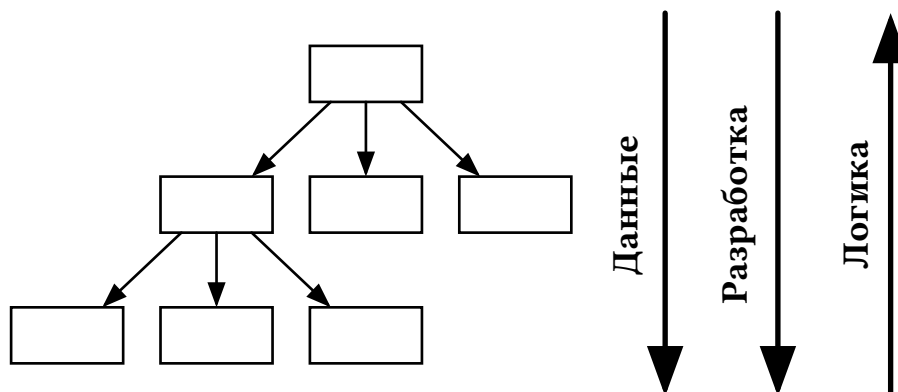
Что нужно сделать

- Перечислить все нюансы и «подводные камни», даже очевидные
- Растолковать требования в терминах, по которым можно тестировать корректность.

Оценка ресурсов

- Оценить время, число разработчиков и материальные ресурсы.

18.1.2. Проектирование (нисходящая разработка)



Идея

- Разбиваем систему на крупные подзадачи, затем каждую — на более мелкие уровни, пока не получим простейшие функции.
- Проектирование идёт сверху вниз.

Чёрный ящик

- При делении учитываем только интерфейсы модулей (входы/выходы, пред/постусловия) — детали реализации скрыты.
- Это даёт независимость разработки и возможность заменять реализации.

Логика — снизу вверх

- На нижних уровнях должны быть простые, детерминированные функции
- Основная логика системы отлаживается на верхних уровнях, при ранней стадии возможна фиксация логических ошибок.

Заглушки

- Для тестирования верхних уровней используют заглушки вместо ещё не реализованных подзадач
- При реализации модуля заглушку заменяют реальной реализацией.

Данные и контрольные точки

- Разбивка идёт и по действиям, и по представлению данных — каждый уровень абстракции служит контрольной точкой, где можно показать заказчику промежуточный результат.

18.1.3. Кодирование

Итеративность проектирования/кодирования/тестирования

- Проектирование, кодирование и тестирование на практике сливаются: спроектировали подзадачу → написали реализацию → протестировали → перешли к следующей.
- Это естественно при нисходящем подходе с заглушками.

Иерархия и простые требования к модулю

- Каждая подзадача должна иметь чёткое, простое условие (одно предложение без перечислений и оборотов).
- Это уменьшает двусмысленность и снижает риск дублирования логики в разных модулях.

Ограничение зависимостей (правило «не более 7 подчинённых»)

- У модуля не должно быть более 7 зависимых модулей — это даёт нужную широту, не перегружая уровень абстракции и увеличивая количество контрольных точек.

Глубина вложенности

- Максимальная рекомендуемая глубина вложенных управляющих конструкций — 3.
- Если нужен дополнительный уровень, выделяй подфункцию.
- Это облегчает понимание и тестирование.

Подход «полного недоверия» к входным данным

- Функция при получении данных сначала проверяет, может ли с ними работать.
- Если нет — возвращает ошибку/статус и не изменяет входные данные.
- Решение о дальнейшем действии принимает более высокий уровень.

Явная передача данных

- Все данные передаются через список параметров (включая константы, которые могут стать переменными).
 - Исключения — редкие константы вроде числа π , оформленные как глобальная константа.
- Стремиться ограничивать количество параметров до 3 (кроме самых низкоуровневых функций).
 - Если данных больше — объединять их в явно названную структуру.
- Если функция использует 1–2 поля из большой структуры — не передавать всю структуру, а выделить подструктуру. Это сохраняет уровень абстракции.

Разбиение данных по категориям

- Входные — инициализированы
- Изменяемые — инициализированы
- Выходные — не инициализированы.
- Функция при ошибке не должна модифицировать данные.

Управление ресурсами

- Функция не должна захватывать ресурсы, если заранее понятно, что не сможет корректно их вернуть
- Владение ресурсом должно быть атомарным и определённым (ответственность за освобождение — у владельца).

Запрещённые практики

- Никаких `abort()` и `exit()` в рабочем коде — только корректный возврат ошибок и обработка на верхних уровнях.
- `assert()` — только в отладочных сборках. Не рассчитывать на него в релизе. Для проверок на этапе компиляции использовать статические проверки (например, `static_assert`).

Переиспользуемость

Низкоуровневые функции — простые и детерминированные, они легко переиспользуются в других проектах.

18.1.4. Тестирование

Тестовые данные готовятся заранее

До начала массовой реализации формируются тестовые случаи для модулей — чтобы не «подгонять» тесты под код.

Тестирование как неотъемлемая часть итерации

Для каждой спроектированной подзадачи: написать реализацию → прогнать модульные тесты (часто вместе с заглушками для соседних модулей) → при успешном результате заменить заглушку реальным модулем и повторить интеграционные проверки.

Контроль корректности интерфейсов

Тесты проверяют не только поведение, но и соблюдение контрактов: функция не меняет входные данные при ошибке, не захватывает ресурсы без гарантии их возврата, возвращает корректный статус/результат.

18.2. Использование базовых логических структур

18.2.1. Последовательность

Это просто выполнение команд одна за другой — без ветвлений и повторов. Любая программа содержит последовательности.

18.2.2. Ветвление

Позволяет выбрать один из путей исполнения в зависимости от условия. Стандартные формы: `if`, `if ... else`, `switch`.

18.2.3. Цикл

Повторение блока команд пока выполняется условие (или заданное число раз). Основные виды: `while`, `do ... while`, `for`, `while (true)`.

- `goto` использовать нельзя.
- Выход из цикла всегда один (без `break` или `return`).
- В цикле должна быть только повторяющаяся часть.

18.3. Сквозной структурный контроль

18.3.1. Проблема традиционного управления

- Руководитель в классическом подходе теряет квалификацию, так как не участвует непосредственно в создании продукта, а программисты развиваются.
- Контроль выполнять сложно: руководитель становится «узким местом» для контроля качества.

18.3.2. Идея

Перенести ответственность за поиск ошибок с менеджера на команду: программисты сами проводят контрольные сессии, где коллективно ищут дефекты до релиза. Руководство не участвует — задача сессии не оценка, а вылов ошибок.

18.3.3. Что это такое

Цель

Найти логические ошибки, несоответствия ТЗ, проблемные места архитектуры и интерфейсов на ранних этапах.

Участники

Автор кода, независимые разработчики из той же группы (и лучше — из соседних команд), представитель заказчика/тестировщик при необходимости.

Запрет

Оценивание/наказание в сессии недопустимы — это снижает эффективность.

18.3.4. Правила и преимущества контрольных сессий

- Каждая сессия короткая, анализируется только небольшой фрагмент работы.
- Составляются плакаты/доски с замечаниями, чтобы коллеги могли в дальнейшем использовать удачные решения и помнить про найденные ошибки.
- Коллективное обсуждение порождает рост квалификации, распространение лучших приёмов, быструю локализацию дефектов и делает процесс программирования прозрачным для команды.

18.3.5. Проблема снижения внивания

- Люди привыкают, перестают тщательно проверять.
- Решения: ротация участников, ограничение длительности сессий, периодические внешние инспекции.

19. Преимущества и недостатки структурного программирования. Идеи Энтони Хоара. Преимущества и недостатки объектно-ориентированного программирования

19.1. Преимущества структурного программирования

19.1.1. Исправление логических ошибок на ранних стадиях

- Самые серьёзные логические ошибки устраняются на ранних этапах разработки благодаря тому, что на низших уровнях (исполнительные функции) код простой и не содержит сложной логики.
- Основная задача разбивается на подзадачи, которые можно тестировать с основной задачей (обеспечивается комплексное тестирование без необходимости писать тесты для каждой функции отдельно).

19.1.2. Комплексная отладка и быстрое начало кодирования

- Не требуется поэтапное завершение всех стадий проектирования: можно почти сразу приступить к кодированию, что удобно для программиста.
- Комплексная отладка: не нужно полностью реализовывать проект, чтобы тестировать отдельные части.

19.1.3. Чёткая структуризация и иерархия кода

- Разделение задачи формирует иерархическую структуру с уровнями абстракции; каждая функция небольшая и легка для понимания.
- Много естественных контрольных точек, за выполнением которых легко наблюдать, и ошибки легко локализовать.

19.1.4. Удобное распределение задач и командная работа

- Благодаря четкой структуре задач удобно делегировать отдельные блоки разным программистам.
- Простота контроля: легко следить за выполнением отдельных частей проекта.

19.1.5. Тесное взаимодействие с заказчиком

- С самого начала разработки постоянно происходит контакт с заказчиком по поводу контрольных точек, что снижает риск несоответствия результата ожиданиям.

19.1.6. Минимизация «неработающего» кода

- Практически нет кода, который идет «в корзину» — весь написанный код участвует в рабочем продукте.

19.1.7. Плавное распределение ресурсов

- В конце проекта приходится писать много кода, но он остается простым — нет авральных ситуаций или перегруза по сложности.

19.1.8. Простой и читабельный код

- Код должен быть прозрачным, понятным, красивым и легко читаемым — если что-то не понятно, лучше переписать.

19.1.9. Ведение собственных библиотек

- Со временем у программиста (и фирмы) накапливаются готовые, хорошо структурированные библиотеки функций, которые можно переиспользовать.

19.2. Недостатки структурного программирования

19.2.1. Трудности масштабирования

- Для больших программ этот подход становится крайне тяжёлым, особенно для систем с графикой или базами данных — категорически не рекомендуется использовать структурный подход.

19.2.2. Проблемы с модификацией/доработкой

- Любое изменение данных или появление новых требований ведёт к необходимости значительных изменений по всему коду, поскольку обработка новых данных требует искать и менять множество мест.
- Из-за необходимости добавлять новые поля или функционал структура иерархий разрушается.

19.2.3. Снижение надёжности кода при изменениях

- Любая доработка или изменение ранее написанного кода неизбежно снижает его надёжность — даже если код простой, накопление изменений ведёт к возникновению «мин замедленного действия» (скрытых ошибок).
- После некоторого количества доработок проект становится плохо читаемым, ненадёжным, возникает «spaghetti code», который невозможно сопровождать.

19.2.4. Проблемы с дальнейшими модификациями

- После нескольких итераций изменений становится проще написать программу заново, чем продолжать модифицировать старую версию.
- Программа «теряет чистоту», становится сложной, абстракции размываются.

19.2.5. Сложности с эксплуатацией

- Программа, хоть и прошла контроль и сдачу, после запуска у заказчика часто выявляет множество неудобств и недостатков, требующих серьёзной переделки — чаще всего изменения оказываются более трудоёмкими, чем сам изначальный проект.

19.2.6. Не годится для курсовых и длительных проектов

- Не подходит для курсовых работ и крупных многоуровневых систем, где требования могут меняться и важно легко модифицировать структуру программы.

19.3. Идеи Энтони Хоара

Хоар обратил внимание на то, что при изменении данных приходится «ползать» по всему коду, внося множество правок, что неудобно и опасно для надёжности программы.

19.3.1. Идея инкапсуляции

- Суть: Чётко выделить данные и определить «что с ними можно делать».
- Выделение набора допустимых операций для определённых данных — это и есть первая ключевая идея, позднее названная «инкапсуляцией».
- Операции должны работать с данными, а к самим данным доступ ограничен (изменение данных должно происходить только через специальные методы).
- В языке Си уже был такой подход в реализации библиотеки потоков (структура FILE и операции с ней, сами поля не открываются напрямую).
- Таким образом, предлагается абстрагироваться от деталей внутреннего устройства данных и работать только через разрешённые действия.

19.3.2. Идея наследования

- Суть: При необходимости расширения данных и/или функций не менять основной (уже написанный и отлаженный) код, а строить над ним надстройки.
- Добавлять новые поля и новые действия через расширение — не «трогая» уже существующую рабочую реализацию.
- Такая технология уменьшает риск ошибок и потери надёжности — иначе каждое изменение существующего кода потенциально портит систему.

19.3.3. Идея полиморфизма

- Суть: Возможность работы с обобщёнными операциями, безразличие к конкретной реализации.
- В местах, где раньше обрабатывали один тип данных и набор операций, можно подменять его на другой, если он соответствует необходимому интерфейсу.
- Позволяет расширять функциональность и гибко заменять части программы.

19.3.4. Идея моделирования программы как множества объектов и их взаимодействий

- Хоар предложил рассматривать программу как совокупность объектов, которые взаимодействуют между собой, аналогично объектам реального мира.
- Взаимодействие объектов бывает двух видов:
 - 1) Акцессорное (синхронное)
Получение информации или свойств объекта (getter, setter).
 - 2) Событийное (асинхронное)
Объекты меняют своё состояние в ответ на внешние события (например, переход состояний у конечного автомата).

19.4. Преимущества объектно-ориентированного программирования

19.4.1. Лёгкость внесения изменений

- Возможность модифицировать и расширять программу без изменения уже написанного и отлаженного кода.
- Исключение необходимости «перелопачивать» весь проект.

- Это повышает надёжность и снижает вероятность случайных ошибок, связанных с изменением старого кода.

19.4.2. Код становится гибким

- Появляется возможность реализовывать разные варианты поведения объектов (через полиморфизм), работать с обобщёнными интерфейсами.
- Позволяет легко заменять одни реализации на другие без необходимости переписывать внешний код (тот, что работает с объектами через их интерфейс, не зная подробностей реализации).

19.4.3. Повторное использование кода

- Создаётся новая функциональность путём расширения и доработки существующих классов (наследование), что уменьшает дублирование и увеличивает повторную используемость компонентов.

19.4.4. Уменьшение связанности модулей

- Инкапсуляция данных и методов внутри объектов позволяет ограничить внешнее взаимодействие с объектом только необходимым интерфейсом, что снижает связанность между частями программы и минимизирует объём данных, передаваемых между объектами.

19.5. Недостатки объектно-ориентированного программирования

19.5.1. Необходимость длительного анализа и проектирования

- Для перехода к кодированию сначала обязательно нужно разработать начальную объектную модель — сделать «слепок» предметной области и всех объектов, формализовать связи и отношения.
- Без хорошей модели к программированию переходить нельзя, иначе система будет плохой архитектурно. Это требует времени и высокой квалификации.

19.5.2. Высокий порог вхождения

- Освоение ООП требует другого уровня абстракции мышления: не всем студентам и даже программистам со стажем удаётся быстро овладеть этой технологией.
- Бывают случаи, когда понимание сути ООП приходит только через годы практики, а иногда — вовсе не приходит.

19.5.3. Значительный рост объёма кода

- Код в рамках ООП становится заметно более объёмным по сравнению со структурным подходом.
- Особо очевидно это становится при выполнении лабораторных работ и небольших учебных проектов: программа принципиально увеличивается «на порядок».

19.5.4. Увеличение времени разработки и компиляции

- Из-за роста кода увеличивается время написания программы и время её компиляции.

- Эффективность этапов разработки снижается при стандартной, неотлаженной схеме проектирования.

19.5.5. Проблема «мёртвого кода»

- При наследовании и перекрывании методов в базовых классах образуется много кода, никогда неиспользуемого (мёртвого).
- В языке C++ весь этот код компилируется и занимает память, снижая производительность.
- В современных технологиях и языках частично решается с помощью динамических библиотек и компиляции «на лету», но в классическом C++ проблема остаётся.

19.5.6. Рост объёма памяти и замедление программы

- Из-за особенностей реализации полиморфизма, наследования и дополнительных структур требуется больше оперативной памяти.
- Время выполнения программы нередко увеличивается.

20. Основные понятия ООП: инкапсуляция, наследование, полиморфизм. Понятие объекта. Категории объектов. Отношения между объектами. Понятие класса. Отношения между классами. Понятие домена.

20.1. Основные понятия ООП

Инкапсуляция — это объединение данных и действий над ними с работой только через эти действия.

Прямой доступ к полям возвращает к проблеме структурного подхода: меняются данные — нам надо «перелопатить» всю программу.

Наследование — это надстройка над уже работающим кодом без его изменения.

- Написано, отлажено — не трогай; надо изменить — сделаем надстройку, добавим поля, подменим действия.
- Любое изменение написанного кода «это понижение надёжности», плюс затраты на повторное вникание и риск «заложить мину замедленного действия».

Полиморфизм — это подмена одного другим, не изменяя написанный код.

- Это понятие неразрывно связано с наследованием.
- «безразличие» к форме: там, где использовалось исходное объединение данных и действий, допускается подмена на расширенное без изменения кода в этом месте.

20.2. Объект

20.2.1. Понятие объекта

Объект — это конкретная реализация какого-то абстрактного типа, обладающая тремя характеристиками: состояние, поведение и индивидуальность.

Состояние — это один из возможных вариантов существования объекта (например, студент может находиться в состоянии сна).

Поведение — описание объекта в терминах изменения его состояния.

Индивидуальность — отличительные особенности, отличающие объект от других.

20.2.2. Модель состояний объекта

На основе возможных состояний объекта строим жизненный цикл (или модель состояний объекта). Для описания модели состояния используется модель Мура:

- 1) Множество состояний.
- 2) Множество инцидентов (событий), которые могут приводить к изменению состояния.

- 3) Правила перехода, определяющие новое состояние объекта при возникновении события.
- 4) С каждым состоянием связывается действие состояния, задача которого — перевести объект в это состояние.

20.2.3. Категории объектов

1) Реальные объекты

Объекты, присутствующие в физическом мире. С них начинается формализация.

2) Роли

Абстракции целей или назначений (студент, сотрудник). Один физический объект в реальной жизни может выполнять много ролей (студент, член семьи, сотрудник) — но в программе у объекта допустима только одна роль.

3) Инциденты (события)

Абстракция чего-либо происшедшего (наводнение, выборы). Особенность — такие объекты проходят стадии и приходят к заключительному состоянию.

4) Объекты взаимодействия

Объекты, возникающие из отношений других объектов (перекрёсток, договор). Большинство объектов при формализации оказываются именно объектами взаимодействия.

5) Спецификации

Представляют стандарты, критерии, правила, графики (расписание движения поездов, расписание занятий). Обычно пассивные объекты: информируют, но не действуют.

20.2.4. Отношения между объектами

- Отношение использования (старшинства). Объекты вступают в трёх ролях:
 - Воздействующие (активные) — объекты воздействуют на другие, но сами не подвергаются воздействию.
 - Исполнители (пассивные) — объекты, подвергаемые воздействию, но сами никогда не воздействуют.
 - Посредники — объекты, которые могут выступать как в роли воздействующих, так и в роли исполнителей. Создаются в интересах активных объектов или других посредников.
- В программе рекомендуется предпочитать включение (композицию), потому что оно обеспечивает целостность объекта и защиту от окружающего мира. Объект может включать в себя другие объекты.

20.3. Класс

20.3.1. Понятие класса

Класс — это абстракция множества предметов реального мира, у которых:

- 1) одинаковые характеристики,
- 2) общее поведение.

Если какой-то объект имеет характеристику, отличающуюся от других, или поведение отличается, значит это объект другого класса или поведение требует пересмотра структуры классов.

20.3.2. Отношения между классами

- 1) Наследование — одно из ключевых отношений между классами.
- 2) Использование — когда один класс использует другой.
 - Например, в теле одного класса могут быть методы, использующие объекты других классов.
- 3) Включение — один класс определяется внутри другого (как вложенный).
 - Пример: итератор может быть определён как внешний класс (и использоваться в другом классе), либо определён внутри класса для изоляции.
- 4) Метаобъекты и метаклассы в современном ООП не используются.

20.4. Понятие домена

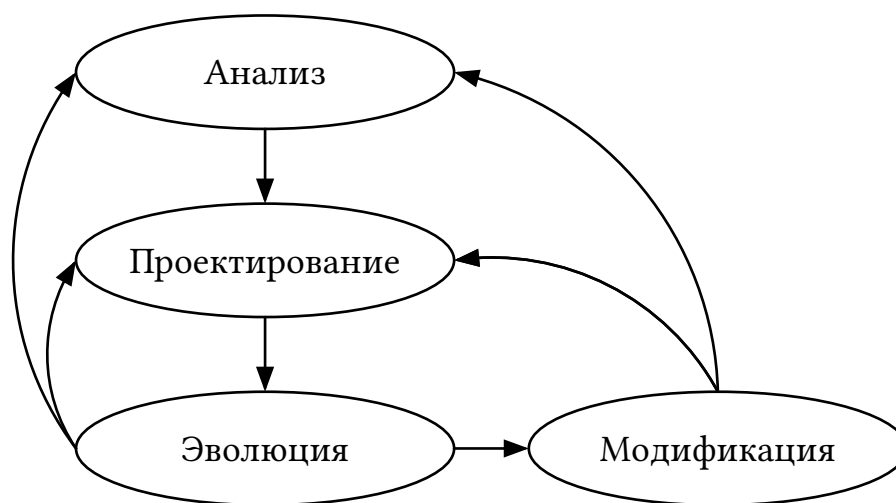
Домен — это отдельный реальный или гипотетический мир, населённый отчётливым набором объектов, которые ведут себя в соответствии с характерными для домена правилами и линиями поведения.

- Для каждого домена можно использовать собственную технологию разработки.
- Наличие объекта или класса в одном домене не требует их наличия в других доменах.
 - Это важно при проектировании: для каждого домена своя структура и свои принципы.

Пример: формы, окна, элементы интерфейса — компоненты одного домена, прикладные задачи и их объекты — другого.

21. Цикл разработки ПО с использованием ООП: анализ, проектирование, эволюция, модификация. Рабочие продукты объектно-ориентированного анализа.

21.1. Цикл разработки ПО с использованием ООП



21.1.1. Анализ

- В ООП задача этапа анализа — формализовать задачу с помощью объектно-ориентированного подхода, то есть построить объектную модель.
- На этом этапе выделяются сущности, их атрибуты, строится объектная модель. Этот процесс может быть долгим и тяжёлым.
- Для помощи в анализе разрабатываются специальные документы по определённым правилам. Такой подход формализации задачи важен не только для программиста, но и для заказчика.
- Заказчики во время такого анализа часто делают для себя открытия, поскольку начинают более осознанно формулировать, какие бывают связи между сущностями, о чём ранее не задумывались.
- Документы анализа важны также для заказчика, так как там фиксируются все выявленные сущности, связи и атрибуты.

Когда заканчивается анализ?

Когда выделены базовые абстракции, можно переходить к следующему этапу. Если базовые абстракции не выделены, это в дальнейшем может привести к очень серьёзным проблемам.

21.1.2. Проектирование

- Основной объём работы на этом этапе — перевод документов, которые были получены во время объектно-ориентированного анализа, в проектные документы.
- Документы из анализа в проект переводятся не всегда напрямую «один в один», иногда требуется использовать специальные преобразования.
- Чтобы перевести документы из анализа в проектные документы, применяются разные паттерны проектирования.

21.1.3. Эволюция

- Эволюция – это развитие программы.
- Для разработки ПО в ООП используется РД-подход (рекурсивный дизайн) — программа развивается постепенно: начинаем писать с абстракций, потом добавляем сущности.
- На этапе эволюции, если возникают какие-то проблемы, можно вернуться к проектированию и анализу. Этот процесс итеративный, нет жёсткого «один раз сделал — и всё».
- В структурном программировании сдача проекта происходит только после написания всего кода, когда функционал полностью реализован. В ООП — система постепенно становится рабочей уже на ранних этапах, нет необходимости делать «заглушки».

Какие изменения могут быть реализованы на этом этапе?

1) Добавление класса

Совершенно безболезненная вещь, не приводит к изменению написанного кода, основной плюс ООП.

2) Изменение реализации класса

Делается за счёт наследования или паттернов (например, мост, стратегия, декоратор); иногда нужно подменить одну сущность на другую. Это, как правило, не требует изменений в уже написанном коде.

3) Изменение представления об объекте/сущности

Например, появляются новые обязанности или роли; делается, например, через адаптер.

4) Реорганизация структуры классов

Более сложное изменение, может потребовать вмешательства в написанный код, обычно связано с устранением дублирования кода, изменением иерархий, выведением функционала на уровень базовых абстракций, использованием паттернов.

5) Важный принцип: категорически нельзя менять базовые абстракции

Это может привести к изменению большого объёма кода. Именно поэтому анализ должен быть закончен только тогда, когда выделены все базовые абстракции.

21.1.4. Модификация

- После сдачи проекта заказчику цикл не заканчивается: следующий этап — поддержка, сопровождение, а главная задача — модификация.
- Технология ООП создавалась именно ради упрощения модификации — то есть чтобы продлить жизненный цикл программного продукта.
- В ООП этап модификации включён прямо в цикл разработки, и к этому этапу нужно подходить так же основательно, как к анализу.
- Ключевое отличие: эволюция — это развитие до сдачи, модификация — уже после сдачи, но технически они ничем не отличаются, используется тот же «рекурсивный дизайн».
- По видам изменений — на этапе модификации применяются те же подходы, что и при эволюции.

21.2. Рабочие продукты объектно-ориентированного анализа

21.2.1. Схема доменов и проектная матрица

- Любая задача разбивается на домены, между ними строятся связи.
- Составляется схема доменов для всей программы.
- Формируется проектная матрица, которая необходима для отслеживания выполнения проекта. Она заполняется в течение всей работы над проектом.

21.2.2. Модель (или диаграмма) «сущность-связь»

- Начинают с выделения сущностей — обычно сначала переносят объекты из физического мира в программу.
- Строится информационная модель, также называемая диаграммой «сущность-связь» (ER-диаграмма), на которой изображаются сущности и связи между ними.

21.2.3. Описание сущностей и их атрибутов

- Для каждой сущности выделяются и описываются характеристики, абстрагируются как атрибуты.
- Атрибуты разбиваются на группы:
 - 1) идентифицирующие (формируют идентификатор),
 - 2) вспомогательные,
 - 3) описательные.
- Для каждого атрибута фиксируется:
 - какую характеристику он представляет,
 - зачем его выделяют,
 - откуда берётся значение,
 - кто может изменять значение и к каким последствиям это приводит.
- Существуют отдельные правила для работы с атрибутами, например:
 - Атрибут всегда имеет значение.
 - Если появляется структура — выделяется новая сущность.
 - Если нет значения — это, возможно, другое объектное множество.

21.2.4. Описание связей между сущностями

- Выделяются и формализуются связи между сущностями, фиксируется, «кто отвечает» за каждую связь.
- Связи классифицируются по типу (один-к-одному, один-ко-многим, многие-ко-многим), по обязательности (условные/безусловные), приводятся примеры на бытовых и учебных ситуациях.

21.2.5. Модели поведения, состояния и событий

- Для каждого объекта создаётся модель поведения: выделяют состояния, в которых может находиться объект, и описывают, при каких событиях происходит переход между состояниями.
- Строятся диаграммы переходов состояний.

- Описываются события, которые могут быть внутренними или внешними по отношению к сущности. Для подсистем создаётся модель взаимодействия на основе событий.

21.2.6. Диаграммы потоков данных и процессов

- Для каждого действия разрабатывается диаграмма потоков данных, описывающая не только алгоритм, но и какие данные куда переходят.
- Составляется «таблица процессов» — перечень элементарных (атомарных) действий, указывается в каких сущностях и подсистемах реализуется каждый процесс.
- Особо выделяются аксессорные процессы (доступ к данным, чтение/запись) — на их основе строится диаграмма доступа.

21.2.7. Диаграммы взаимодействия и доступа

- Диаграмма взаимодействия объектов (событийная модель), в том числе взаимодействие между подсистемами.
- Модель доступа — синхронная схема, описывающая, как подсистемы обращаются друг к другу аксессорно (через методы).

22. Концепции информационного моделирования. Понятие атрибута. Типы атрибутов. Правила атрибутов. Понятие связи. Типы связей. Формализация связей. Композиция связей. Подтипы и супертипы. Диаграмма сущность-связь.

22.1. Концепции информационного моделирования

Информационное моделирование — это процесс выделения сущностей и абстракций, с которыми мы работаем в рамках домена или подсистемы.

- **Задача информационного моделирования** — определить, какие сущности реально существуют в физическом мире, выделить их абстракции и характеристики (атрибуты), а также связь между этими сущностями.
- Для домена или подсистемы строится информационная модель.

22.1.1. Диаграмма «сущность-связь»

- На этой диаграмме отображаются атрибуты (характеристики сущностей) и сами сущности, а также связи между ними.
- Каждая сущность на диаграмме записывается в прямоугольнике.
- Каждой сущности присваивается:
 - 1) Уникальный номер.
 - 2) Имя (например, «Студент»).
 - 3) Ключевой литерал — создаётся из имени сущности, обычно 1-3 буквы из её названия.
- Для каждой выделенной сущности и каждого выделенного атрибута должно быть ясно, зачем они вводятся и какую характеристику физического объекта описывают.

22.1.2. Один из критериев корректности информационного моделирования

Если объект (в физическом мире), который мы рассматриваем, не имеет какой-либо выделенной характеристики, то, возможно, он не относится к данной сущности.

22.2. Атрибуты

22.2.1. Понятие атрибута

Атрибуты — это выделяемые характеристики сущностей, по которым их можно отличить и описать.

22.2.2. Типы атрибутов

Идентифицирующие

- С помощью которых объект однозначно идентифицируется (выделяется среди других).
- Среди идентифицирующих атрибутов выделяют привилегированный идентификатор (основной), который помечается звёздочкой (*).

Описательные

- Любая характеристика объекта (например, вес, рост студента и т.д.).
- Это то, что хранит дополнительную информацию о предмете.

Вспомогательные

- Могут объединять две характеристики или использоваться для формализации связей и состояний.
- Пример: статус (состояние) объекта.

22.2.3. Правила атрибутов

- 1) Каждый атрибут должен иметь значение в любой момент времени.
 - Если какой-то объект сущности не имеет значения для определённого атрибута, значит, он не относится к этой сущности.
- 2) Атрибут должен быть простым (не содержать внутренней структуры).
 - Если атрибут сложный — это отдельная сущность, нужно вынести её отдельно.
- 3) Если атрибут — идентификатор или часть идентификатора, он должен характеризовать объект целиком, а не часть объекта.
 - Например, ФИО относится к студенту в целом, а не к его частям.
- 4) Описательные атрибуты тоже должны характеризовать объект целиком, а не какие-то его под-объекты или внешние сущности.
 - Важно не смешивать характеристики — не должно быть «классов-помоек».

22.3. Связи

22.3.1. Понятие связи

- Связи определяют взаимодействие между объектами, описываемыми сущностями.
- После выделения сущностей и их атрибутов обязательно выявляются связи между этими сущностями.

22.3.2. Типы связей

Мощность связи

- 1) Один к одному ($1 : 1$)
 - Пример: муж и жена (один муж — одна жена).
- 2) Один ко многим ($1 : N$)
 - Пример: руководитель и студенты. Один руководитель может руководить несколькими студентами, но у каждого студента только один руководитель.
- 3) Многие ко многим ($M : N$)
 - Пример: студенты и учебные курсы. Один студент может посещать несколько курсов, и один курс могут слушать несколько студентов.

Условность связи

- 1) Безусловная связь — объект обязательно должен участвовать в связи (например, муж и жена).

- 2) Условная связь — объект может не участвовать (например, не у каждого преподавателя есть курсовики).
- 3) Биусловная связь — это связь, где и с одной, и с другой стороны объект может не принимать участие в связи.

22.3.3. Формализация связей

Формализация связи один к одному (1:1)

- Достаточно добавить вспомогательный атрибут с идентификатором другой сущности.
- Пример: для связи «муж-жена» (1 : 1) у жены может быть атрибут «ID мужа».
- Также важно определить, кто отвечает за связь — кто «держит» связь, то есть с чьей стороны она реализуется.

Формализация связи один ко многим (1:N)

- Связь формализуется со стороны «многих».
- В таблице (или списке) «многих» добавляется идентификатор (ключ) «одного».
- Пример: у каждого студента есть атрибут «ID руководителя».
- Это важно: «формализуем со стороны студента».

Формализация связи многие ко многим (M:N)

- В этом случае простого вспомогательного атрибута недостаточно.
- Необходимо добавить промежуточную (ассоциативную) сущность (или объект).
- Эта сущность содержит идентификаторы двух связанных объектов — формирует составной идентификатор.
- Такой объект может иметь свои собственные атрибуты, характеризующие саму связь (например, количество часов курса для студента).

Пример на «студент-учебный курс»:

Ассоциативная сущность (например, «Посещение» или «Запись на курс»), ключ — из двух идентификаторов: студента и курса, могут добавляться дополнительные атрибуты (например, число часов по курсу для данного студента).

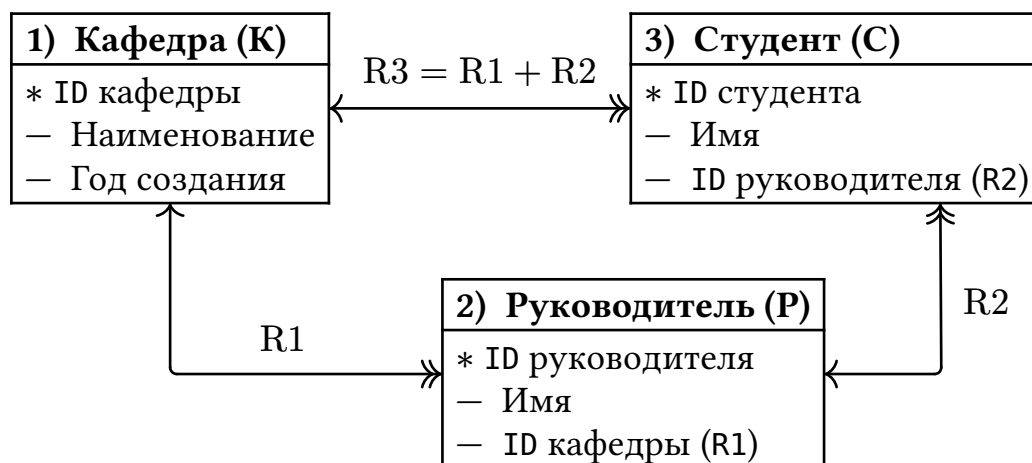
Формализация «динамических» связей

- Если связь обладает динамическим поведением (например, может изменяться во времени, требует дополнительного управления), её всегда нужно выделять отдельной ассоциативной сущностью (объектом).
- Пример: если жизненные циклы объектов связаны (изменение состояния одного должно влиять на другой), нужен отдельный контролирующий объект (например, «куратор», если речь о распределении студентов по руководителям).

22.3.4. Композиция связей

- Если в нашей системе возникает цикл (кольцо) безусловных связей между сущностями, одну из таких связей делают композитной, то есть выражают её через другие и не формализуют как отдельную.

- Для этого добавляется вспомогательный атрибут, который выражает нужную композицию связей и разрывает кольцо.



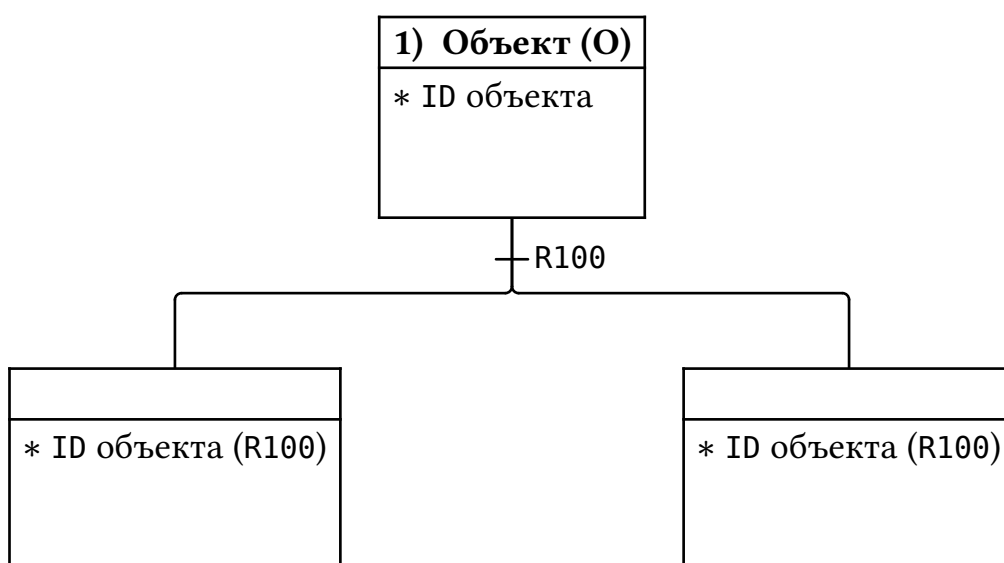
22.4. Подтипы и супертипы

- Когда мы выделяем различные сущности, может оказаться, что разные сущности имеют общие характеристики (атрибуты).
- В таких случаях эти характеристики необходимо вынести на более высокий уровень абстракции — объединить в суперкласс (супертип).

Суперкласс — это абстракция, которая отражает нечто общее, чего реально не существует в физическом мире как отдельного объекта.

- На уровень суперкласса выносятся только те характеристики, которые действительно являются общими для всех подчинённых типов (подтипов).

Подтипы (или подклассы) — это конкретные сущности, которые наследуют характеристики суперкласса и добавляют к ним свои специфические атрибуты или свойства.



23. Модель поведения объектов. Жизненный цикл и диаграмма перехода в состояния (ДПС). Виды состояний. События, данные событий. Действия состояний. Таблица перехода в состояния (ТПС). Правила переходов.

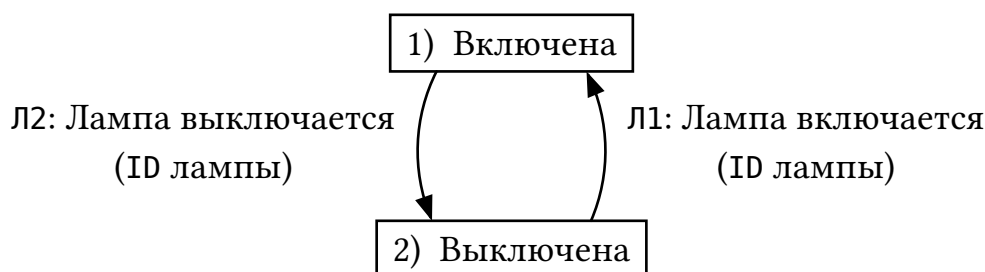
23.1. Модель поведения объектов. Жизненный цикл

- Модель поведения — это описание того, как конкретный объект изменяет свои состояния во времени, представляемое как конечный автомат.
- Жизненный цикл фиксирует возможные состояния и переходы, а ДПС — графическая схема этих переходов и состояний для анализа системы на уровне объектов.
- Жизненный цикл объекта бывает двух основных типов: циклический и «рождение–смерть» с возможными повторными появлениями в системе
- Для описания жизненного цикла используется подход «конечного автомата» и применяется модель Мура как основа формализации на ДПС.
 - В модели Мура перечисляются «множество состояний», «множество событий» и «правила перехода»
 - Также добавляется связь «действие–состояние», то есть «обработчик состояния» как часть формализации поведения.

23.1.1. Для кого формируется жизненный цикл

- Для физических объектов, у которых можно выделить осмысленные состояния в процессе существования.
- Для «запросов» и «задач», поскольку они обрабатываются и меняют своё состояние по ходу обработки.
- Для ассоциативных объектов, если связь имеет динамическое поведение и отслеживается через ассоциативный объект.
- Для пассивных объектов — чтобы включать их в асинхронные схемы взаимодействия с активными объектами.

23.2. Диаграмма перехода в состояния (ДПС)



- Диаграмма перехода в состояния изображает состояния прямоугольниками и фиксирует, в каких состояниях может находиться объект и как он переходит между ними.
- Каждому состоянию присваиваются имя (что с объектом происходит) и уникальный номер в пределах модели состояний

23.2.1. Виды состояний

- Начальное состояние (состояние создания): объект появляется впервые; переход в него отмечается как переход «ниоткуда».
- Текущие состояния: рабочие состояния, в которых объект находится в процессе жизни между созданием и завершением.
- Заключительные состояния:
 - 1) Либо объект остаётся в этом состоянии без дальнейшего анализа.
 - 2) Либо происходит уничтожение объекта (пунктирный прямоугольник).
- Переходные состояния: добавляются, если в физическом мире переход не мгновенный (например, «двери открываются/закрываются»), чтобы отразить протяжённость процесса во времени.

23.2.2. События, данные событий

- Событие переводит объект из одного состояния в другое.
- События связаны с моделью состояний как причина переходов. На ДПС события указываются на стрелках переходов.
- Событие может порождаться самим объектом для других объектов, а также приниматься объектом как вход для изменения состояния.

Идентификация событий

- Каждому событию присваиваются «ключевой литерал/номер» и название — фиксируется, «что происходит» («включается лампочка», «открывается дверь»).
- Событие связано с конкретным объектом: при обработке мы помечаем, к какому объекту оно относится.

Данные событий

- События переносят данные; если событие адресовано другому объекту, оно обязательно несёт идентификатор этого объекта.
 - Исключение: для состояния создания идентификатор объекта событие не несёт.
- Кроме идентификатора, событие может переносить вспомогательные данные, необходимые для обработки.

Правила

- 1) События никогда не теряются: если в данный момент выполняется действие объекта, событие будет обработано после завершения действия.
- 2) После того как событие принято объектом, оно прекращает своё существование как событие.
- 3) Все события, переводящие объект в одно и то же состояние, должны нести одни и те же данные, чтобы корректно вызывалось действие состояния.

23.2.3. Действия состояний

- Обработчик состояния переводит объект в это состояние, и ничего другого он не должен делать, то есть не втягивать «всю программу» и не выполнять то, что не относится к текущему состоянию.

- Обработчики состояний нельзя вызывать как методы; вызывать их нужно только через события.

Что действие может делать

- Допускается работа с атрибутами не только самого объекта, но и «любых объектов любых классов» (позже это будет ограничено моделью доступа).
- Выполнять предметные операции (действия с таймером: запуск, сброс, считывание времени).
- Порождать события как для себя, так и для других объектов.

Требования к действиям

- Действие обязательно должно изменять статус, то есть фиксировать переход объекта в соответствующее состояние.
- Действие не должно оставлять атрибуты самого объекта противоречивыми, то есть обязано обеспечивать целостность и непротиворечие.
- Не нарушать целостность связей с другими сущностями (связь «муж–жена» не должна становиться противоречивой после выполнения обработчика).
- Для одного объекта в данный момент времени может выполняться только одно действие.

23.3. Таблица перехода в состояния (ТПС)

- Таблица перехода в состояния строится в дополнение к ДПС, потому что анализ «всевозможных переходов» по одной диаграмме неудобен, а ТПС делает перебор полным и наглядным.
- ТПС выявляет «аномальное поведение» модели: ячейки с невозможными событиями показывают пробелы в проекте и подсказывают, где нужны новые состояния или обработка исключений.
- В ТПС каждая строка — это состояние, каждый столбец — событие, а содержимое ячейки описывает реакцию.
- Таблица должна быть полностью заполнена.

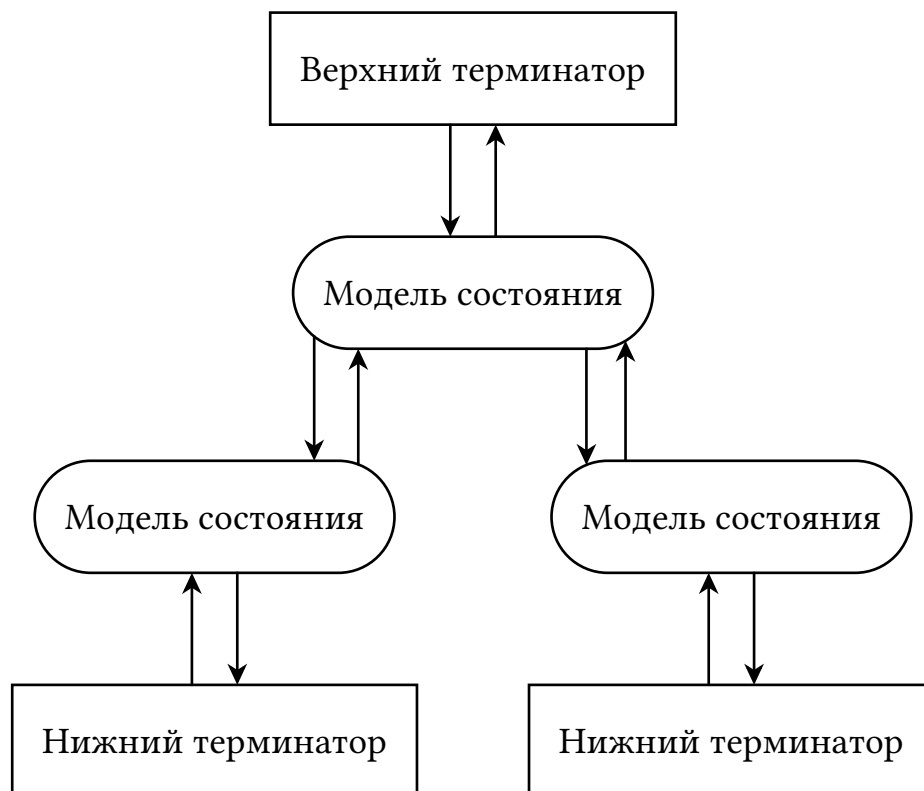
23.3.1. Как заполняются ячейки

- 1) Если, находясь в состоянии S , при событии T объект переходит в другое состояние S' , в ячейке ставится номер целевого состояния S' .
- 2) Если событие в этом состоянии не приводит к изменению состояния, оно «игнорируется», и в ячейке ставится прочерк «—».
- 3) Событие не может произойти в данном состоянии; такую ячейку помечают специальным знаком « $\times$ » и далее трактуют как исключительную ситуацию, требующую явной обработки.

24. Модель взаимодействия объектов (МВО). Диаграмма взаимодействия объектов в подсистеме. Типы событий. Схемы управления. Имитирование. Каналы управления.

24.1. Модель взаимодействия объектов (МВО). Диаграмма взаимодействия объектов в подсистеме.

- Мы должны построить модель взаимодействия объектов для нашей подсистемы, где взаимодействие рассматривается именно как событийное: стрелками показывается, какая модель состояния порождает событие и какая его принимает.
- Подсистема не живёт в вакууме и взаимодействует с окружающим миром; на МВО это фиксируется через события между объектами внутри подсистемы и внешними источниками/приёмниками.
- Диаграмма служит проверкой связности и корректности жизненных циклов объектов в подсистеме перед имитацией и анализом схем управления.



24.1.1. Границы и терминаторы

- Систему всегда ограничиваем сверху и снизу
Рисуются прямоугольники-терминаторы верхнего уровня и нижнего уровня, которые обозначают внешние по отношению к подсистеме сущности и каналы событий.
- Рассматривается только наша подсистема
 - Верхние терминаторы показывают, откуда приходят внешние события.
 - Нижние терминаторы — куда уходят исходящие.
 - Но внутренняя детализация других доменов на этой диаграмме не раскрывается.

24.1.2. Расположение моделей

- Модели состояний, более осведомлённые о всей системе, рисуются сверху, менее осведомлённые — внизу; это подчёркивает управление сверху вниз (например, «кабина управляет дверьми»).
- Для верхнего уровня стараемся всё свести к работе с одной сущностью, выбирая такую сущность, которая естественно выступает фасадом подсистемы; для нижнего уровня могут участвовать разные сущности.

24.1.3. Правила изображения взаимодействий

На МВО события рисуются стрелками: явным образом указывается «кто порождает и кто принимает», то есть от какой модели состояния к какой идёт событие.

24.2. Типы событий

24.2.1. Внутренние события

Внутренние события — это сигналы, которые порождаются объектами/моделями состояний внутри подсистемы и потребляются другими внутренними объектами для изменения их состояний.

На диаграмме взаимодействия они изображаются стрелками от порождающей модели состояния к принимающей, подчёркивая асинхронную событийную схему.

24.2.2. Внешние события

Внешние события — приходят от окружающих доменов/подсистем через верхние или нижние терминаторы и инициируют реакции внутри нашей подсистемы.

Они являются источником входных воздействий, которые запускают каналы управления и проверяются далее при имитации поведения.

Запрашиваемые и незапрашиваемые

Запрашиваемые события — это внешние события, являющиеся результатом предыдущего действия нашей подсистемы (продолжение начатого нами взаимодействия).

Незапрашиваемые события — внешние события, которые не являются следствием предыдущей деятельности подсистемы и возникают как самостоятельные входы из внешней среды.

24.3. Схемы управления

В схеме управления верхнего уровня события приходят от терминатора верхнего уровня, а в схеме нижнего уровня — от терминатора нижнего уровня.

24.4. Имитирование. Каналы управления

24.4.1. Имитирование

Имитирование — это прогон событийной модели по каналам управления от каждого допустимого начального состояния по каждому внешнему незапрашиваемому событию до достижения заключительных состояний; каналы управления фиксируют последовательность переходов и порождённых событий между объектами вплоть до завершения реакции.

- Процесс начинается с выделения списка возможных начальных состояний объектов подсистемы, чтобы задать стартовые конфигурации поведения.
- Далее анализируется реакция на каждое внешнее незапрашиваемое событие; при 3 начальных состояниях и четырёх 4 таких событиях придётся провести 12 имитаций.

24.4.2. Каналы управления

Канал управления — это диаграмма, где для каждого объекта отводится вертикальный столбец, а стрелками показываются переходы состояния и порождённые события между объектами в ответ на одно входное незапрашиваемое событие до наступления завершения.

- Если поведение подсистемы порождает внешнее запрашиваемое событие, оно включается в тот же канал, поскольку является продолжением реакции.
- Канал должен иметь конец: после обработки входного незапрашиваемого события система приходит в заключительные состояния, причём проверяется, может ли такое заключительное состояние быть начальным для новой реакции.

Представление на диаграмме

- В канале переход состояния одной сущности рисуется как стрелка внутри её столбца, а событие, адресованное другой сущности, — как стрелка, переводящая ту сущность в новое состояние.
- Все сущности и их начальные состояния видны одновременно, что позволяет проследить цепочку взаимодействий до завершения.

Время

- Для реалистичной проверки учитываются два времени на состоянии:
 - 1) Время действия (сколько выполняется обработчик).
 - 2) Время задержки (минимальное пребывание в состоянии).
- Если время действия сравнимо с задержкой, его учитывают; если больше задержки, возникает проблема реального масштаба времени и требуется принимать дополнительные решения, чтобы восстановить корректность моделирования.

25. Диаграмма потоков данных действий (ДПДД). Типы процессов: аксессоры, генераторы событий, преобразования, проверки. Таблица процессов (ТП). Модель доступа к объектам (МДО).

25.1. Диаграмма потоков данных действий (ДПДД)

25.1.1. Что такое ДПДД, зачем нужна

- Диаграмма потоков данных действий (ДПДД) — используется для формализации описания алгоритмов каждого действия или обработчика состояния.
- Впервые предложена в 1972-м году Де Марко, изначально для выявления параллелизма: понять, какие процессы можно выполнять одновременно при появлении многопроцессорной вычислительной техники.

25.1.2. Главное отличие от обычных блок-схем

- ДПДД не столько показывает последовательность выполнения, сколько формально фиксирует потоки данных между действиями и даёт явное понимание, что может выполняться параллельно, какие данные нужны процессу для запуска.
- Позволяет определить, какие процессы могут быть общими для разных моделей состояний и где возникает дублирование кода.

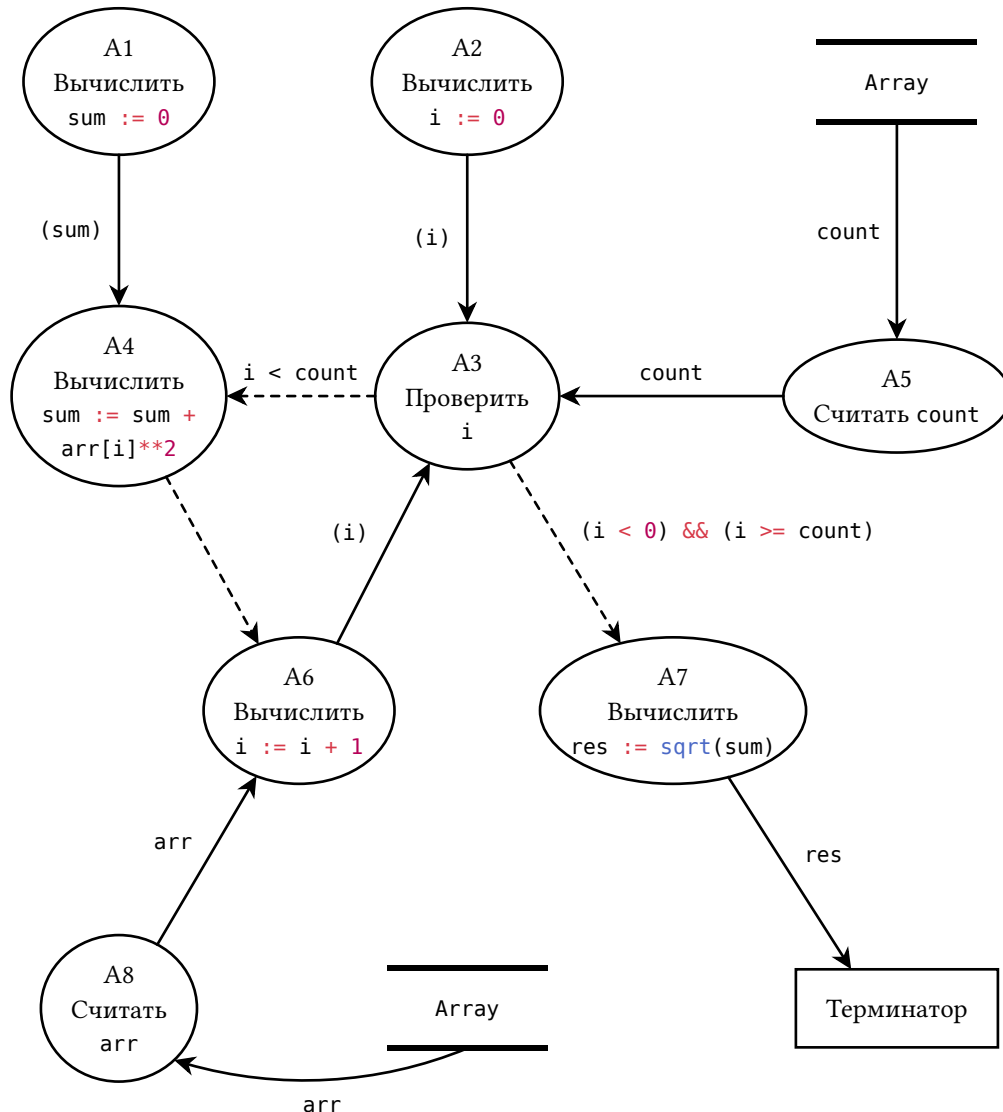
25.1.3. Последовательность построения ДПДД

- 1) Определяем цель действия (что хотим получить).
- 2) Расписываем необходимые процессы (каждое действие — отдельный овал).
- 3) Фиксируем источники данных (архивы, временные, результат других процессов).
- 4) Рисуем потоки данных стрелками (показываем и подписываем условия, если есть).

25.1.4. Обозначения на ДПДД

- Процесс изображается овалом.
 - В овале пишется: что за процесс (например, «вычислить сумму», «инициализация i »).
- Стрелки:
 - Показывают поток данных (откуда/куда нужен результат или данные).
 - Сплошная стрелка: обычный (безусловный) переход с передачей данных.
 - Пунктирная стрелка:
 - 1) Условный переход (пишем рядом условие, при котором осуществляется переход).
 - 2) Безусловный переход без передачи данных.
- Данные:
 - Устойчивые данные (архив данных): изображаются отдельным блоком, всегда доступны; получение атрибута — стрелка из блока архива данных.
 - Неустойчивые данные: временные; изображаются в круглых скобках, становятся доступны после выполнения определённых процессов.
- Терминатор (внешняя сущность):

- Используется для обозначения, если результат или входные данные поступают/отправляются вне системы.
- Обозначения переменных и условий:
 - Условия явно отмечаются рядом с пунктирными стрелками (например, $i < n$).
 - Если переменная появляется или используется в разных процессах — стрелками показывается источник значения.



25.2. Типы процессов

- 1) **Аксессорные процессы (аксессоры):**
 - Это процессы, которые читают, записывают, создают или уничтожают архив данных (то есть работают с атрибутами объектов/сущностей).
 - На диаграмме всегда есть процессы для доступа к архиву данных.
- 2) **Генераторы событий (процессы порождения событий):**
 - Это процессы, которые порождают события (фактически инициируют внешнюю реакцию или сигнал к другому объекту/подсистеме).
 - Событие тоже переносит данные.
 - Обозначаются как переход в «никуда»
- 3) **Вычислительные процессы (преобразования):**
 - Это процессы, выполняющие произвольные вычисления, преобразования данных.

- Например: суммирование, умножение, и другие операции.
- 4) Процессы-проверки (процессы проверки):
 - Это процессы, в которых происходит проверка каких-либо условий, обычно они обеспечивают условные переходы между другими процессами.
 - На диаграмме является тем процессом, после которого идут разные ветвления в зависимости от условий.

25.3. Таблица процессов

- После построения ДПДД для всех моделей состояний все выявленные процессы сводятся в единую таблицу — таблицу процессов.
- Это делается для борьбы с дублированием кода, чтобы видеть, какие процессы:
 - выполняются в разных действиях или даже в разных моделях состояний,
 - можно использовать повторно,
 - стоит вынести в общие абстракции (например, создать суперкласс).

25.3.1. Что отражается в таблице

- 1) Идентификатор процесса (номер или уникальное имя).
 - 2) Тип процесса (один из: аксессор, генератор событий, преобразование/вычисление, проверка).
 - 3) Название процесса (кратко: что делает).
 - 4) Где используется:
 - 4)1) Модель состояния,
 - 4)2) Конкретное действие.
- (Дополнительно можно пронумеровать процессы для удобства.)

25.3.2. Преимущество таблицы

- По таблице легко отследить, какие процессы повторяются (работают в нескольких действиях/моделях состояния).
- Это позволяет выявить дублирование и решить, как его устранить:
 - Выделить общий процесс,
 - Использовать паттерны проектирования или суперклассы,
 - Продумать, где удобно разместить общий код.

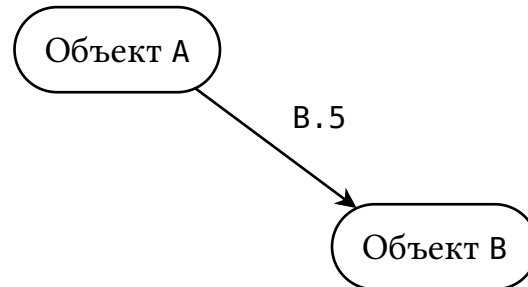
25.4. Модель доступа к объектам (МДО)

- Модель доступа к объектам строится для описания всех возможных взаимодействий объектов между собой через аксессорные процессы, то есть через чтение/запись данных в «архивы данных» объектов.
- Главная цель: чётко отразить, кто, к каким данным и из каких объектов может получить доступ при выполнении действий системы.

25.4.1. Как строится МДО

- МДО строится только через аксессорные связи
- Каждый объект изображается вытянутым овалом.

- Между объектами рисуются связи (рёбра), показывающие факт доступа одного объекта к архиву данных другого.
 - Например, объект А использует аксессор объекта В — рисуем стрелку от А к В, при этом подписываем номер процесса или его идентификатор.
- Такие связи иллюстрируют:
 - Какие объекты являются «клиентами» (получают доступ к данным),
 - Какие — «серверами» (предоставляют данные).



26. Домены. Модели доменного уровня. Типы доменов. Мосты, клиенты, сервера.

26.1. Домены

Домен — это отдельный реальный или гипотетический мир, населённый отчётливым набором объектов, которые ведут себя в соответствии с характерными для домена правилами и линиями поведения.

Домен — это определённая часть предметной области, которую мы выделяем при проектировании системы.

- Организация доменов позволяет использовать разные технологии и разделять части системы логически.

26.2. Типы доменов

1) Прикладной (бизнес) домен:

- Прикладной домен (бизнес-логика) — основная задача, это то, что реализует основной функционал системы.

2) Сервисный домен:

- Предоставляют набор функций, часто низкоуровневых.
 - Например: набор функций, реализующий какие-то алгоритмы, которые используются остальной системой, или пользовательский интерфейс.
- Реализуют функции для объектной модели.

3) Архитектурный домен:

- Предоставляют набор механизмов, например, для организации взаимодействия между частями кода.
- Отвечает за обеспечение целостности системы, регулирует управление и доступность данных.
- Проектирование масштабных систем всегда начинается с архитектурного домена.

4) Домен реализации:

- Домен реализации — это библиотеки, которые мы используем.
- Нужно строить домен таким образом, чтобы учесть возможность работы с новыми версиями, или заменить одну библиотеку на другую.

- Каждый домен может реализовываться разными технологиями (интерфейсный, сервисный, архитектурный — это могут быть разные технические решения).

26.3. Мосты, клиенты, сервера

- Если между доменами есть связь, мы говорим, что между доменами есть мост.
 - Мост — это элемент модели, который соединяет два домена: один выступает как сервер, другой как клиент.
- Сервер к мосту подходит как сторона, которой предъявляют требования.
 - То есть сервер — это домен, предоставляющий функции и возможности, которые другие могут использовать.

- Клиент к мосту подходит как сторона, которой нужны эти возможности.
 - То есть клиент — это домен, который использует возможности сервера.

26.4. Модели доменного уровня

На уровне подсистем используется та же логика моделей, что и на уровне объектов — событийная модель взаимодействия и синхронная модель доступа, но участниками становятся подсистемы (системы), а не отдельные объекты.

- Модель доменного уровня строится для отражения связей между этими доменами, их взаимодействий и назначения в системе.
- Модель доменного уровня визуализируется отдельной диаграммой, где показаны сервера, клиенты и мосты между ними.
- Если в домене очень много классов (сущностей), такой домен нужно разбивать на подсистемы.
- На диаграммах более высокого уровня все подсистемы изображаются прямоугольниками, и далее указываются связи между этими прямоугольниками.

26.4.1. Что для домена

- 1) (ДСП) Диаграмма связей подсистем, где перечисляются все подсистемы (в прямоугольниках) и их связи.
- 2) (ДВП) Диаграмма взаимодействия подсистем как событийная часть на уровне подсистем.
- 3) (ДДП) Диаграмма доступа между подсистемами, фиксирующая акцессорные связи между ними.

Эти представления отражают асинхронную часть (события между подсистемами) и синхронную часть (доступ/акцессорные связи между подсистемами), как и на объектном уровне.

26.4.2. Что для подсистемы

- 1) (ДСС) Своя информационная модель (диаграмма сущность-связь) именно этой подсистемы
- 2) (МВО) Модель взаимодействия объектов внутри подсистемы (событийная)
- 3) (МДО) Модель доступа к объектам этой подсистемы (синхронная).

На этих диаграммах участниками являются конкретные объекты/классы подсистемы, а не другие подсистемы.

27. Объектно-ориентированное проектирование. Диаграмма класса. Структура класса. Диаграмма зависимостей. Диаграмма наследования.

27.1. Объектно-ориентированное проектирование.

Объектно-ориентированное проектирование — это этап разработки, следующий после объектно-ориентированного анализа.

Его задача — на основе результатов анализа и выявленных абстракций (сущностей, атрибутов, обработчиков, взаимосвязей) сформировать детализированную структуру программной системы: определить классы, их атрибуты и методы, связи между ними, уровни доступа, механизмы наследования и взаимодействия, а также общий архитектурный скелет программы.

- Документы анализа фиксируют, что должно быть реализовано — какие есть сущности, их свойства, поведение на высоком уровне, но не регламентируют, как это будет реализовано в коде.
- Проектирование добавляет детализацию: как организована структура классов, где проходят границы между приватными/защищёнными/публичными методами, как обеспечивается целостность объекта, какие методы порождаются, как реализуется наследование и защищённость.
- На этапе проектирования встаёт задача выбрать архитектурные подходы, паттерны, минимизировать дублирование кода, корректно организовать доступ к данным, спроектировать взаимодействие между подсистемами.

Объектно-ориентированное проектирование использует различные нотации, чаще всего UML, но эта нотация крайне избыточная. Мы рассматриваем нотацию Гради Буча, которая имеет большое количество иллюстраций.

27.2. Диаграмма класса

27.2.1. Как строится диаграмма класса

- Каждый класс изображается прямоугольником.
- Вверху — имя класса.
- Атрибуты изображают в отдельных шестиугольниках. В этих блоках сверху пишется имя, а снизу тип атрибута.



Методы

- Методы изображаются в прямоугольниках.
- Методы могут быть двух категорий:
 - 1) Методы класса (статические) располагают с правой стороны прямоугольника.
 - 2) Методы объекта (обычные) — с левой.

- Для методов можно дополнительно указывать данные в шестиугольниках (как атрибуты). Для этого к методу извне проводится линия.
 - 1) Принимаемые данные рисуются над линией.
 - 2) Изменяемые — линия проходит через середину шестиугольника.
 - 3) Возвращаемые рисуются под линией.
- Дополнительно отмечаются различные свойства методов
 - Обработка исключений — ромб на ребре прямоугольника.
 - Асинхронность — пунктирный прямоугольник.
 - Callback — прямоугольник с вертикальной чертой внутри.
- Методы, которые взаимодействуют с данными только внутри класса и не выходят наружу, изображаются как прямоугольники, «не касающиеся сторон» класса — подчёркивается их приватность.

27.3. Структура класса

27.3.1. Слои методов

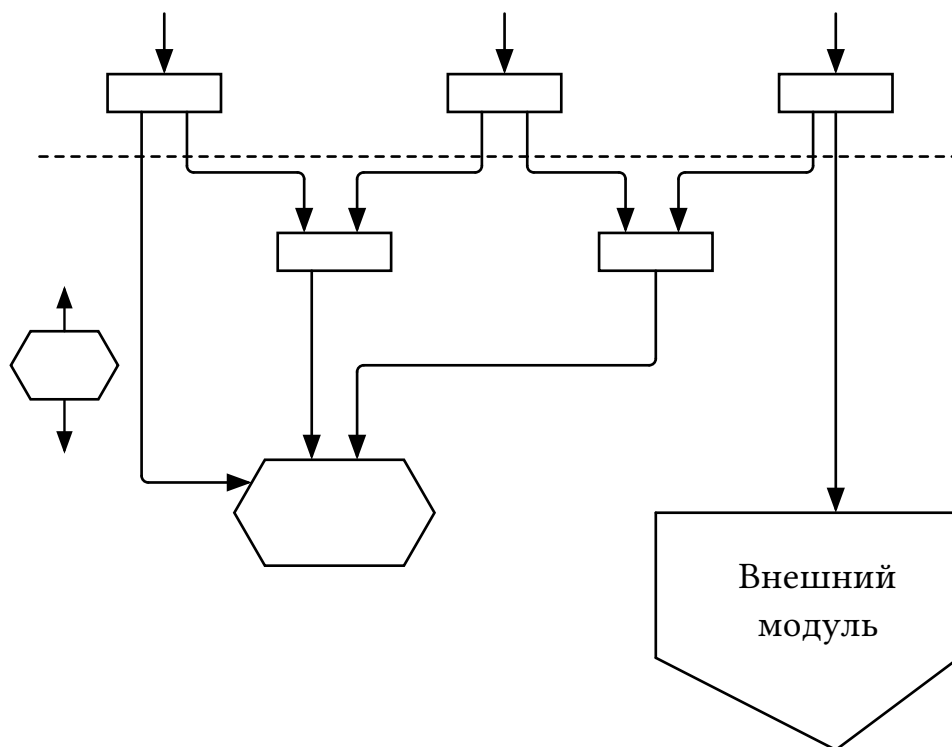
- Существует правило: методы одного слоя не должны вызывать друг друга — проводится граница (пунктирная линия) между внешним (верхним) и внутренним уровнями (нижними).
- На внутреннем уровне выделяют общий код и общие процессы, которые используются несколькими методами.

27.3.2. Защищённые методы

- Контроль целостности объекта часто выносится в защищённые методы (protected), чтобы они были доступны для классов-потомков, но не для всего остального кода.
- Это обеспечивает как безопасность базового класса, так и возможность безопасного наследования.

27.3.3. Данные

- Возле линии к методу в виде шестиугольника указывается, как и с какими данными (атрибутами) работает метод.
 - 1) У принимаемых рисуется стрелка вниз.
 - 2) У возвращаемых — стрелка вверх.
 - 3) У изменяемых — обе стрелки.
- Если данные передаются во внешний модуль, рисуется пятиугольник.



27.4. Диаграмма зависимостей

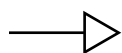
Диаграмма зависимостей предназначена для отображения связей между классами, показывая, как одни классы используют другие не через наследование, а через различные формы взаимодействия — «агрегацию» и «композицию».

- Классы изображаются отдельными прямоугольниками. В заголовке указывается имя класса.
- Между ними проводятся стрелки:
 - Одинарная стрелка ($\longrightarrow$) — обозначает агрегацию — более слабую форму связи, когда один класс «имеет» другой, но их жизненные циклы не зависят друг от друга.
 - Двойная стрелка ($\Longrightarrow$) — обозначает композицию — более сильную связь, «часть-целое», когда составной объект владеет частью и управляет её временем жизни.

27.5. Диаграмма наследования

Диаграмма наследования отражает иерархию классов, показывая, какие классы являются базовыми (родительскими), а какие производными (потомками). Она позволяет наглядно представить, как распространяются атрибуты и методы по дереву наследования, какие элементы абстрагируются в суперклассах, а какие реализуются в дочерних классах.

- На схеме классы изображают так же в прямоугольниках..
- Связь наследования отмечается стрелкой с треугольником.



- Прямоугольник разбит на три части:
 - 1) Имя класса
 - 2) Атрибуты/методы защищённые.
 - 3) Незащищённые методы.

Это позволяет показать, какие элементы наследуются, а какие остаются недоступными в потомках.

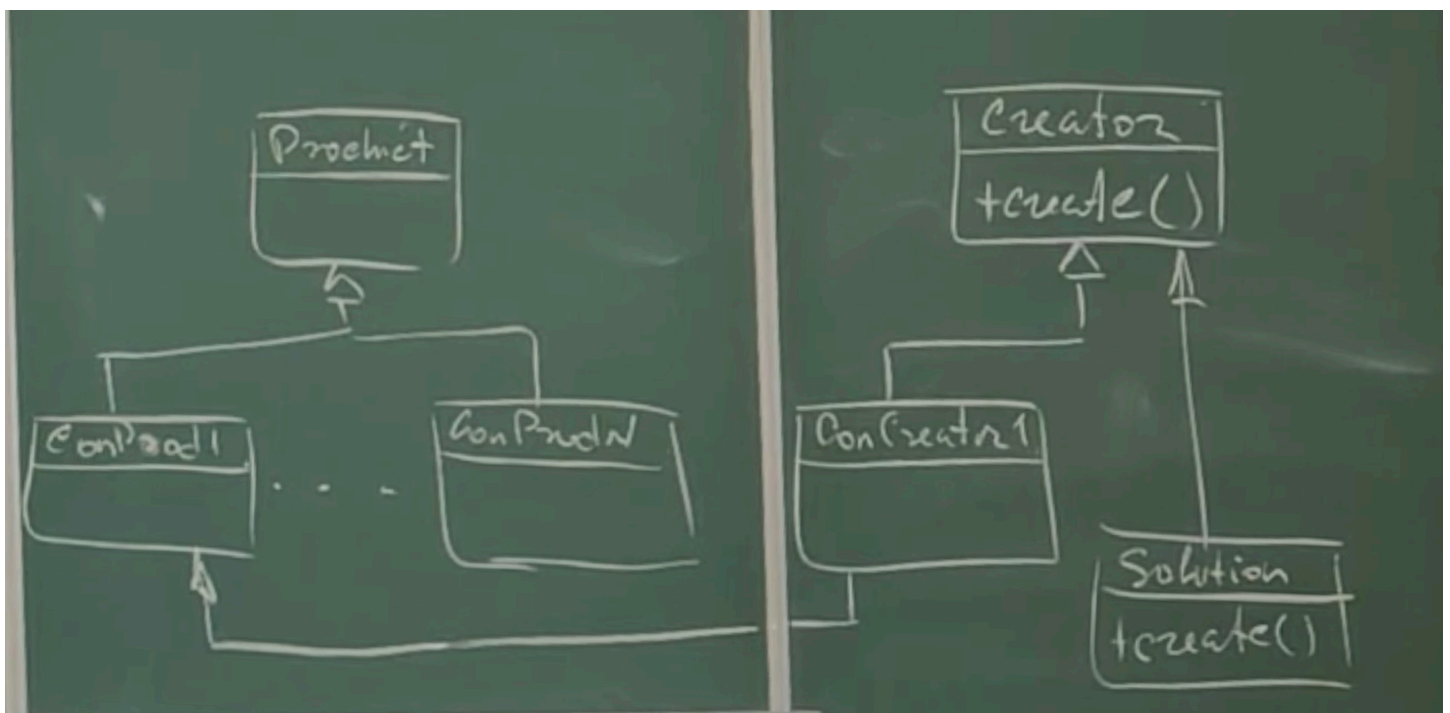
28. Порождающие паттерны: фабричный метод (Factory Method), абстрактная фабрика (Abstract Factory), строитель (Builder). Их преимущества и недостатки.

28.1. Порождающие паттерны

Все порождающие паттерны имеют одну основную цель — модификация программы без изменения уже написанного кода. Вместо явного создания объектов для полиморфных классов выделяются роли/объекты, которые принимают решение, какой экземпляр создавать, и порождают его, тем самым очищая код от конкретных типов. Такой перенос ответственности делает выбор типа централизованным и контролируемым.

28.2. Фабричный метод (Factory Method)

- Фабричный метод — это порождающий паттерн, который инкапсулирует создание объектов — вместо прямого вызова конструктора в коде используется метод. Клиент работает с интерфейсом продукта и не знает (и не должен знать) конкретного класса, создаваемого фабричным методом.
- Основная идея: избавить прикладной код от явного создания конкретных объектов полиморфных классов. Класс, называемый фабрикой или креатором, принимает решение, какой конструктор вызывать и какой экземпляр вернуть.
- При работе с полиморфизмом не следует явно в коде создавать объекты производных классов — это неудобно для модификации кода.
- Решение о том, какой объект создавать, выносится в отдельную сущность Solution.
- В идеале, чтобы невозможным было создание объекта напрямую, делают конструкторы производных классов `protected` — тогда создавать их может только фабрика.
- Чтобы полностью закрыть доступ к конструктору за пределами фабрики, используют идиому прокси-класса для доступа к `protected`-конструктору.



28.2.1. Идиома неvirtуального интерфейса (NVI)

- NVI — это когда публичный метод в базовом классе не является виртуальным, но внутри него вызывается защищённый виртуальный метод, который реализуют производные классы.
 - Пример: публичный метод `getProd()` не виртуален, а внутри него вызывается защищённый виртуальный метод (`createProd()` и т.п.), который реализует логику в производных классах.

Зачем это нужно

- Позволяет базовому классу держать под контролем общий алгоритм, не давая производным полностью подменять поведение.
- Базовый класс может реализовать общие проверки, кеширование, логику повторного использования, а производный класс реализует только создание или обработку специфического продукта.

28.2.2. Преимущества

- + Очищение клиентского кода от создания конкретных объектов: клиент код становится независимым от конкретных типов, его легко модифицировать.
- + Безопасность: программисты не могут напрямую создавать объекты производных классов, защищая код от ошибок.
- + Гибкость: можно менять создаваемый тип на этапе выполнения, можно разносить решение и создание по разным частям кода и во времени.

28.2.3. Недостатки

- Нужно «таскать» по коду указатель/ссылку на фабрику/креатор, если из разных мест нужно создать объекты — это бывает неудобно.
- Когда по фабричному методу мы создаём объект в одном месте, а потом в другом месте нам нужен точно такой же объект, происходит следующее: если мы каждый раз вызываем фабрику, она создаст новый объект. Это проблема, особенно если это тяжёлый объект.
- Если необходимо создавать объекты с разными параметрами, стандартная реализация становится громоздкой (разные данные — разные конструкторы, нужен шаблон).
- В коде появляются дополнительные классы-фабрики, что усложняет структуру программы.

28.2.4. Пример

```
1 #include <concepts>
2 #include <iostream>
3 #include <memory>
4
5 using namespace std;
6
7 template <typename Derived, typename Base>
8 concept Derivative = is_abstract_v<Base> && is_base_of_v<Base, Derived>;
```

```

9
10 template <typename Type>
11 concept NotAbstract = !is_abstract_v<Type>;
12
13 template <typename Type>
14 concept DefaultConstructible = is_default_constructible_v<Type>;
15
16 class Car;
17
18 class CarCreator
19 {
20     public:
21         virtual ~CarCreator() = default;
22         virtual unique_ptr<Car> create() const = 0;
23 };
24
25 template <Derivative<Car> TCar>
26     requires NotAbstract<TCar> && DefaultConstructible<TCar>
27 class ConcreteCarCreator : public CarCreator
28 {
29     public:
30         unique_ptr<Car> create() const override { return make_unique<TCar>(); }
31 };
32
33 class Car
34 {
35     public:
36         virtual ~Car() = default;
37         virtual void drive() = 0;
38 };
39
40 class Sedan : public Car
41 {
42     public:
43         Sedan() { cout << "Sedan constructor called\n"; }
44         ~Sedan() override { cout << "Sedan destructor called\n"; }
45         void drive() override { cout << "Driving sedan\n"; }
46 };
47
48 class User
49 {
50     public:
51         void use(const shared_ptr<CarCreator>& creator)
52         {
53             shared_ptr<Car> car = creator->create();
54
55             car->drive();
56         }

```

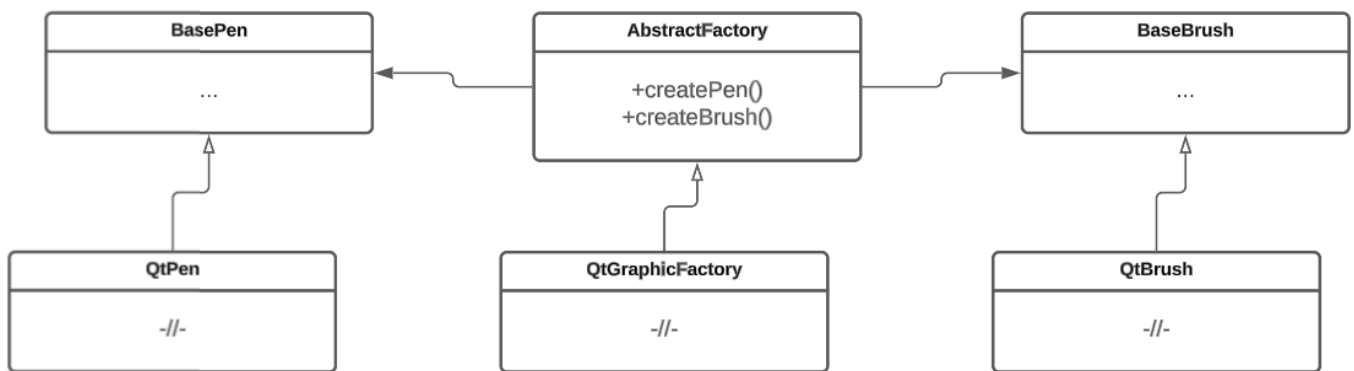
```

57 };
58
59 int main()
60 {
61     shared_ptr<CarCreator> creator =
62     make_shared<ConcreteCarCreator<Sedan>>();
63     User{}.use(creator);
64 }

```

28.3. Абстрактная фабрика (Abstract Factory)

- Абстрактная фабрика объединяет несколько креаторов в одном классе для создания объектов, относящихся к одному семейству.
- Нужно, чтобы создавались объекты только одного семейства, например, для графических библиотек: Graphics, Pen, Brush и т.д.
- Не стоит использовать объекты из разных семейств вместе, чтобы избежать ошибок совместимости.



28.3.1. Преимущества

- + Гарантируется совместимость создаваемых объектов (создаются только объекты одного семейства).
- + Упрощается конфигурирование системы: достаточно выбрать одну фабрику — и все компоненты будут подходящими друг к другу.
- + Упрощается поддержка и расширяемость: вся логика создания семейства объектов локализована в одной фабрике.

28.3.2. Недостатки

- Главный недостаток тот же, что и у фабричного метода: нужно «таскать» фабрику везде, где создаются объекты, это не всегда удобно.
- В разных семействах могут быть неравносильные составы объектов (в каком-то семействе может не быть конкретного продукта) — приходится решать эту проблему, например, созданием фиктивных объектов.
- Абстрактная фабрика, как и другие креаторы, может «разрастаться» из-за увеличения числа продуктов и их вариантов.

28.3.3. Пример

```
1 #include <iostream>
2 #include <memory>
3
4 using namespace std;
5
6 class Image { };
7 class Color { };
8
9 class BaseGraphics
10 {
11     public:
12         virtual ~BaseGraphics() = 0;
13 };
14 BaseGraphics::~~BaseGraphics()
15 {
16 }
17
18 class BasePen { };
19 class BaseBrush { };
20
21 class QtGraphics : public BaseGraphics
22 {
23     public:
24         QtGraphics(shared_ptr<Image> im) { cout << "Calling the QtGraphics
25 constructor;\n"; }
26         ~QtGraphics() override { cout << "Calling the QtGraphics destructor;
27 \n"; }
28 };
29
30 class QtPen : public BasePen { };
31 class QtBrush : public BaseBrush { };
32
33 class AbstractGraphFactory
34 {
35     public:
36         virtual ~AbstractGraphFactory() = default;
37
38         virtual unique_ptr<BaseGraphics> createGraphics(shared_ptr<Image> im)
39         = 0;
40         virtual unique_ptr<BasePen> createPen(shared_ptr<Color> cl) = 0;
41         virtual unique_ptr<BaseBrush> createBrush(shared_ptr<Color> cl) = 0;
42 };
43
44 class QtGraphFactory : public AbstractGraphFactory
45 {
46     public:
```

```

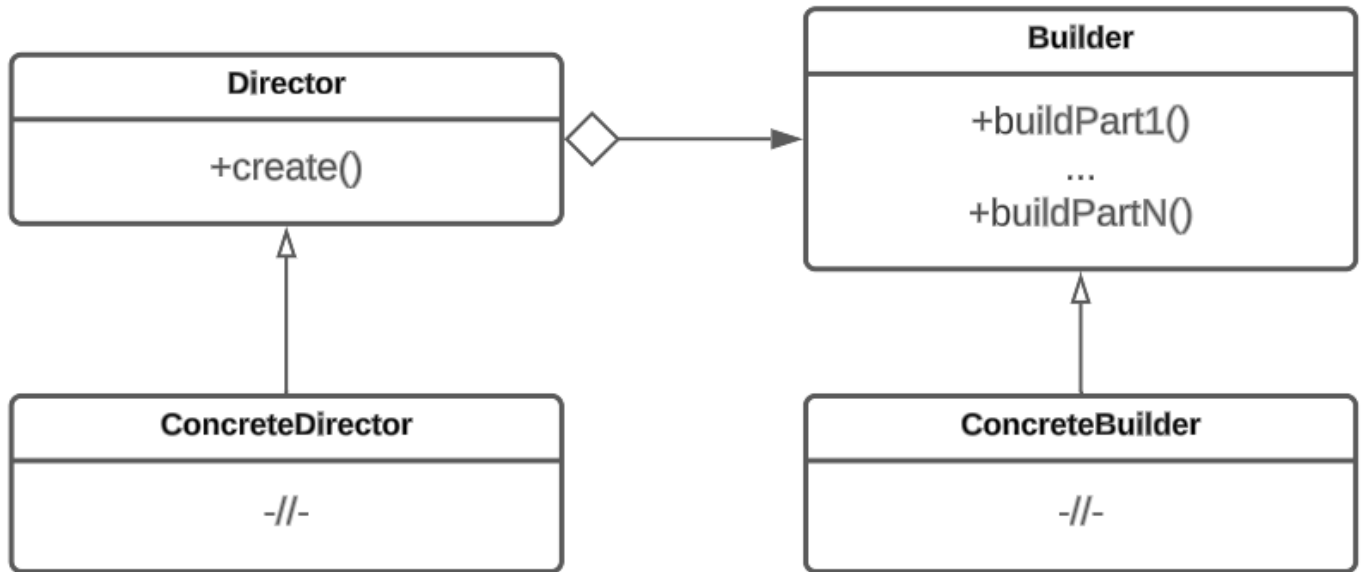
44     unique_ptr<BaseGraphics> createGraphics(shared_ptr<Image> im) override
    { return make_unique<QtGraphics>(im); }
45
46     unique_ptr<BasePen> createPen(shared_ptr<Color> cl) override { return
    make_unique<QtPen>(); }
47
48     unique_ptr<BaseBrush> createBrush(shared_ptr<Color> cl) override
    { return make_unique<QtBrush>(); }
49 };
50
51 class User
52 {
53     public:
54     void use(shared_ptr<AbstractGraphFactory>& cr)
55     {
56         shared_ptr<Image> image = make_shared<Image>();
57         auto graphics = cr->createGraphics(image);
58     }
59 };
60
61 int main()
62 {
63     shared_ptr<AbstractGraphFactory> grfactory =
    make_shared<QtGraphFactory>();
64
65     unique_ptr<User> us = make_unique<User>();
66
67     us->use(grfactory);
68 }

```

28.4. Строитель (Builder)

- Строитель — это порождающий паттерн для поэтапного создания сложного объекта, когда для его сборки нужно выполнить несколько действий последовательно и осмысленно организовать процесс создания.
- Выделяется класс «строитель (Builder)», у которого есть какие-то этапы строительства, как на реальной стройке, где работы идут по шагам. Эти этапы определяют порядок и способ сборки целевого объекта, оставаясь внутри ответственности строителя.
- Кто-то должен вызывать эти методы, кто-то должен контролировать выполнение этих этапов — вводится роль «директор (Director)». Задача директора — проконтролировать выполнение этих этапов и вернуть созданный объект, причём у директора должен быть метод create, который запускает сборку и отдаёт результат.
- Типичная ошибка: директор «подносит кирпичи строителю», то есть сам делает то, что обязан делать строитель.

- Директор может передавать какие-то данные, но он не должен выполнять этапы строительства.



28.4.1. Преимущества

- + Разделяет создание сложного объекта на этапы, делая процесс прозрачным и управляемым через строитель с последовательными шагами.
- + Вводит директора, который контролирует выполнение этапов и возвращает созданный объект через метод create, удерживая процесс создания в одном месте.
- + Можно включить директора в общую иерархию создателей, то есть сделать директора наследником креатора, чтобы единообразно вызывать create для простых и сложных случаев, скрывая различие реализации за общим интерфейсом.

28.4.2. Недостатки

- Паттерн предполагает дополнительные роли и этапы по сравнению с простым креатором, что увеличивает структурную сложность ради корректной поэтапной сборки.

28.4.3. Пример

```

1 #include <iostream>
2 #include <memory>
3
4 using namespace std;
5
6 class Product
7 {
8     public:
9         virtual ~Product() = default;
10
11         virtual void run() = 0;
12 };
13
14 class ConProd1 : public Product
15 {

```

```

16     public:
17         ConProd1() { cout << "Calling the ConProd1 constructor;\n"; }
18         ~ConProd1() override { cout << "Calling the ConProd1 destructor;\n"; }
19
20         void run() override { cout << "Calling the run method;\n"; }
21 };
22
23 class Builder
24 {
25     public:
26         virtual ~Builder() = default;
27
28         virtual bool buildPart1() = 0;
29         virtual bool buildPart2() = 0;
30
31         shared_ptr<Product> getProduct();
32
33     protected:
34         virtual shared_ptr<Product> createProduct() = 0;
35
36         shared_ptr<Product> product{nullptr};
37         size_t part{0};
38 };
39
40 class ConBuilder : public Builder
41 {
42     public:
43         bool buildPart1() override
44         {
45             if (!part) {
46                 ++part;
47             }
48             if (part != 1) {
49                 return false;
50             }
51
52             cout << "Completed part: " << part << ";\n";
53             return true;
54         }
55         bool buildPart2() override
56         {
57             if (part == 1) {
58                 ++part;
59             }
60             if (part != 2) {
61                 return false;
62             }
63

```

```

64         cout << "Completed part: " << part << ";\n";
65         return true;
66     }
67
68     protected:
69         shared_ptr<Product> createProduct() override;
70 };
71
72 class Creator
73 {
74     public:
75         virtual ~Creator() = default;
76
77         virtual shared_ptr<Product> create() = 0;
78 };
79
80 class Director : public Creator
81 {
82     public:
83         Director(shared_ptr<Builder> builder) : br(builder) {}
84
85         shared_ptr<Product> create() override
86         {
87             if (br->buildPart1() && br->buildPart2()) {
88                 return br->getProduct();
89             }
90
91             return nullptr;
92         }
93
94     private:
95         shared_ptr<Builder> br;
96 };
97
98 shared_ptr<Product> Builder::getProduct()
99 {
100     if (!product) {
101         product = createProduct();
102     }
103     return product;
104 }
105
106 shared_ptr<Product> ConBuilder::createProduct()
107 {
108     if (part == 2) {
109         product = make_shared<ConProd1>();
110     }
111     return product;

```

```

112 }
113
114 class User
115 {
116     public:
117         void use(shared_ptr<Creator>& cr)
118         {
119             shared_ptr<Product> prod = cr->create();
120             if (prod) {
121                 prod->run();
122             }
123         }
124 };
125
126 int main()
127 {
128     shared_ptr<Builder> builder = make_shared<ConBuilder>();
129     shared_ptr<Creator> cr = make_shared<Director>(builder);
130
131     User{}.use(cr);
132 }

```

29. Порождающие паттерны: одиночка (Singleton), прототип (Prototype), пул объектов (Object Pool). Их преимущества и недостатки.

29.1. Порождающие паттерны

Все порождающие паттерны имеют одну основную цель — модификация программы без изменения уже написанного кода. Вместо явного создания объектов для полиморфных классов выделяются роли/объекты, которые принимают решение, какой экземпляр создавать, и порождают его, тем самым очищая код от конкретных типов. Такой перенос ответственности делает выбор типа централизованным и контролируемым.

29.2. Одиночка (Singleton)

- Класс-одиночка запрещает создание своих экземпляров привычным способом (отсутствуют public-конструкторы), а вместо этого имеет статический член, который держит объект, что гарантирует один объект будет один для конкретного типа.
- Получение ссылки/указателя на этот объект осуществляется только через имя класса, то есть через статический метод.
- Singleton называют антипаттерном, потому что вызов через имя класса фактически даёт глобальный вызов.

29.2.1. Альтернатива

- Вместо одиночки можно создать креатора, который будет держать указатель на объект — современный подход склоняется к такому решению.
- Однако «это взламывается» — можно создать второй конкретный креатор и породить второй объект, то есть гарантия единственности нарушается при наличии нескольких конкретных создателей.
- Таким образом, реальная гарантия опирается на организационную дисциплину кода, а не на жёсткое свойство конструкции.

29.2.2. Преимущества

- + Гарантирует наличие лишь единственного экземпляра конкретного типа за счёт статического члена, который держит объект.

29.2.3. Недостатки

- Считается антипаттерном: доступ к объекту через имя класса фактически даёт глобальный доступ.
- Теряется возможность контроля жизненного цикла объекта.
- Тип создаваемого объекта определяется на этапе компиляции.

29.2.4. Пример

```
1 #include <iostream>
2 #include <memory>
3
```

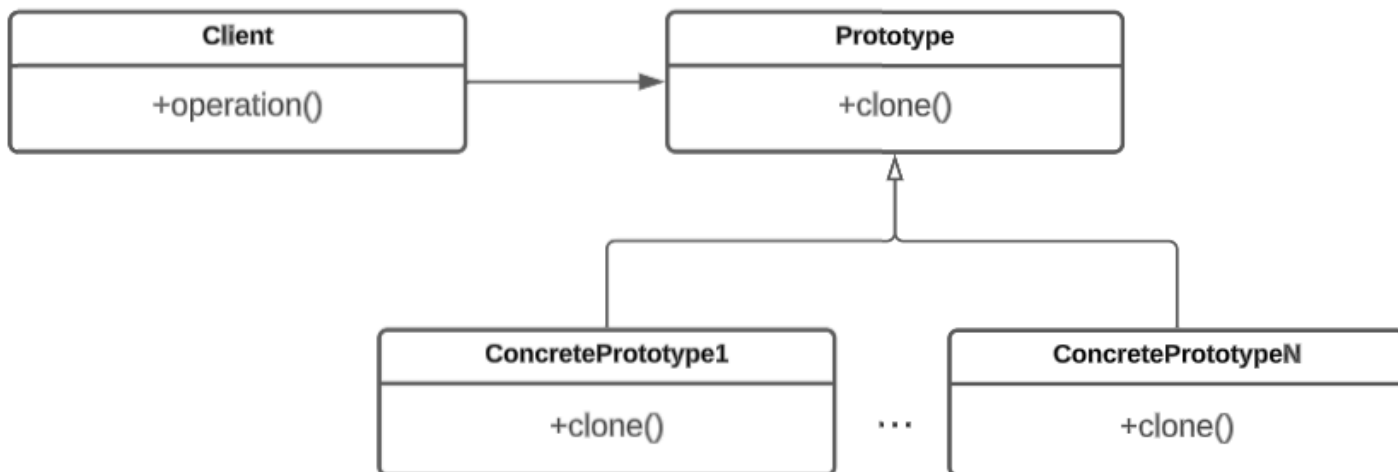
```

4 class Product
5 {
6     public:
7         static shared_ptr<Product> instance()
8         {
9             class Proxy : public Product {};
10
11             static std::shared_ptr<Product> myInstance =
std::make_shared<Proxy>();
12
13             return myInstance;
14         }
15         ~Product() {}
16
17         void f() {}
18
19         Product(const Product&) = delete;
20         Product& operator=(const Product&) = delete;
21
22     private:
23         Product() {}
24 };
25
26 int main()
27 {
28     shared_ptr<Product> ptr(Product::instance());
29
30     ptr->f();
31 }

```

29.3. Прототип (Prototype)

- Возлагаем ответственность за создание новых экземпляров на сам продукт, чтобы он мог создавать копию себя.
- В класс добавляется виртуальный метод `clone()`, который возвращает копию себя.
- Важно, чтобы `clone()` возвращал объект корректного типа.
- Проблема: если в наследуемом классе не реализовать свой метод `clone()`, вызовется базовый, и скопируется не тот, что нужен. Поэтому реализация `clone()` во всех производных классах обязательна.
- Для контроля правильности клонирования может использоваться идиома не виртуального интерфейса (non-virtual interface, NVI):
 - Создать не виртуальный `public`-метод, который внутри вызывает виртуальный `protected`-метод.
 - Тогда можно внутри базовой реализации контролировать процесс копирования, например, сверить тип объекта.



29.3.1. Преимущества

- + Позволяет создавать новые объекты на основе уже имеющихся без необходимости знать их точный класс.
- + Позволяет копировать сложные конфигурации объектов.
- + Упрощает создание копий (особенно когда процесс создания объекта сложен или ресурсоёмок).
- + Помогает реализовать повторное использование объектов без лишней зависимости от системы создания новых экземпляров.

29.3.2. Недостатки

- Необходимость корректно реализовывать метод `clone()` во всех наследуемых типах — если в каком-то классе забыть сделать свою реализацию, будет ошибка: скопируется не тот тип.
- Проблемы могут возникать при копировании сложных объектов с нестандартными связями — нужно аккуратно реализовывать глубокое/поверхностное копирование и следить за корректностью работы метода `clone()`.

29.3.3. Пример

```

1 #include <exception>
2 #include <iostream>
3 #include <memory>
4
5 using namespace std;
6
7 class BaseObject
8 {
9     public:
10         virtual ~BaseObject() = default;
11
12         unique_ptr<BaseObject> clone();
13
14     private:
15         virtual unique_ptr<BaseObject> doClone() = 0;
  
```

```

16 };
17
18 class Object1 : public BaseObject
19 {
20     public:
21         Object1() {}
22         Object1(const Object1& obj) {}
23         ~Object1() override {}
24
25     private:
26         unique_ptr<BaseObject> doClone() override
27         {
28             return make_unique<Object1>(*this);
29         }
30 };
31
32 class Object11 : public Object1
33 {
34 };
35
36 unique_ptr<BaseObject> BaseObject::clone()
37 {
38     unique_ptr<BaseObject> objclone = doClone();
39
40     if (typeid(*objclone) != typeid(*this)) {
41         throw runtime_error("Error clone!");
42     }
43
44     return objclone;
45 }
46
47 class User
48 {
49     public:
50         void use(shared_ptr<BaseObject>& obj) { auto obj1 = obj->clone(); }
51 };
52
53 int main()
54 {
55     try {
56         shared_ptr<BaseObject> obj = make_shared<Object11>();
57         User{}.use(obj);
58     } catch (exception& err) {
59         cout << err.what() << endl;
60     }
61 }

```

29.4. Пул объектов (Object Pool)

- Пул объектов — это структура, которая держит не один, а много заранее созданных экземпляров «тяжёлых» объектов, чтобы их не создавать заново при каждой необходимости.
- Пул хранит набор объектов; по запросу возвращает один и помечает его как занятый, чтобы параллельно им никто другой не пользовался.
- После завершения работы объект явно «отдают» обратно в пул, и он переходит в состояние свободного (состояние должно изменяться именно пулом).
- Пул может быть фиксированного размера или расширяемым; на практике часто нужен именно фиксированный лимит ресурсов.

Откуда берётся необходимость

Повторное создание тяжёлых объектов «каждый раз» долго и затратно, особенно если сопровождается выделением памяти и системными вызовами. Для одного объекта можно держать указатель в креаторе и не пересоздавать его, но если требуется несколько таких объектов, их следует держать сразу «несколько» и переиспользовать — это и решает пул.

29.4.1. Преимущества

- + Снижает число повторных созданий «тяжёлых» объектов, экономит время и издержки на выделение памяти и системные вызовы.
- + Обеспечивает повторное использование уже созданных экземпляров в разных местах программы.
- + Позволяет явно ограничивать количество одновременно используемых ресурсов (фиксированный пул).

29.4.2. Недостатки

- Требуется явное управление состоянием объектов в пуле (занят/свободен).
- При фиксированном размере, если «все номера заняты», новые запросы не получают объект — ресурс исчерпан до освобождения.
- Если объект не очищается или его состояние не сбрасывается перед возвращением в пул, может возникнуть утечка информации. Например, если объект содержит конфиденциальные данные или ссылки на другие объекты, эта информация может остаться в объекте после его возврата в пул.

29.4.3. Пример

```
1 #include <concepts>
2 #include <iostream>
3 #include <memory>
4 #include <vector>
5
6 using namespace std;
7
8 template <typename T>
9 concept PoolObject = requires(T t) { t.clear(); };
10
```

```

11 class Product
12 {
13     private:
14         static size_t count;
15
16     public:
17         Product() { ++count; }
18         ~Product() { count--; }
19
20         void clear() {}
21 };
22
23 size_t Product::count = 0;
24
25 template <PoolObject Type>
26 class Pool
27 {
28     public:
29         static shared_ptr<Pool<Type>> instance();
30
31         shared_ptr<Type> getObject();
32         bool releaseObject(shared_ptr<Type>& obj);
33         size_t count() const { return pool.size(); }
34
35         Pool(const Pool&) = delete;
36         Pool& operator=(const Pool&) = delete;
37
38     private:
39         vector<pair<bool, shared_ptr<Type>>> pool;
40
41         Pool() {}
42
43         pair<bool, shared_ptr<Type>> create();
44 };
45
46 template <PoolObject Type>
47 shared_ptr<Pool<Type>> Pool<Type>::instance()
48 {
49     static shared_ptr<Pool<Type>> myInstance(new Pool<Type>());
50
51     return myInstance;
52 }
53
54 template <PoolObject Type>
55 shared_ptr<Type> Pool<Type>::getObject()
56 {
57     size_t i = 0;
58     while (i < pool.size() && pool[i].first) {

```

```

59         ++i;
60     }
61
62     if (i < pool.size()) {
63         pool[i].first = true;
64     } else {
65         pool.push_back(create());
66     }
67
68     return pool[i].second;
69 }
70
71 template <PoolObject Type>
72 bool Pool<Type>::releaseObject(shared_ptr<Type>& obj)
73 {
74     size_t i = 0;
75     for (pool[i].second != obj && i < pool.size()) {
76         ++i;
77     }
78
79     if (i == pool.size()) {
80         return false;
81     }
82
83     obj.reset();
84     pool[i].first = false;
85     pool[i].second->clear();
86
87     return true;
88 }
89
90 template <PoolObject Type>
91 pair<bool, shared_ptr<Type>> Pool<Type>::create()
92 {
93     return {true, make_shared<Type>()};
94 }
95
96 int main()
97 {
98     shared_ptr<Pool<Product>> pool = Pool<Product>::instance();
99
100    vector<shared_ptr<Product>> vec(4);
101
102    for (auto& elem : vec) {
103        elem = pool->getObject();
104    }
105
106    pool->releaseObject(vec[1]);

```


30. Структурные паттерны: адаптер (Adapter), декоратор (Decorator), компоновщик (Composite), мост (Bridge), заместитель (Proxy), фасад (Facade). Их преимущества и недостатки.

30.1. Структурные паттерны

Структурные паттерны — это группа шаблонов проектирования, которая решает задачи, связанные с организацией классов и объектов в более крупные структуры.

30.2. Адаптер (Adapter)

- 1) В одном месте программы с сущностью работают через один интерфейс, в другом — через другой. Это делается адаптерами. Программист, модифицируя свою часть, не трогает саму сущность — он работает только со «своим» интерфейсом (своим адаптером). Остальные части кода не страдают, всё продолжает работать.
— Если сущность меняется, для неё делают новые адаптеры. Каждый у себя создаёт адаптер — написанный код не трогается.

- 2) Второе использование адаптера: на одну роль могут возлагаться несколько обязанностей; возникает потребность менять каждую обязанность независимо. Тогда для каждой обязанности делают адаптер.

- 3) Проблема абстрактной фабрики/семейств: хотим подменять одно семейство объектов на другое (например, Рен из разных графических библиотек), но у них нет общей базы.

Идея: сделать адаптер «над Рен-ом» одной библиотеки и адаптер — над другой, связать их общей абстракцией. Тогда можно подменять объекты между библиотеками.

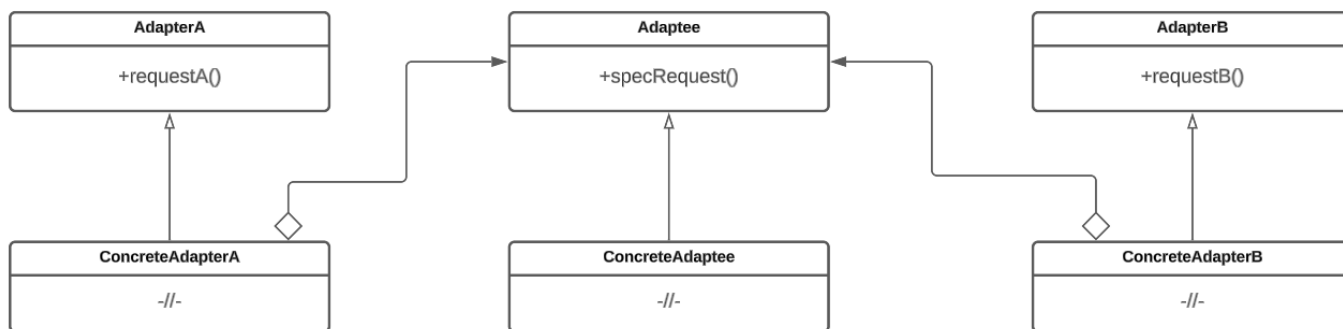
— Жёсткое правило: если используем нестандартную библиотеку, работать с её сущностями нужно только через адаптер. Библиотеку могут перестать поддерживать; найдём лучше — заменим. Через адаптеры это делается легко, без выпиливания по всему коду, не изменяя написанный код.

- 4) Ещё одна важная проблема — «реализация» в наследниках (добавляем методы, которых нет в базовой абстракции) — это плохо. Как работать с расширенным интерфейсом, не меняя базовую абстракцию и не переписывая иерархию? Идея: расширить интерфейс за счёт адаптера.

— Адаптер может адаптировать целую иерархию (добавить функционал сразу всем).

— Уходим от дублирования кода и от проблем с приведением типов (спрашивать «ты кто» и поддерживаешь ли интерфейс — не нужно).

— Через адаптер можно «расширенно» работать с любой сущностью.



30.2.1. Преимущества

- + Изоляция изменений: программист меняет только свой интерфейс (свой адаптер), сущность и чужие части кода не трогаются; написанный код не изменяется.
- + Лёгкая подмена библиотек и семейств: через адаптеры связываем разные реализации общей абстракцией и подменяем без переписывания.
- + Расширение интерфейса без изменения иерархии: можно добавить функционал через адаптер, сразу для всей иерархии.
- + Уходим от проблем с приведением типов и от дублирования (в случае добавления общего функционала через один адаптер).

30.2.2. Недостатки

- Резкое увеличение числа классов-обёрток.
- Всё держится на полиморфизме — это влияет на время выполнения, причём резко.
- При нескольких ролях интерфейсы могут пересекаться — дублирование кода.

30.2.3. Пример

```

1  #include <iostream>
2  #include <memory>
3
4  using namespace std;
5
6  class BaseAdaptee
7  {
8  public:
9      virtual ~BaseAdaptee() = default;
10
11     virtual void specificRequest() = 0;
12 };
13
14 class ConAdaptee : public BaseAdaptee
15 {
16 public:
17     void specificRequest() override {}
18 };
19
20 class Adapter
21 {

```

```

22     public:
23         virtual ~Adapter() = default;
24
25         virtual void request() = 0;
26 };
27
28 class ConAdapter : public Adapter
29 {
30     private:
31         shared_ptr<BaseAdaptee> adaptee;
32
33     public:
34         ConAdapter(shared_ptr<BaseAdaptee> ad) : adaptee(ad) {}
35
36         void request() override;
37 };
38
39 void ConAdapter::request()
40 {
41     cout << "Adapter: ";
42
43     if (adaptee) {
44         adaptee->specificRequest();
45     } else {
46         cout << "Empty!" << endl;
47     }
48 }
49
50 int main()
51 {
52     shared_ptr<BaseAdaptee> adaptee = make_shared<ConAdaptee>();
53     shared_ptr<Adapter> adapter = make_shared<ConAdapter>(adaptee);
54
55     adapter->request();
56 }

```

30.3. Декоратор (Decorator)

30.3.1. Контекст проблемы (почему нужен)

При рекурсивном дизайне мы наследуемся и добавляем поведение в производных классах (до/после базового метода). Это ведет к разрастанию иерархии и дублированию кода, особенно если одно и то же «добавление» нужно ко многим сущностям или в разных комбинациях.

30.3.2. Идея и определение

— То, что «добавляем», выносим в отдельную сущность того же класса, которая будет добавлять поведение к каждой конкретной сущности.

- Паттерн Декоратор: имеет тот же интерфейс, что и базовый компонент, и содержит указатель на компонент. Мы «оборачиваем» объект и добавляем нужное поведение.

30.3.3. Структура и как работает

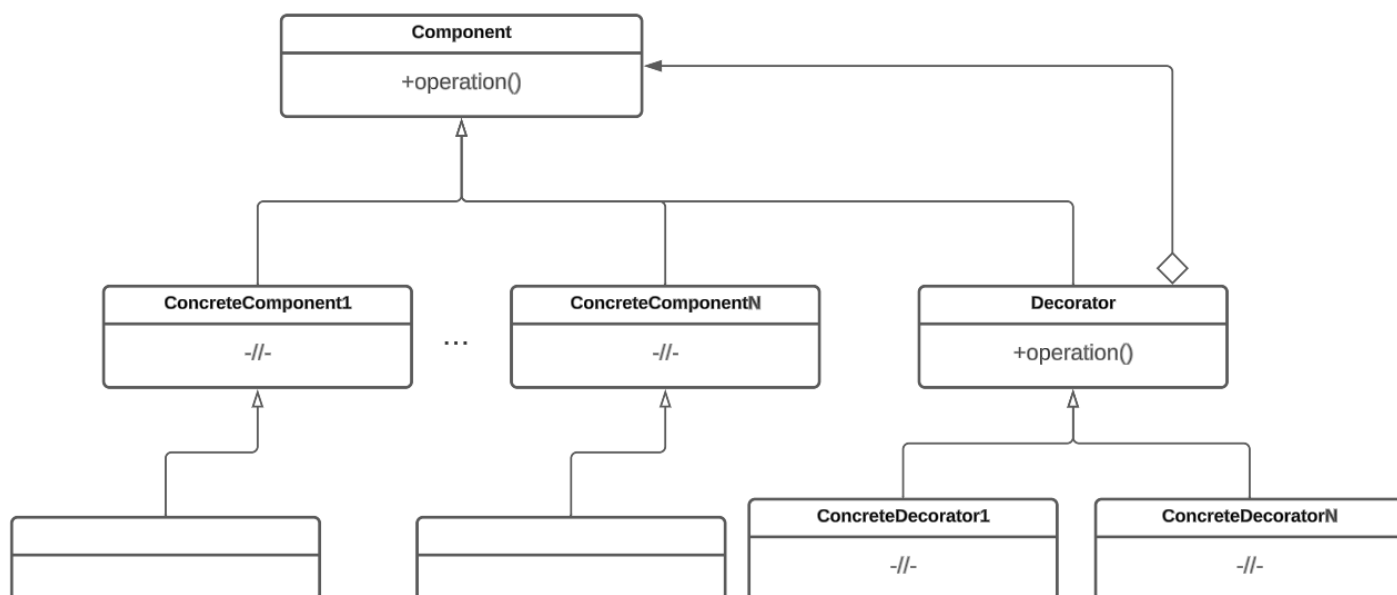
- Декоратор — производный от компонента (тот же интерфейс) и держит указатель на этот компонент.
- Можно декорировать декоратор: декоратор сам может быть ещё чем-то продекорирован. Таким образом, можно наращивать функциональность цепочкой.
- Добавления до и после базового вызова лучше разносить в разные декораторы.
- Декоратор не обязан вызывать метод базового компонента. Пример: «заккрыть» объект — сам объект тогда не будет рисоваться (декоратор может перехватить и не вызывать базовый метод).
- В коде мы фактически заменяем прямое обращение к объекту на обращение через указатель на компонент (который может указывать на декорированный объект).

30.3.4. Использование

- Объект создаем «креатором» (фабрикой) как обычно, а декорировать можем на этапе выполнения: поведение объекта может меняться во время выполнения программы.
- Можно легко «мигрировать» поведение: не нужно перегонять объект из одного класса в другой — просто продекорировали.
- Можно менять декораторы или «переназначать» указатель на другой объект — и тот окажется продекорирован.

30.3.5. Замечания по проектированию

- Декоратор часто делают абстрактным, а конкретные «добавления» — отдельными конкретными декораторами.
- Нужен «главный решала», кто принимает решение и знает, как именно и в какой последовательности декорировать (эту логику надо учитывать в креаторах).



30.3.6. Преимущества

- + Нет разрастания иерархии: то, что добавляем, выносим в декораторы.
- + Нет дублирования кода: общее добавление реализуется один раз в декораторе.
- + Гибкость: можно декорировать во время выполнения, поведение объекта меняется динамически.
- + Можно декорировать декоратор (комбинаторика добавлений).
- + Декоратор может перехватывать вызовы и не вызывать базовый метод (например, «закрыть» отрисовку объекта).

30.3.7. Недостатки

- Создание продекорированных объектов — «не простой» креатор: нужна правильная последовательность декорирования, это сложная логика, её надо учитывать в фабриках; должен быть «главный решала».
- Нельзя просто «выкинуть один декоратор из середины» цепочки: чтобы исключить декоратор, нужно заново продекорировать объект.
- Производительность: всё на полиморфизме и виртуальных таблицах; цепочка декораторов — куча виртуальных таблиц, проход по ним замедляет выполнение.
- Риск ошибок конфигурации: «программисту доверять нельзя» — легко неправильно собрать цепочку, если не централизовать решение.

30.3.8. Пример

```
1 #include <iostream>
2 #include <memory>
3
4 using namespace std;
5
6 class Component
7 {
8     public:
9         virtual ~Component() = default;
10
11         virtual void operation() = 0;
12 };
13
14 class ConComponent : public Component
15 {
16     public:
17         void operation() override { cout << "ConComponent; "; }
18 };
19
20 class Decorator : public Component
21 {
22     protected:
23         shared_ptr<Component> component;
24
25     public:
```

```

26     Decorator(shared_ptr<Component> comp) : component(comp) {}
27 };
28
29 class ConDecorator : public Decorator
30 {
31     public:
32         using Decorator::Decorator;
33
34         void operation() override;
35 };
36
37 void ConDecorator::operation()
38 {
39     if (component) {
40         component->operation();
41         cout << "ConDecorator; ";
42     }
43 }
44
45 int main()
46 {
47     shared_ptr<Component> component = make_shared<ConComponent>();
48     shared_ptr<Component> decorator1 =
49     make_shared<ConDecorator>(component);
50
51     decorator1->operation();
52     cout << endl;
53
54     shared_ptr<Component> decorator2 =
55     make_shared<ConDecorator>(decorator1);
56
57     decorator2->operation();
58     cout << endl;
59 }

```

30.4. Компоновщик (Composite)

30.4.1. Основные идеи компоновщика

- Компоновщик (Composite) нужен, когда есть объекты, которые могут объединяться в группы и над которыми нужно выполнять одинаковые операции.
- Компоновщик сам является компонентом, то есть Composite наследуется от того же интерфейса/абстракции, что и отдельные компоненты. Благодаря этому над композитом можно выполнять те же действия (например, повернуть, перенести), что и над отдельным компонентом.
- Содержит коллекцию компонентов. Для этого в компоновщике используется контейнер (чаще всего стандартный контейнер типа `vector`, `list`):

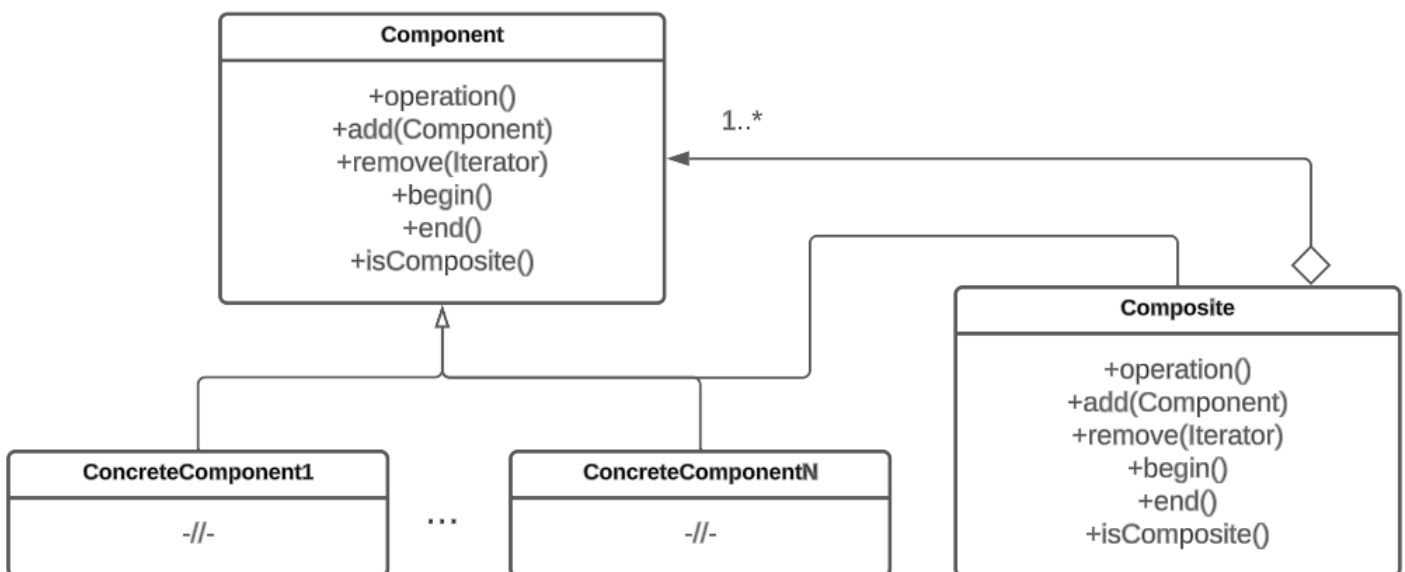
- С композитом можно работать так же, как и с отдельными элементами, не погружаясь в детали и структуру вложенности — это главное преимущество.

30.4.2. Особенности интерфейса

- Абстракция (интерфейс компонента) должна предусматривать методы для работы с группами (например, добавить, удалить компонент, получить доступ к компоненту), хотя отдельный компонент эти методы реализовать не может.
- Для типизации всегда добавляют метод, позволяющий отличить композит от простого компонента:
- Один и тот же объект может входить в разные композиты. Изменение объекта в одном композите приведёт к изменению и в других.
- Это удобно для проектирования и позволяет строить иерархии объектов разного уровня вложенности.

30.4.3. Примеры компоновщиков

- Машина: это композит из двигателя, коробки передач, и других частей, которые, в свою очередь, могут быть тоже композитами.
- Один и тот же объект может быть частью разных композитов (например, компонент двигателя и компонент всей машины).



30.4.4. Преимущества

- + Прячутся циклы: работа с коллекциями компонентов организована через компоновщик (внутри), снаружи же пользователь работает с композитом как с обычным объектом.
- + Гибкое добавление и удаление компонентов: можно легко расширять систему, добавляя новые виды компонентов и композитов.
- + Удобство проектирования и расширяемость: можно создавать разветвлённые и иерархические структуры произвольной глубины.

30.4.5. Недостатки

- Вызовы по виртуальным функциям: обращение происходит через виртуальный интерфейс, что может накладывать издержки на производительность.
- Необходимость итерирования по композиту: для некоторых задач (например, удержание фокуса на отдельном объекте) требуется обход и итерирование внутри композита, что может усложнять логику.
- Один и тот же объект в разных композитах требует особого внимания — изменение объекта в одном месте влияет и на все остальные композиты с этим объектом.
- Некоторые методы интерфейса (добавление/удаление компонента) бессмысленны для простых компонентов: их приходится реализовывать как пустые или «заглушки», чтобы соответствовать общему интерфейсу.

30.4.6. Пример

```
1 #include <initializer_list>
2 #include <iostream>
3 #include <memory>
4 #include <vector>
5
6 using namespace std;
7
8 class Component;
9
10 using PtrComponent = shared_ptr<Component>;
11 using VectorComponent = vector<PtrComponent>;
12
13 class Component
14 {
15     public:
16         using value_type = Component;
17         using size_type = size_t;
18         using iterator = VectorComponent::const_iterator;
19         using const_iterator = VectorComponent::const_iterator;
20
21         virtual ~Component() = default;
22
23         virtual void operation() = 0;
24
25         virtual bool isComposite() const { return false; }
26         virtual bool add(initializer_list<PtrComponent> comp) { return
false; }
27         virtual bool remove(const iterator& it) { return false; }
28         virtual iterator begin() const { return iterator(); }
29         virtual iterator end() const { return iterator(); }
30 };
31
32 class Figure : public Component
33 {
```

```

34     public:
35         virtual void operation() override { cout << "Figure method;" <<
endl; }
36     };
37
38     class Camera : public Component
39     {
40     public:
41         virtual void operation() override { cout << "Camera method;" <<
endl; }
42     };
43
44     class Composite : public Component
45     {
46     private:
47         VectorComponent vec;
48
49     public:
50         Composite() = default;
51         Composite(PtrComponent first, ...);
52
53         void operation() override;
54
55         bool isComposite() const override { return true; }
56         bool add(initializer_list<PtrComponent> list) override;
57         bool remove(const iterator& it) override
58         {
59             vec.erase(it);
60             return true;
61         }
62         iterator begin() const override { return vec.begin(); }
63         iterator end() const override { return vec.end(); }
64     };
65
66     Composite::Composite(PtrComponent first, ...)
67     {
68         for (shared_ptr<Component>* ptr = &first; *ptr; ++ptr) {
69             vec.push_back(*ptr);
70         }
71     }
72
73     void Composite::operation()
74     {
75         cout << "Composite method:" << endl;
76         for (auto elem : vec) {
77             elem->operation();
78         }
79     }

```

```

80
81 bool Composite::add(initializer_list<PtrComponent> list)
82 {
83     for (auto elem : list) {
84         vec.push_back(elem);
85     }
86     return true;
87 }
88
89 int main()
90 {
91     using Default = shared_ptr<Component>;
92     PtrComponent fig = make_shared<Figure>();
93     PtrComponent cam = make_shared<Camera>();
94     auto compositel = make_shared<Composite>(fig, cam, Default{});
95
96     compositel->add({make_shared<Figure>(), make_shared<Camera>()});
97     compositel->operation();
98     cout << endl;
99
100     auto it = compositel->begin();
101
102     compositel->remove(++it);
103     compositel->operation();
104     cout << endl;
105
106     auto composite2 = make_shared<Composite>(make_shared<Figure>(),
107     compositel, Default());
108     composite2->operation();
109 }

```

30.5. Мост (Bridge)

30.5.1. Определение и основная идея

Паттерн «Мост» предназначен для того, чтобы разделить сущность и её реализацию, что позволяет независимо изменять и сущность, и реализацию.

30.5.2. Зачем нужен паттерн Мост

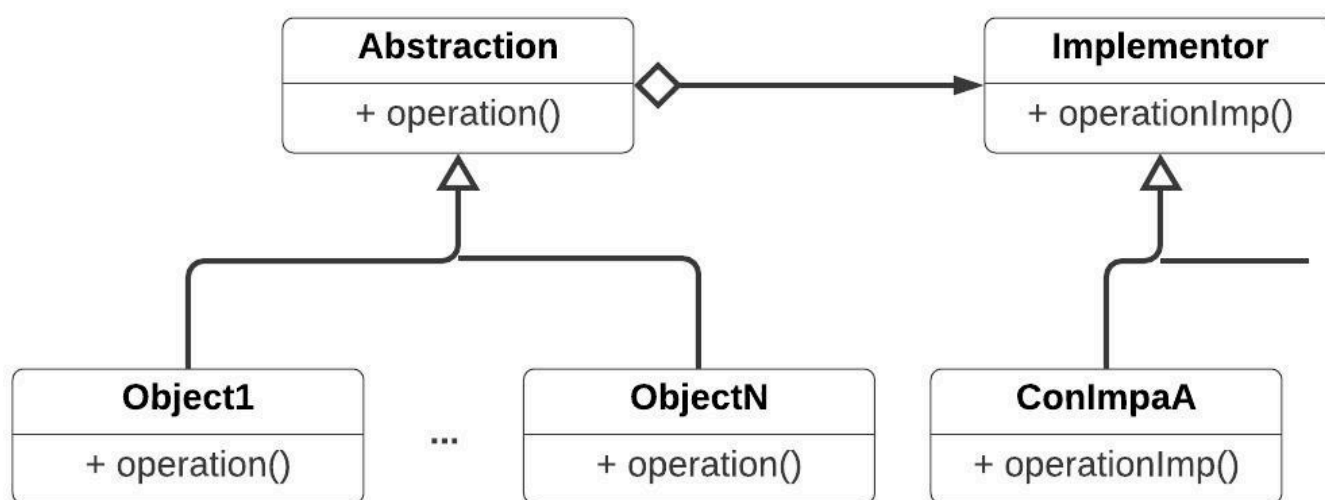
- Когда у объекта есть несколько возможных реализаций, и эти реализации могут меняться независимо от самой абстракции.
- Если мы заранее понимаем, что реализация может меняться, используем Мост.
 - Пример: модель можно представлять списком вершин и рёбер или в виде матрицы смежности — объект остаётся тем же, реализация — разная.

30.5.3. Реализация

- Обычно изображают схемой (диаграммой): одна ветка — абстракция, другая — реализация, между ними устанавливается связь.
- Не всегда удастся свести к дереву базовых абстракций, потому что реализации не обязаны быть древовидными.
- Иногда понимание использования «моста» видно по суффиксу Impl (implementation) в названиях методов и объектов.

30.5.4. Эффект от применения

- Мы можем легко породить новые производные классы, а также новые реализации — то есть, расширяемость по двум измерениям сразу.
- В коде становится очевидно, что сущность отделена от своей реализации — так повышается гибкость.



30.5.5. Преимущества

- + Гибкость архитектуры: мы можем независимо менять сущность и независимо менять реализацию.
- + Легко добавлять новые реализации без изменения существующих абстракций.
- + Обеспечивает удобную структуру для поддержки множества вариантов реализации.

30.5.6. Недостатки

- Этот паттерн довольно сложный, он требует более сложного проектирования.
- На практике не всегда удастся свести к связи базовых абстракций, потому что реализация не всегда выглядит в виде дерева.
- Может появиться избыточность связей, если архитектура проекта не требует такой гибкости.
- Приводит к усложнению кода и необходимости тщательнее проектировать и следить за структурой системы.

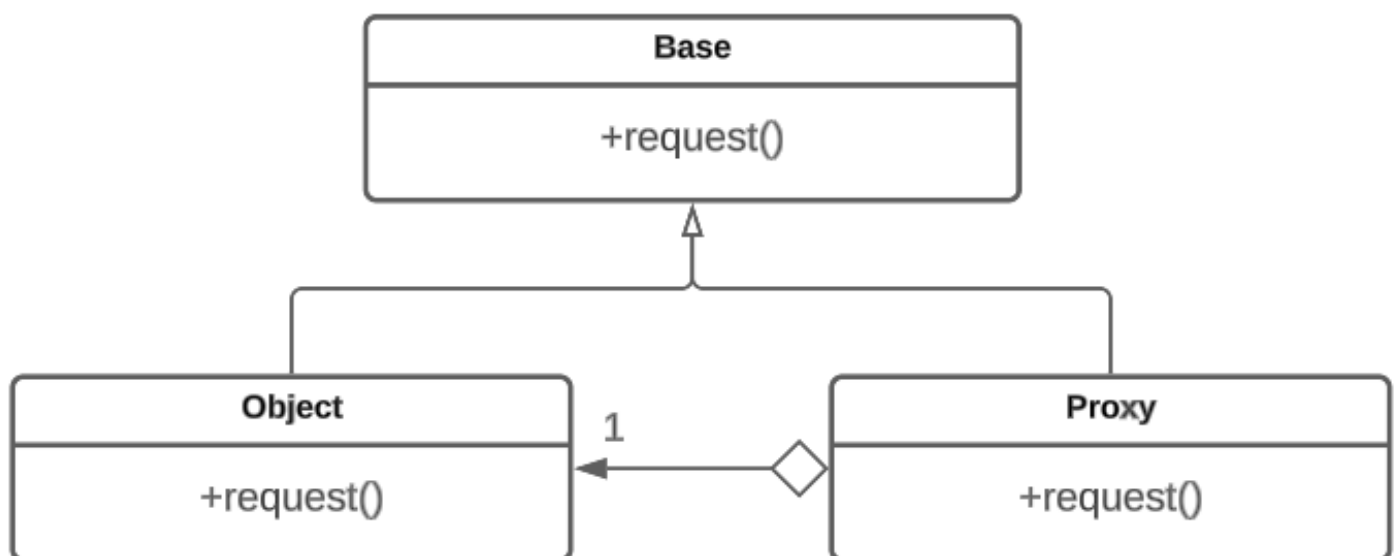
30.5.7. Пример

```
1 #include <iostream>
2 #include <memory>
3
4 using namespace std;
5
6 class Implementor
7 {
8     public:
9         virtual ~Implementor() = default;
10
11         virtual void operationImp() = 0;
12 };
13
14 class Abstraction
15 {
16     protected:
17         shared_ptr<Implementor> implementor;
18
19     public:
20         Abstraction(shared_ptr<Implementor> imp) : implementor(imp) {}
21         virtual ~Abstraction() = default;
22
23         virtual void operation() = 0;
24 };
25
26 class ConImplementor : public Implementor
27 {
28     public:
29         void operationImp() override { cout << "Implementor;\n"; }
30 };
31
32 class Entity : public Abstraction
33 {
34     public:
35         using Abstraction::Abstraction;
36
37         void operation() override
38         {
39             cout << "Entity: ";
40             implementor->operationImp();
41         }
42 };
43
44 int main()
45 {
46     shared_ptr<Implementor> implementor = make_shared<ConImplementor>();
47     shared_ptr<Abstraction> abstraction = make_shared<Entity>(implementor);
```

```
48  
49     abstraction->operation();  
50 }
```

30.6. Заместитель (Proxy)

- Мы создаём объект, через который будем получать доступ к нашим объектам.
 - Заместитель, или прокси (проху), входит в общую иерархию с объектом и через него мы обращаемся к нашему объекту.
- 1) Прокси нужен для того, чтобы, например, реализовать функцию кэширования запросов:
 - Когда мы делаем какой-то запрос через проху, он смотрит: был такой запрос или нет.
 - Если такой запрос уже был, проху даёт уже сохранённый ответ, не обращаясь к самому объекту.
 - Однако есть нюанс: объект мог измениться, и старые ответы уже нерелевантны — тогда надо снова обращаться к объекту.
 - 2) Проху также может фильтровать запросы, т.е. не все запросы допускать до реального объекта (например, фильтрация по правам пользователя).
 - 3) Проху может выполнять шифрование и дешифрование запросов и ответов.
 - 4) Можно использовать проху для сбора статистики, аналитики по обращениям к объекту и других служебных функций.
 - 5) Проху может отвечать за жизненный цикл объекта:
 - Иногда объекты «тяжёлые», поэтому их не стоит создавать сразу — проху может сам управлять созданием, когда это действительно потребуется.
 - Важно понимать ситуацию, что объекта ещё нет или он уже отсутствует, а проху возвращает, будто он есть.



30.6.1. Преимущества

- + Экономия ресурсов: позволяет кэшировать запросы и повторно использовать уже полученные результаты.

- + Обеспечивает фильтрацию или разграничение доступа к объекту за счёт дополнительных проверок на уровне проху.
- + Расширение функционала без изменения кода «основного объекта»: может брать на себя функции шифрования, дешифрования, сбора статистики, аналитики.
- + Управляет жизненным циклом тяжелых объектов: создаёт их только тогда, когда это необходимо.

30.6.2. Недостатки

- Необходимость учитывать, что данные в проху могут устаревать (если основной объект изменился).
- Иногда приводит к усложнению архитектуры за счёт необходимости отслеживать актуальность кэшированных данных и состояния реального объекта.
- Увеличивается время выполнения операций: каждый запрос идёт через дополнительный уровень (проху).

30.6.3. Пример

```

1  #include <iostream>
2  #include <map>
3  #include <memory>
4  #include <random>
5
6  using namespace std;
7
8  class Subject
9  {
10 public:
11     virtual ~Subject() = default;
12
13     virtual pair<bool, double> request(size_t index) = 0;
14     virtual bool changed() { return true; }
15 };
16
17 class RealSubject : public Subject
18 {
19 private:
20     bool flag{false};
21     size_t counter{0};
22
23 public:
24     pair<bool, double> request(size_t index) override;
25     bool changed() override;
26 };
27
28 class Proxy : public Subject
29 {
30 protected:
31     shared_ptr<RealSubject> realsubject;

```

```

32
33     public:
34         Proxy(shared_ptr<RealSubject> real) : realsubject(real) {}
35 };
36
37 class ConProxy : public Proxy
38 {
39     private:
40         map<size_t, double> cache;
41
42     public:
43         using Proxy::Proxy;
44
45         pair<bool, double> request(size_t index) override;
46 };
47
48 bool RealSubject::changed()
49 {
50     if (counter == 0) {
51         flag = true;
52     }
53     if (++counter == 7) {
54         counter = 0;
55         flag = false;
56     }
57     return flag;
58 }
59
60 pair<bool, double> RealSubject::request(size_t index)
61 {
62     random_device rd;
63     mt19937 gen(rd());
64
65     return pair<bool, double>(true, generate_canonical<double, 10>(gen));
66 }
67
68 pair<bool, double> ConProxy::request(size_t index)
69 {
70     pair<bool, double> result;
71
72     if (!realsubject) {
73         cache.clear();
74         result = pair<bool, double>(false, 0.);
75     } else if (!realsubject->changed()) {
76         cache.clear();
77         result = realsubject->request(index);
78         cache.insert(map<size_t, double>::value_type(index,
result.second));

```

```

79     } else {
80         map<size_t, double>::const_iterator it = cache.find(index);
81
82         if (it != cache.end()) {
83             result = pair<bool, double>(true, it->second);
84         } else {
85             result = realsubject->request(index);
86             cache.insert(map<size_t, double>::value_type(index,
result.second));
87         }
88     }
89
90     return result;
91 }
92
93 int main()
94 {
95     shared_ptr<RealSubject> subject = make_shared<RealSubject>();
96     shared_ptr<Subject> proxy = make_shared<ConProxy>(subject);
97
98     for (size_t i = 0; i < 21; ++i) {
99         cout << "( " << i + 1 << ", " << proxy->request(i % 3).second
<< " )\n";
100
101         if ((i + 1) % 3 == 0) {
102             cout << endl;
103         }
104     }
105 }

```

30.7. Фасад (Facade)

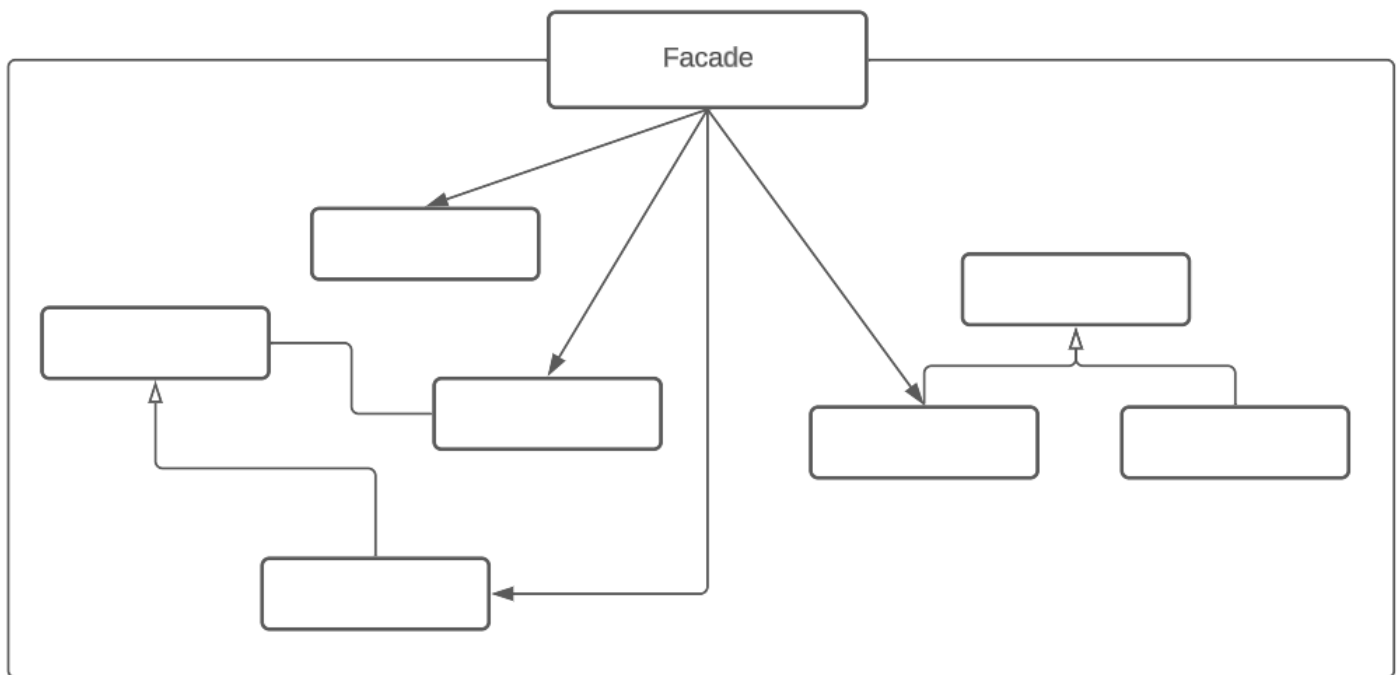
- Представим, что у нас не один объект, а «мир объектов», между которыми существуют связи, и часто нужно следить за этими связями, за целостностью этих объектов.
- Паттерн «Фасад» похож на паттерн «Адаптер», но это не адаптер. Фасад скрывает от нас множество объектов.

Я (лектор) общаюсь с вами сейчас как раз через фасад — не взаимодействую с каждым из вас по отдельности, а работаю с вашим обобщённым интерфейсом (фасадом).

- Фасад скрывает внутреннюю реализацию множества объектов и их связей.
- Задача фасада — упростить взаимодействие с «миром объектов» и чётко выделить интерфейс этого мира.
- Функция фасада — обеспечить целостность мира.

Я (лектор) взаимодействую с вами через фасад, который не даёт мне «разрушить вашу целостность».

- Важно проектировать систему так, чтобы извне нельзя было нарушить целостность: ничего не должно торчать наружу.
- Фасад выступает в роли оболочки.



30.7.1. Преимущества

- + Облегчает взаимодействие с системой, предоставляя единый, простой интерфейс.
- + Скрывает сложную внутреннюю структуру множества объектов и их связей.
- + Обеспечивает целостность внутреннего мира объектов — защищает объекты от некорректного внешнего воздействия.
- + Внешний код не может «разрушить» или «нарушить» внутреннюю структуру, если взаимодействует только через фасад.

30.7.2. Недостатки

- Если что-то торчит наружу, то есть интерфейс неудачно спроектирован, то фасад теряет смысл — целостность может быть нарушена.
- Фасад не всегда предотвращает попытки пользователя «ворваться» в систему, особенно если изначально не был чётко выделен внешний интерфейс.

30.7.3. Пример

```

1 #include <iostream>
2 #include <string>
3
4 // --- Подсистемы ---
5 class Amplifier
6 {
7     public:
8         void on() { std::cout << "Amplifier: on\n"; }
9         void off() { std::cout << "Amplifier: off\n"; }
10        void setVolume(int v) { std::cout << "Amplifier: volume " << v <<
11        "\n"; }
12 };
13 class DvdPlayer

```

```

14 {
15     public:
16         void on() { std::cout << "DVD: on\n"; }
17         void off() { std::cout << "DVD: off\n"; }
18         void play(const std::string& title) { std::cout << "DVD: play \"" <<
title << "\"\n"; }
19         void stop() { std::cout << "DVD: stop\n"; }
20 };
21
22 class Projector
23 {
24     public:
25         void on() { std::cout << "Projector: on\n"; }
26         void off() { std::cout << "Projector: off\n"; }
27         void wideMode() { std::cout << "Projector: wide mode\n"; }
28 };
29
30 class Screen
31 {
32     public:
33         void down() { std::cout << "Screen: down\n"; }
34         void up() { std::cout << "Screen: up\n"; }
35 };
36
37 class Lights
38 {
39     public:
40         void dim(int level) { std::cout << "Lights: dim to " << level <<
"%\n"; }
41         void on() { std::cout << "Lights: on\n"; }
42 };
43
44 // --- Фасад ---
45 class HomeTheaterFacade
46 {
47     Amplifier amp;
48     DvdPlayer dvd;
49     Projector projector;
50     Screen screen;
51     Lights lights;
52
53     public:
54         // Сценарий «посмотреть фильм»
55         void watchMovie(const std::string& title)
56         {
57             std::cout << "\n== Watch movie ==\n";
58             lights.dim(20);
59             screen.down();

```

```

60     projector.on();
61     projector.wideMode();
62     amp.on();
63     amp.setVolume(7);
64     dvd.on();
65     dvd.play(title);
66     std::cout << "== Enjoy! ==\n\n";
67 }
68
69 // Сценарий «закончить просмотр»
70 void endMovie()
71 {
72     std::cout << "== End movie ==\n";
73     dvd.stop();
74     dvd.off();
75     amp.off();
76     projector.off();
77     screen.up();
78     lights.on();
79     std::cout << "== Done ==\n\n";
80 }
81 };
82
83 // --- Клиент ---
84 int main()
85 {
86     HomeTheaterFacade theater;
87     theater.watchMovie("Inception");
88     theater.endMovie();
89 }

```

31. Паттерны поведения: стратегия (Strategy), команда (Command), цепочка обязанностей (Chain of Responsibility), подписчик-издатель (Publish-Subscribe), посредник (Mediator). Их преимущества и недостатки.

31.1. Паттерны поведения

31.1.1. Что такое паттерны поведения

- Это подход, где мы выносим изменяемые части поведения (реализации алгоритмов/методов) из сущности в отдельные сущности и подставляем их при работе.
- Формулировка через сравнение: «вырожденный мост» — это паттерн поведения; если мост отделяет сущность от реализации «в целом», то в поведенческом подходе отделяется реализация одного конкретного метода (алгоритма).

31.1.2. Зачем они нужны (какие проблемы решают)

- Уход от реализации: позволяют уйти от реализации в смысле разрастания иерархий за счет разных производных, когда методы по-разному реализуются. Вместо расширения иерархии — выносим то, что меняем, в отдельные сущности поведения.
- Борьба с разрастанием иерархии и дублированием кода: изменения концентрируются в «поведении», базовая сущность не плодит наследников и не копирует одинаковые куски.
- Гибкость: легко менять поведение объекта во время выполнения (подменой поведения).
- Переиспользование: одно и то же «поведение» можно применять в разных ролях/для разных сущностей.

31.2. Стратегия (Strategy)

31.2.1. Главная идея паттерна

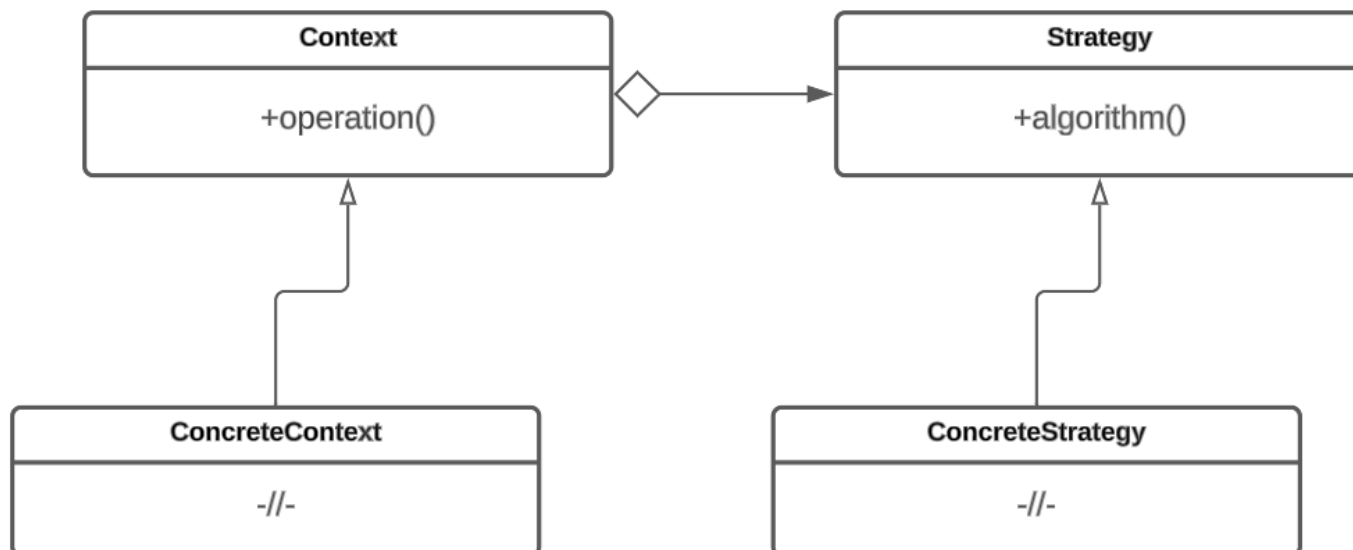
Всё, что меняется (например, алгоритм), выносится в отдельный объект стратегии. Можно легко поменять поведение объекта во время выполнения, просто подменяя стратегию.

31.2.2. Пример с сортировкой (классическая реализация)

- В языке Си: чтобы отсортировать, например, студентов по различным признакам, передаётся указатель на функцию сравнения — это callback-функция.
- Но с функциями работать мы не будем, а будем оборачивать функцию в класс — вот это и есть паттерн стратегия.

31.2.3. Роль стратегии

- Выносит конкретную реализацию метода (алгоритма) в отдельную сущность.
- У одного объекта может быть несколько методов, и для каждого — отдельная иерархия стратегий.
- Всё изменение поведения происходит за счёт стратегий — «нам не нужно разрастание иерархии сущностей, весь функционал выносим в стратегии».



31.2.4. Преимущества

- + Позволяет легко менять поведение объекта во время выполнения.
- + Избавляет от дублирования кода.
- + Избавляет от разрастания иерархии классов.
- + Одну и ту же стратегию можно использовать для разных ролей и сущностей.
- + Позволяет реализовать гибкое расширение поведения, альтернативу традиционному наследованию.

31.2.5. Недостатки

- Происходит разрастание числа классов для стратегий.
- Все изменения функциональности привязаны к стратегиям — возможны сложности с их организацией и поддержкой.
- Иерархия стратегий может быть громоздкой, если для каждого метода нужна своя стратегия.
- Конкретная стратегия может не работать с данными определенного класса, что приводит к появлению зависимостей между конкретными сущностями и стратегиями.

31.2.6. Пример

```

1 #include <iostream>
2 #include <memory>
3
4 using namespace std;
5
6 class Strategy
7 {
8     public:
9         virtual ~Strategy() = default;
10
11         virtual void algorithm() = 0;
12 };
13
  
```

```

14 class ConStrategy1 : public Strategy
15 {
16     public:
17         void algorithm() override { cout << "Algorithm 1;" << endl; }
18 };
19
20 class ConStrategy2 : public Strategy
21 {
22     public:
23         void algorithm() override { cout << "Algorithm 2;" << endl; }
24 };
25
26 class Context
27 {
28     protected:
29         unique_ptr<Strategy> strategy;
30
31     public:
32         explicit Context(unique_ptr<Strategy> ptr =
make_unique<ConStrategy1>()) : strategy(move(ptr)) {}
33         virtual ~Context() = default;
34
35         virtual void algorithmStrategy() = 0;
36 };
37
38 class Client1 : public Context
39 {
40     public:
41         using Context::Context;
42
43         void algorithmStrategy() override { strategy->algorithm(); }
44 };
45
46 int main()
47 {
48     shared_ptr<Context> obj =
make_shared<Client1>(make_unique<ConStrategy2>());
49
50     obj->algorithmStrategy();
51 }

```

31.3. Команда (Command)

31.3.1. Что такое «Команда»

- Наиболее распространённый паттерн для работы с запросами.
- Каждый запрос — переносчик данных. При формировании команды в ней «зашивается» метод, который нужно вызвать (и, по крайней мере, к какому классу относится адресат).

- Выполнение команды — это вызов одного метода. По сути «команда — это один метод».

31.3.2. Как устроено выполнение

- Формируем запрос/команду, передаём её «ответственному», который может её выполнить; он берёт и выполняет, т.е. вызывает нужный метод.
- Обычно объект-адресат либо подставляется перед выполнением (до вызова метода), либо передаётся прямо в метод исполнения.

31.3.3. Данные и параметры

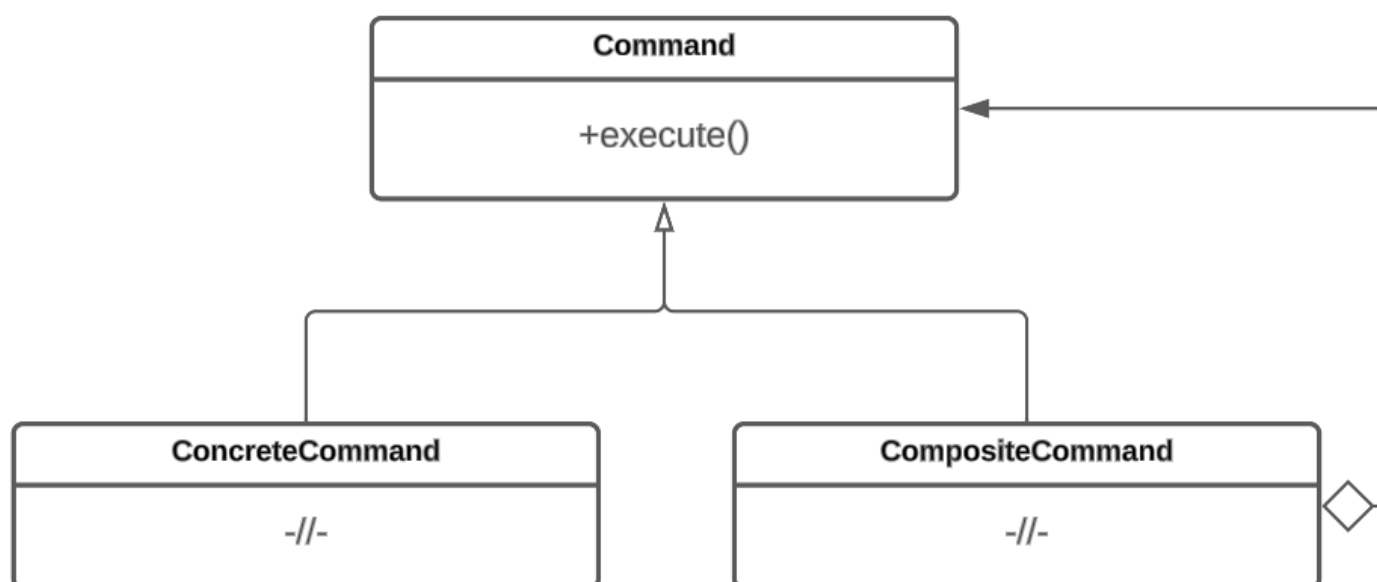
- Данные команды передаются при создании (в конструкторе).
- Параметры можно передавать; чаще всего — это сам объект, для которого вызывается метод.

31.3.4. Составные команды

- Можно создавать «сложные команды», включающие подкоманды.
 - Пример (графика): «повернуть объект» нажатой кнопкой = очистить окно → повернуть объект → нарисовать.
- Составная команда — фактически контейнер под простые команды; важен чёткий порядок выполнения. Специальной сложной манипуляции подкомандами обычно не требуется — главное, чтобы порядок был фиксирован.

31.3.5. Работа с выполнением

- Команду не обязательно выполнять сразу: можно поставить в очередь, выполнить позже или даже проигнорировать (логика обработки может быть произвольной).



31.3.6. Преимущества

- + Простота исполнения: выполнение — вызов одного метода.
- + Можно ставить команды в очередь, откладывать или игнорировать.
- + Есть возможность собирать сложные команды из простых с гарантированным порядком выполнения.

31.3.7. Недостатки

- «Команда — очень жёсткий паттерн»: метод, который нужно вызвать, должен быть зашит в команде — «жёсткая связка с прикладным доменом» и с конкретным методом.
- Рассчитано на одного адресата: если запрос должен выполняться несколькими объектами, это выходит за рамки Команды (нужна «Цепочка обязанностей»).
- На практике часто приходится знать не только метод, но и объект-адресат (его передают заранее или в момент вызова).

31.3.8. Пример

```
1 #include <initializer_list>
2 #include <iostream>
3 #include <memory>
4 #include <vector>
5
6 using namespace std;
7
8 class Command
9 {
10 public:
11     virtual ~Command() = default;
12
13     virtual void execute() = 0;
14 };
15
16 template <typename Receiver>
17 class SimpleCommand : public Command
18 {
19     using Action = void (Receiver::*)();
20     using Pair = pair<shared_ptr<Receiver>, Action>;
21
22 private:
23     Pair call;
24
25 public:
26     SimpleCommand(shared_ptr<Receiver> r, Action a) : call(r, a) {}
27
28     void execute() override { ((*call.first).*call.second)(); }
29 };
30
31 class CompoundCommand : public Command
32 {
33     using VectorCommand = vector<shared_ptr<Command>>;
34
35 private:
36     VectorCommand vec;
37
38 public:
```

```

39     CompoundCommand(initializer_list<shared_ptr<Command>> lt);
40
41     void execute() override;
42 };
43
44 CompoundCommand::CompoundCommand(initializer_list<shared_ptr<Command>> lt)
45 {
46     for (auto&& elem : lt) {
47         vec.push_back(elem);
48     }
49 }
50
51 void CompoundCommand::execute()
52 {
53     for (auto& com : vec) {
54         com->execute();
55     }
56 }
57
58 class Object
59 {
60 public:
61     void run() { cout << "Run method;" << endl; }
62 };
63
64 int main()
65 {
66     shared_ptr<Object> obj = make_shared<Object>();
67     shared_ptr<Command> command = make_shared<SimpleCommand<Object>>(obj,
68     &Object::run);
69
70     command->execute();
71
72     shared_ptr<Command> complex(new CompoundCommand{
73         make_shared<SimpleCommand<Object>>(obj, &Object::run),
74         make_shared<SimpleCommand<Object>>(obj, &Object::run)
75     });
76
77     complex->execute();
78 }

```

31.4. Цепочка обязанностей (Chain of Responsibility)

31.4.1. Определение и идея

- Идея: из тех, кто может обработать запрос, формируется список (цепочка). Мы формируем запрос и отдаём его по списку.
- Первый в списке принимает запрос, смотрит, может ли он его выполнить. Далее варианты:

- 1) может обработать и передать дальше;
 - 2) может обработать и не передавать дальше;
 - 3) может не обрабатывать и передать дальше.
- Запрос передаётся по цепочке, пока не будет обработан (или не дойдёт до конца).

31.4.2. Отвязка от конкретных обработчиков

- Мы чётко отвязываемся: просто формируем запрос и передаём его без привязки к конкретному объекту/методу.
- Запрос — это по существу данные (возможно с какой-то информацией о том, кто бы мог обработать).
- В отличие от «команды», где всё равно есть жёсткая привязка к конкретному методу, здесь хотим полностью отвязаться от конкретного объекта и метода, потому что могут выполнять разные объекты.

31.4.3. Динамика и порядок

- Мы можем динамически формировать тех, кто может обрабатывать запрос: обработчики могут появляться и исчезать, их можно убирать/добавлять в список.
- От порядка обработки (порядка формирования списка) может изменяться результат — это важный момент.

31.4.4. Изменяемость запроса

Запрос может видоизменяться, «обрастать» данными по мере прохождения по цепочке: один обработчик добавил информацию — дальше она уже идёт в обогащённом виде.

Пример (камеры фотовидеофиксации)

- Есть факт прохождения транспортного средства. Сервер распознавания (на столбе) выдаёт один или несколько кадров, пытается распознать ГРЗ, марку автомобиля; в пакет обычно включают также скорость.
- Далее по цепочке возможны обработки:
 - проверка превышения скорости;
 - обращение к базе ГИБДД (сверка распознанной марки и номера);
 - проверка по базе розыска;
 - фильтр на ТС, представляющие оперативный интерес (оперативный сотрудник получает уведомление).
- Если, например, марка не распознана, обработчик, работающий с базой ГИБДД, просто пропускает запрос дальше.

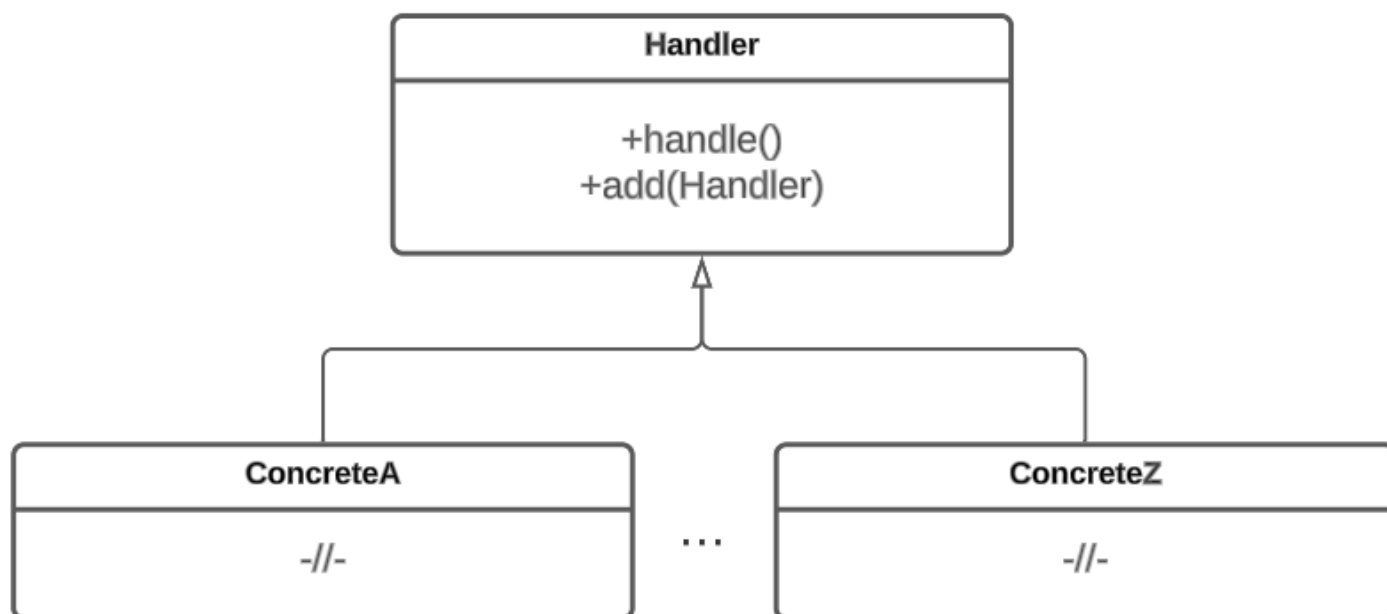
31.4.5. Замечание о невыполнении

- В «команде» в любом случае будет вызван метод конкретного класса (команда выполнится).
- В «цепочке обязанностей» запрос может не выполниться (никто его не обработает).

31.4.6. Структурные заметки (как хранить цепочку)

- Обработчик можно рассматривать как узел списка, который «обрабатывает и передаёт следующему».

- Или рассматривать обработчик как информационную часть узла, а сам список — как контейнер.
- Удобно, что формируем свой контейнер и интерфейс под него:
 - операции добавления/удаления обработчиков;
 - метод «принять запрос»;
 - метод, который возвращает данные/признак, на основе которых понимаем — обрабатываем запрос или нет.
- По цепочке запрос может «обрастать» новой информацией (например, добавились данные из ГИБДД, затем — о владельце ТС и т.д.).



31.4.7. Преимущества

- + Отвязка: запрос формируется и передаётся без жёсткой привязки к конкретному объекту/методу.
- + Динамика: можно динамически формировать список обработчиков (добавлять/убирать).
- + Расширяемость данных: запрос может видоизменяться, «обрастать» информацией по мере прохождения цепочки.
- + Удобство реализации: собственный контейнер и интерфейс под нужные операции.

31.4.8. Недостатки

- Запрос может не выполняться (никто из обработчиков его не обработает).
- Результат зависит от порядка обработки (изменение порядка может менять поведение/итог).

31.4.9. Пример

```

1 #include <initializer_list>
2 #include <iostream>
3 #include <memory>
4
  
```

```

5 using namespace std;
6
7 class AbstractHandler
8 {
9     using PtrAbstractHandler = shared_ptr<AbstractHandler>;
10
11 protected:
12     PtrAbstractHandler next;
13
14     virtual bool run() = 0;
15
16 public:
17     using Default = shared_ptr<AbstractHandler>;
18
19     virtual ~AbstractHandler() = default;
20
21     virtual bool handle() = 0;
22
23     void add(PtrAbstractHandler node);
24     void add(initializer_list<PtrAbstractHandler> list);
25 };
26
27 class ConHandler : public AbstractHandler
28 {
29 private:
30     bool condition{false};
31
32 protected:
33     bool run() override
34     {
35         cout << "Method run;\n";
36         return true;
37     }
38
39 public:
40     ConHandler() : ConHandler(false) {}
41     ConHandler(bool c) : condition(c) { cout << "Constructor;\n"; }
42     ~ConHandler() override { cout << "Destructor;\n"; }
43
44     bool handle() override
45     {
46         if (!condition) {
47             return next ? next->handle() : false;
48         }
49
50         return run();
51     }
52 };

```

```

53
54 void AbstractHandler::add(PtrAbstractHandler node)
55 {
56     if (next) {
57         next->add(node);
58     } else {
59         next = node;
60     }
61 }
62
63 void AbstractHandler::add(initializer_list<PtrAbstractHandler> list)
64 {
65     for (auto elem : list) {
66         add(elem);
67     }
68 }
69
70 int main()
71 {
72     shared_ptr<AbstractHandler> chain = make_shared<ConHandler>();
73
74     chain->add({
75         make_shared<ConHandler>(false),
76         make_shared<ConHandler>(true),
77         make_shared<ConHandler>(true)
78     });
79
80     cout << boolalpha << "Result = " << chain->handle() << ";\n";
81 }

```

31.5. Подписчик-издатель (Publish-Subscribe)

31.5.1. Основная идея

Есть два класса — издатель и подписчик. Подписчик может подписываться на какие-то события или данные от издателя..

31.5.2. Принцип работы

Чтобы получать информацию (например, журнал), нужно «подписаться» — предоставить о себе информацию издателю. Издатель должен держать у себя пару: объект и метод, который нужно вызвать у этого подписчика.

31.5.3. Реализация подписки

- Есть специальный метод (например, `subscribe`), которым подписчик передаёт издателю свои данные, чтобы получать уведомления о событиях.
- Издатель может уведомлять много подписчиков. Для этого он вызывает нужный метод у каждого подписчика.

- Подписка может быть реализована через абстрактный или конкретный класс подписчика.

31.5.4. Ссылки и жизненный цикл объектов

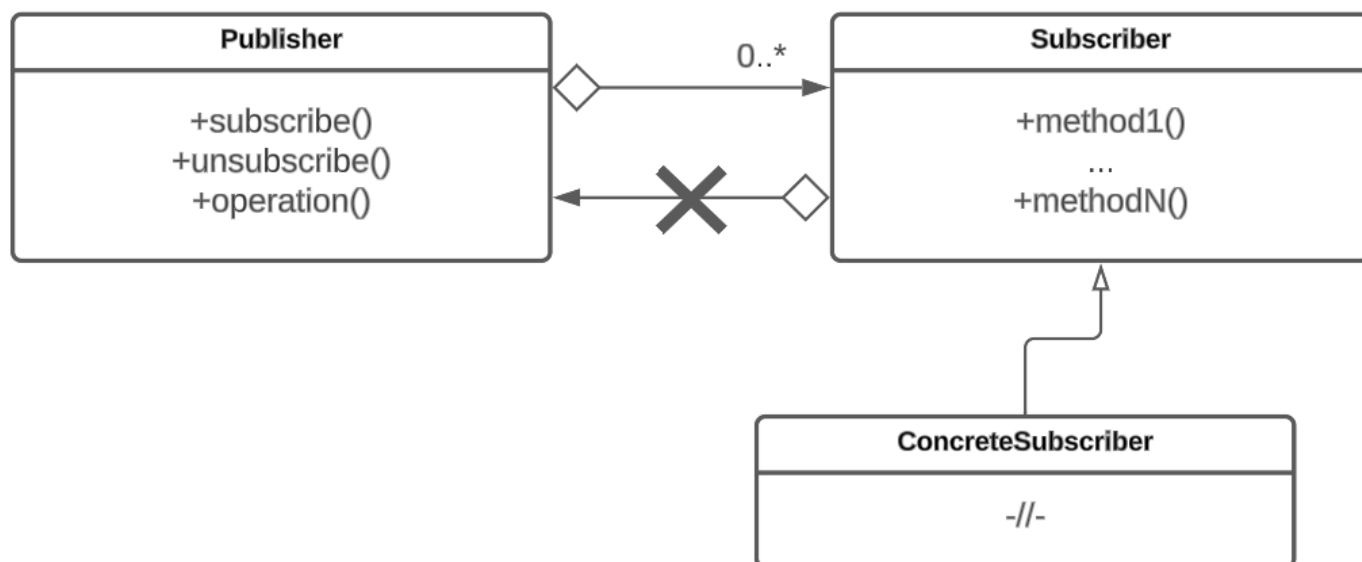
Издатель может держать умный указатель (`weak_ptr` или `shared_ptr`) на подписчика, чтобы отслеживать его существование и избежать утечек памяти. Если подписчику больше не нужно событие — он должен явно отписаться (`unsubscribe`).

31.5.5. Возможные кольцевые зависимости

Если подписчик одновременно становится издателем (и наоборот), могут образоваться кольца — большое количество объектов взаимно ссылаются друг на друга. Это приводит к потенциальным утечкам памяти.

31.5.6. Проблема со списками подписчиков

Каждый издатель должен держать список пар (объект + метод). Если множество объектов и связей между ними, список становится очень большим, что приводит к неудобствам разработки и эксплуатации.



31.5.7. Преимущества

- + Позволяет легко масштабировать систему по числу подписчиков и событий.
- + Подходит для построения асинхронных и событийных систем.
- + Каждый подписчик и издатель могут быть реализованы независимо.
- + Возможна динамическая подписка/отписка — подписчики могут появляться и исчезать во время выполнения программы.

31.5.8. Недостатки

- Кольцевые ссылки (когда подписчики и издатели ссылаются друг на друга) могут привести к утечкам памяти (особенно при использовании `shared_ptr`).
- Каждый издатель вынужден держать у себя список объектов и методов — это расход памяти, рост сложности.

- Если не реализовать правильно механизм отписки, подписчик может получать уведомления даже тогда, когда это больше не нужно.

31.5.9. Пример

```
1 #include <iostream>
2 #include <memory>
3 #include <vector>
4
5 using namespace std;
6
7 class Subscriber;
8
9 using Receiver = Subscriber;
10
11 class Publisher
12 {
13     using Action = void (Receiver::*)();
14     using Pair = pair<shared_ptr<Receiver>, Action>;
15
16     private:
17         vector<Pair> callback;
18
19         int indexOf(shared_ptr<Receiver> r);
20
21     public:
22         bool subscribe(shared_ptr<Receiver> r, Action a);
23         bool unsubscribe(shared_ptr<Receiver> r);
24         void run();
25 };
26
27 class Subscriber
28 {
29     public:
30         virtual ~Subscriber() = default;
31
32         virtual void method() = 0;
33 };
34
35 class ConSubscriber : public Subscriber
36 {
37     public:
38         void method() override { cout << "method;\n"; }
39 };
40
41 bool Publisher::subscribe(shared_ptr<Receiver> r, Action a)
42 {
43     if (indexOf(r) != -1) {
44         return false;
```

```

45     }
46
47     Pair pr(r, a);
48     callback.push_back(pr);
49
50     return true;
51 }
52
53 bool Publisher::unsubscribe(shared_ptr<Receiver> r)
54 {
55     int pos = indexOf(r);
56     if (pos != -1) {
57         callback.erase(callback.begin() + pos);
58     }
59
60     return pos != -1;
61 }
62
63 void Publisher::run()
64 {
65     cout << "Run:\n";
66     for (auto& elem : callback) {
67         ((*elem.first).*(elem.second))();
68     }
69 }
70
71 int Publisher::indexOf(shared_ptr<Receiver> r)
72 {
73     int i = 0;
74     for (auto it = callback.begin(); it != callback.end() && r !=
75 (*it).first; i++, ++it) {}
76
77     return i < callback.size() ? i : -1;
78 }
79
80 int main()
81 {
82     shared_ptr<Subscriber> subscriber1 = make_shared<ConSubscriber>();
83     shared_ptr<Subscriber> subscriber2 = make_shared<ConSubscriber>();
84     shared_ptr<Publisher> publisher = make_shared<Publisher>();
85
86     publisher->subscribe(subscriber1, &Subscriber::method);
87     if (publisher->subscribe(subscriber2, &Subscriber::method)) {
88         publisher->unsubscribe(subscriber1);
89     }
90
91     publisher->run();
92 }

```

31.6. Посредник (Mediator)

31.6.1. Идея паттерна «Посредник»

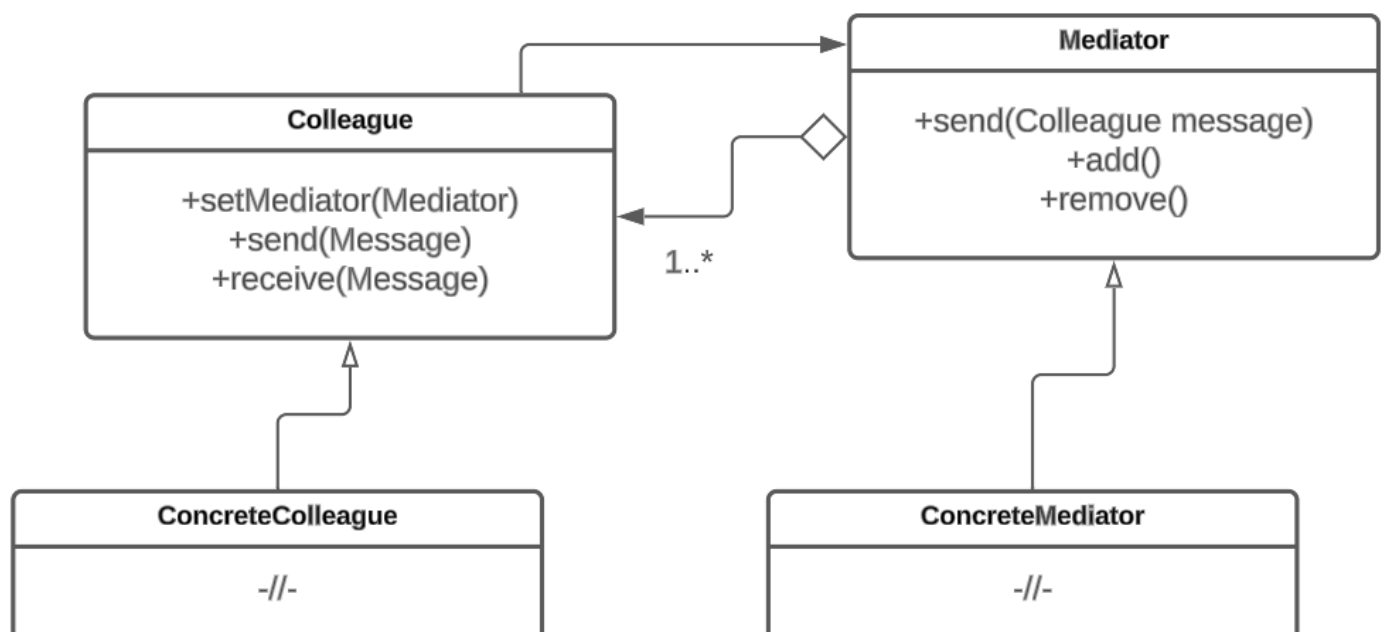
- Все эти связи подписчик-издатель вынести на один объект. Этот объект будет держать все связи, все списки, а объекты будут взаимодействовать только с ним. Соответственно, объекты не держат списков подписчиков или обработчиков — у них связь только с посредником, а посредник, получив сообщение/запрос, решает, кому его отправить.
- Мы разрываем кольцо связей между объектами, у нас один объект-посредник, который знает обо всех, а остальные объекты знают только о посреднике.

31.6.2. Что делает посредник

- Объекты подписываются на посредника, чтобы отправлять запросы.
- Посредник хранит информацию о том, кому и какое сообщение нужно передать.
- Посредник реализует метод, который получает запрос/сообщение и решает, кому его перенаправить.

31.6.3. Возможны проблемы

- У посредника может быть достаточно сложная логика, ведь теперь он отвечает за правильную маршрутизацию запросов.
- Всё равно необходим некий список (например, кто от кого может принимать запросы), и требуется следить за тем, существуют ли нужные объекты.
- Проблем, связанных с замыканиями и утечками памяти, как у подписчика-издателя, здесь уже нет, поскольку нет замкнутых ссылок.
- Посредник — это решение проблемы жёсткой связанности и кольцевых ссылок. Но за это приходится платить повышенной сложностью объекта-посредника, который теперь должен знать всю логику пересылки сообщений между объектами.



31.6.4. Преимущества

- + Разорваны «кольца» ссылок между объектами — нет прямых или взаимных ссылок между всеми элементами системы.
- + Все управляющие списки сосредоточены в одном месте.
- + Нет проблем с утечкой памяти, характерных для схем подписчик-издатель.

31.6.5. Недостатки

- Сложная логика внутри посредника, он становится «узким местом», т.к. должен реализовать все маршруты обмена.
- При больших иерархиях объектов количество логики и связей в посреднике становится большим, это затрудняет сопровождение системы.

31.6.6. Пример

```
1 #include <iostream>
2 #include <list>
3 #include <memory>
4
5 using namespace std;
6
7 class Message
8 {
9 }; // Request
10
11 class Mediator;
12
13 class Colleague
14 {
15     private:
16         weak_ptr<Mediator> mediator;
17
18     public:
19         virtual ~Colleague() = default;
20
21         void setMediator(shared_ptr<Mediator> mdr) { mediator = mdr; }
22
23         virtual bool send(shared_ptr<Message> msg);
24         virtual void receive(shared_ptr<Message> msg) = 0;
25 };
26
27 class ColleagueLeft : public Colleague
28 {
29     public:
30         void receive(shared_ptr<Message> msg) override { cout << "Right - >
Left;" << endl; }
31 };
32
33 class ColleagueRight : public Colleague
```

```

34 {
35     public:
36         void receive(shared_ptr<Message> msg) override { cout << "Left - >
Right;" << endl; }
37 };
38
39 class Mediator
40 {
41     protected:
42         list<shared_ptr<Colleague>> colleagues;
43
44     public:
45         virtual ~Mediator() = default;
46
47         virtual bool send(const Colleague* colleague, shared_ptr<Message> msg)
= 0;
48
49         static bool add(shared_ptr<Mediator> mediator,
initializer_list<shared_ptr<Colleague>> list);
50 };
51
52 class ConMediator : public Mediator
53 {
54     public:
55         bool send(const Colleague* colleague, shared_ptr<Message> msg)
override;
56 };
57
58 bool Colleague::send(shared_ptr<Message> msg)
59 {
60     shared_ptr<Mediator> mdr = mediator.lock();
61
62     return mdr ? mdr->send(this, msg) : false;
63 }
64
65 bool Mediator::add(shared_ptr<Mediator> mediator,
initializer_list<shared_ptr<Colleague>> list)
66 {
67     if (!mediator || list.size() == 0) {
68         return false;
69     }
70
71     for (auto elem : list) {
72         mediator->colleagues.push_back(elem);
73         elem->setMediator(mediator);
74     }
75
76     return true;

```

```

77 }
78
79 bool ConMediator::send(const Colleague* colleague, shared_ptr<Message>
msg)
80 {
81     bool flag = false;
82     for (auto&& elem : colleagues) {
83         if (dynamic_cast<const ColleagueLeft*>(colleague) &&
dynamic_cast<ColleagueRight*>(elem.get())) {
84             elem->receive(msg);
85             flag = true;
86         } else if (dynamic_cast<const ColleagueRight*>(colleague) &&
dynamic_cast<ColleagueLeft*>(elem.get())) {
87             elem->receive(msg);
88             flag = true;
89         }
90     }
91
92     return flag;
93 }
94
95 int main()
96 {
97     shared_ptr<Mediator> mediator = make_shared<ConMediator>();
98
99     shared_ptr<Colleague> col1 = make_shared<ColleagueLeft>();
100     shared_ptr<Colleague> col2 = make_shared<ColleagueRight>();
101     shared_ptr<Colleague> col3 = make_shared<ColleagueLeft>();
102     shared_ptr<Colleague> col4 = make_shared<ColleagueLeft>();
103
104     Mediator::add(mediator, {col1, col2, col3, col4});
105
106     shared_ptr<Message> msg = make_shared<Message>();
107
108     col1->send(msg);
109     col2->send(msg);
110 }

```

32. Паттерны поведения: посетитель (Visitor), опекун (Memento), шаблонный метод (Template Method), хранитель (Holder), итератор (Iterator), свойство (Property). Их преимущества и недостатки

32.1. Паттерны поведения

- Это подход, где мы выносим изменяемые части поведения (реализации алгоритмов/методов) из сущности в отдельные сущности и подставляем их при работе.
- Формулировка через сравнение: «вырожденный мост» — это паттерн поведения; если мост отделяет сущность от реализации «в целом», то в поведенческом подходе отделяется реализация одного конкретного метода (алгоритма).

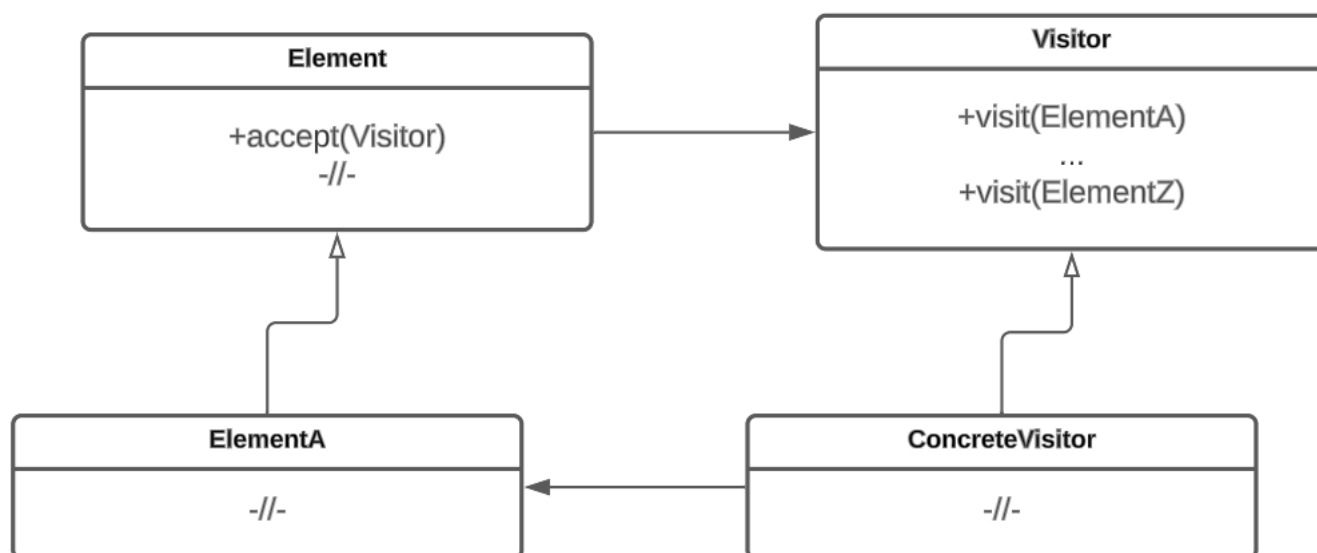
32.2. Посетитель (Visitor)

32.2.1. Идея и цель

- Избавиться от привязки к конкретной реализации и не плодить множество иерархий стратегий для каждой сущности.
- Объединить стратегии для разных сущностей (модель, камера, источник света и т. п.) в одну иерархию посетителей.

32.2.2. Как устроен (по словам лектора)

- У посетителя определяются методы, перегруженные по конкретным типам объектов.
- Сущность должна иметь указатель на посетителя; вводится простой метод ассерт, который принимает посетителя и инициирует вызов нужного метода у него.



32.2.3. Преимущества

- + Объединяет кучу иерархий стратегий в одну иерархию визитёров, уменьшая разрастание классов.

- + Помогает избавиться от реализации в том смысле, что операции над разными сущностями сводятся к единому интерфейсу посетителя и выбираются по конкретному типу аргумента (перегрузка).

32.2.4. Недостатки

- При развитии иерархии и появлении новых сущностей нужно модифицировать всех посетителей — это плохо.
- Метод должен иметь доступ к реализации — дружба: необходимость `friend`-связей — «не очень», но используется ради упрощения иерархии.

32.2.5. Пример

```
1 #include <iostream>
2 #include <memory>
3 #include <vector>
4
5 using namespace std;
6
7 class Circle;
8 class Rectangle;
9
10 class Visitor
11 {
12 public:
13     virtual ~Visitor() = default;
14
15     virtual void visit(Circle& ref) = 0;
16     virtual void visit(Rectangle& ref) = 0;
17 };
18
19 class Shape
20 {
21 public:
22     virtual ~Shape() = default;
23
24     virtual void accept(shared_ptr<Visitor> visitor) = 0;
25 };
26
27 class Circle : public Shape
28 {
29 public:
30     void accept(shared_ptr<Visitor> visitor) override { visitor-
31 >visit(*this); }
32 };
33
34 class Rectangle : public Shape
35 {
36 public:
```

```

36     void accept(shared_ptr<Visitor> visitor) override { visitor-
>visit(*this); }
37 };
38
39 class ConVisitor : public Visitor
40 {
41     public:
42     void visit(Circle& ref) override { cout << "Circle;\n"; }
43     void visit(Rectangle& ref) override { cout << "Rectangle;\n"; }
44 };
45
46 class Figure : public Shape
47 {
48     using Shapes = vector<shared_ptr<Shape>>;
49
50     private:
51     Shapes shapes;
52
53     public:
54     Figure(initializer_list<shared_ptr<Shape>> list)
55     {
56         for (auto&& elem : list) {
57             shapes.emplace_back(elem);
58         }
59     }
60
61     void accept(shared_ptr<Visitor> visitor) override
62     {
63         for (auto& elem : shapes) {
64             elem->accept(visitor);
65         }
66     }
67 };
68
69 int main()
70 {
71     shared_ptr<Shape> figure = make_shared<Figure>(
72         initializer_list<shared_ptr<Shape>>({
73             make_shared<Circle>(),
74             make_shared<Rectangle>(),
75             make_shared<Circle>()
76         }));
77
78     shared_ptr<Visitor> visitor = make_shared<ConVisitor>();
79
80     figure->accept(visitor);
81 }

```

32.3. Опекун (Memento)

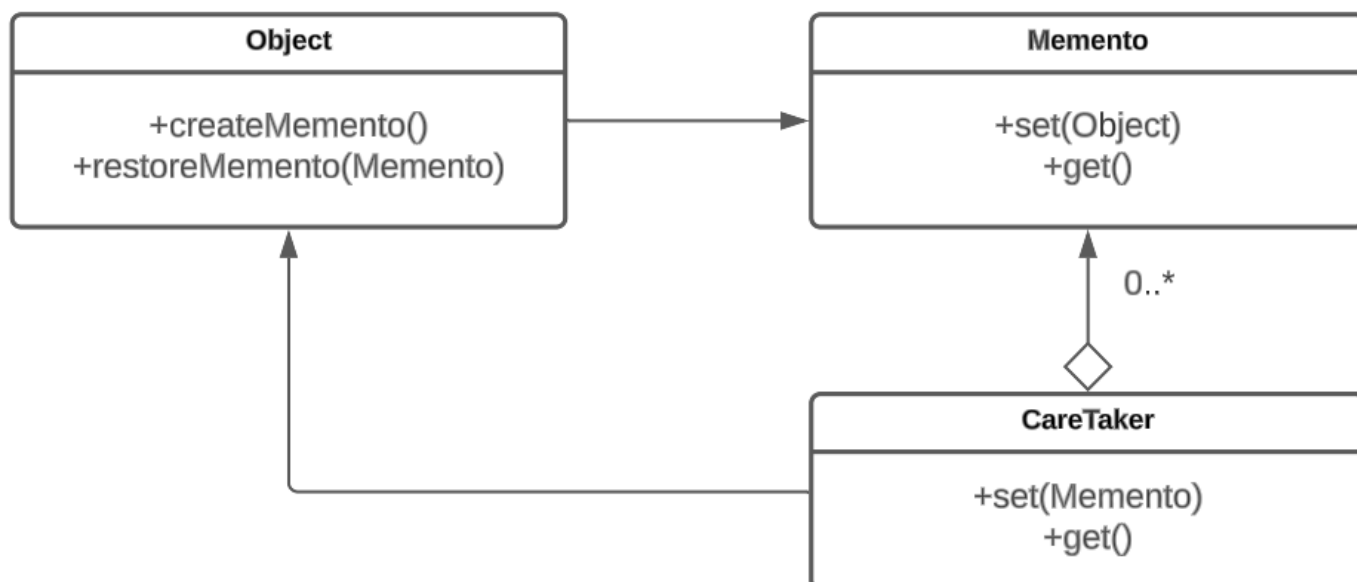
- Очень часто в задачах, особенно в программах, нужно выполнять действия и иметь возможность вернуться в предыдущее состояние.
- Задача: реализовать механизм отката к предыдущим состояниям объекта.
- Реализация не должна возлагать обязанность хранения предыдущих состояний на сам изменяемый объект, потому что на любой объект должна быть возложена только одна обязанность. Поэтому создаётся отдельный объект, который хранит состояния другого объекта — это и есть паттерн «Опекун» (Memento).
- Опекун (англ. Memento — снимок/кадр): объект, который может сохранить внутреннее состояние другого объекта, чтобы восстановить его позже.

32.3.1. Идея

- Внутренние данные объекта, которые определяют его состояние, оборачиваются в отдельный объект — «снимок» (memento). Этот снимок возвращается по запросу и используется для восстановления состояния.
- В дополнение к основному интерфейсу объекта добавляется метод(-ы) для возврата и восстановления состояния на основе снимка.

32.3.2. Как устроена архитектура

- Есть основной объект, в котором помимо стандартного интерфейса появляется возможность возвращать «снимок состояния».
- Опекун отвечает за хранение снимков состояний. Он хранит коллекцию снимков (часто реализуется как контейнер, например стек).
- Снимков может быть сколько угодно, они могут быть полными или частичными, и логика их хранения (сколько, как долго, какие именно) определяется отдельно. Например, можно ограничивать хранение по времени, по количеству, хранить только опорные снимки, а остальные восстанавливать на лету.
- Управляет процессом отката отдельный управляющий компонент, который обращается к опекуну за нужным снимком. Таким образом, ни основной объект, ни опекун не управляют самим процессом возврата; опекун просто хранит снимки и возвращает их по запросу, а управление внешнее.
- Для хранения снимков может использоваться контейнер (например, стек). Опекун должен реализовывать внутреннюю (часто абстрактную) логику хранения снимков, от которой можно наследоваться для расширенного функционала (например, ограничение числа снимков, фильтрация одинаковых снимков, хранение только опорных, и т.п.).



32.3.3. Преимущества

- + Позволяет откатываться к предыдущим состояниям объектов без изменения самого объекта.
- + Разделяет ответственность: объект занимается своим состоянием, а хранение вынесено отдельно.
- + Гибкая структура хранения: можно реализовывать различные логики хранения состояния.

32.3.4. Недостатки

- Может быть тяжёлым по памяти, если хранить множество или полные снимки (особенно для «тяжёлых» объектов).
- Реализация сложной логики хранения (например, фильтрация одинаковых снимков, оптимизация числа снимков) требует дополнительной архитектурной работы.
- Не подходит для объектов, содержащих внешние ресурсы или состояние, не представленное простыми полями (например, открытые дескрипторы файлов, соединения с БД).
- Для сложных объектов может быть сложно корректно формализовать их состояние для снимка.

32.3.5. Пример

```

1 #include <iostream>
2 #include <list>
3 #include <memory>
4
5 using namespace std;
6
7 class Memento;
8
9 class Caretaker
  
```

```

10 {
11     public:
12         unique_ptr<Memento> getMemento();
13         void setMemento(unique_ptr<Memento> memento);
14
15     private:
16         list<unique_ptr<Memento>> mementos;
17 };
18
19 class Originator
20 {
21     public:
22         Originator(int s) : state(s) {}
23
24         const int getState() const { return state; }
25         void setState(int s) { state = s; }
26
27         std::unique_ptr<Memento> createMemento() { return
make_unique<Memento>(*this); }
28         void restoreMemento(std::unique_ptr<Memento> memento);
29
30     private:
31         int state;
32 };
33
34 class Memento
35 {
36     friend class Originator;
37
38     public:
39         Memento(Originator o) : originator(o) {}
40
41     private:
42         void setOriginator(Originator o) { originator = o; }
43         Originator getOriginator() { return originator; }
44
45     private:
46         Originator originator;
47 };
48
49 void Caretaker::setMemento(unique_ptr<Memento> memento)
50 {
51     mementos.push_back(move(memento));
52 }
53
54 unique_ptr<Memento> Caretaker::getMemento()
55 {
56     unique_ptr<Memento> last = move(mementos.back());

```

```

57     mementos.pop_back();
58     return last;
59 }
60
61 void Originator::restoreMemento(std::unique_ptr<Memento> memento)
62 {
63     *this = memento->getOriginator();
64 }
65
66 int main()
67 {
68     auto originator = make_unique<Originator>(1);
69     auto caretaker = make_unique<Caretaker>();
70
71     cout << "State = " << originator->getState() << endl;
72     caretaker->setMemento(originator->createMemento());
73
74     originator->setState(2);
75     cout << "State = " << originator->getState() << endl;
76     caretaker->setMemento(originator->createMemento());
77     originator->setState(3);
78     cout << "State = " << originator->getState() << endl;
79     caretaker->setMemento(originator->createMemento());
80
81     originator->restoreMemento(caretaker->getMemento());
82     cout << "State = " << originator->getState() << endl;
83     originator->restoreMemento(caretaker->getMemento());
84     cout << "State = " << originator->getState() << endl;
85     originator->restoreMemento(caretaker->getMemento());
86     cout << "State = " << originator->getState() << endl;
87 }

```

32.4. Шаблонный метод (Template Method)

Шаблонный метод — это класс, который решает задачу, разбивая её на последовательные этапы с чёткими входами и выходами, и позволяет наблюдать и анализировать результат каждого этапа.

32.4.1. Преимущества

- + Гибкость за счёт вынесения реализаций этапов в стратегии; «каждый этап — своя стратегия».
- + Простые структурные варианты: один класс со стратегиями или иерархия с подменой.

32.4.2. Недостатки

- Внесение изменений в общую структуру алгоритма может потребовать соответствующих изменений во всех подклассах.
- При значительном увеличении числа этапов алгоритма класс может стать слишком сложным и трудным для поддержки и понимания

32.4.3. Пример

```
1 #include <iostream>
2
3 using namespace std;
4
5 class AbstractClass
6 {
7     public:
8         void templateMethod()
9         {
10             primitiveOperation();
11             concreteOperation();
12             hook();
13         }
14         virtual ~AbstractClass() = default;
15
16     protected:
17         virtual void primitiveOperation() = 0;
18         void concreteOperation() { cout << "concreteOperation;\n"; }
19         virtual void hook() { cout << "hook Base;\n"; }
20 };
21
22 class ConClassA : public AbstractClass
23 {
24     protected:
25         void primitiveOperation() override { cout << "primitiveOperation A;
26 \n"; }
27 };
28
29 class ConClassB : public AbstractClass
30 {
31     protected:
32         void primitiveOperation() override { cout << "primitiveOperation B;
33 \n"; }
34         void hook() override { cout << "hook B;\n"; }
35 };
36
37 int main()
38 {
39     ConClassA ca;
40     ConClassB cb;
41
42     ca.templateMethod();
43     cb.templateMethod();
44 }
```

32.5. Хранитель (Holder)

Это концептуальный прототип умного указателя, разработанный для устранения проблем ручного управления памятью. Он реализует принцип RAII: объект `Holder<T>` хранит `T*`, а при разрушении сам вызывает `delete`.

32.5.1. Идея

- Указатель помещается в оболочку, которая:
 - располагается на стеке;
 - управляет временем жизни объекта;
 - при уничтожении удаляет объект.
- Интерфейс `Holder` — максимально прозрачный, с перегрузкой `->`, `*`, `bool` и др.
- Обязательные компоненты:
 - запрет на копирование (нельзя допустить двойное удаление);
 - разрешение перемещения (move-семантика);
 - передача владения через методы `get()`, `release()`, `reset()`.
- `Holder` — концепция, которая ближе к `unique_ptr`.

32.5.2. Пример использования

```
1 Holder<A> obj(new A{});
2 obj->f(); // безопасный вызов
```

Даже если `f()` выбросит исключение, `Holder` вызовет `delete` в своём деструкторе.

32.5.3. Пример

```
1 template <typename T>
2 class Holder
3 {
4     T* ptr = nullptr;
5     public:
6     explicit Holder(T* p = nullptr) noexcept : ptr(p) {}
7     ~Holder() { delete ptr; }
8
9     Holder(const Holder&) = delete;
10    Holder& operator=(const Holder&) = delete;
11
12    Holder(Holder&& other) noexcept :
13        ptr(std::exchange(other.ptr, nullptr)) {}
14    Holder& operator=(Holder&& other) noexcept
15    {
16        if (this != &other) {
17            delete ptr;
18            ptr = std::exchange(other.ptr, nullptr);
19        }
20        return *this;
21    }
22 }
```

```

23     T* get() const noexcept { return ptr; }
24     T& operator*() const noexcept { return *ptr; }
25     T* operator->() const noexcept { return ptr; }
26     operator bool() const noexcept { return ptr != nullptr; }
27
28     T* release() noexcept // отдать владение
29     {
30         return std::exchange(ptr, nullptr);
31     }
32     void reset(T* p = nullptr) noexcept
33     {
34         if (ptr != p) {
35             delete ptr;
36             ptr = p;
37         }
38     }
39 };

```

32.6. Итератор (Iterator)

Итератор предоставляет способ последовательного доступа ко всем элементам составного объекта, не раскрывая его внутреннего представления.

32.6.1. Решаемые задачи

- Сохранение инкапсуляции объектов при переборе в структуре данных.
- Поддержка нескольких активных обходов одного и того же агрегированного (составленного из подобъектов) объекта.
- Предоставление единообразного интерфейса с целью обхода различных агрегированных структур (поддержка полиморфной итерации).

32.6.2. Преимущества

- + Упрощение работы со структурами данных: обход элементов без знания об особенностях внутренней реализации структуры данных.
- + Возможность работать с различными типами структур данных независимо от их реализации.
- + Возможность реализовывать различные алгоритмы обработки структур данных.

32.6.3. Недостатки

- Добавление новых типов структур данных может потребовать изменения кода итератора.

32.6.4. Пример

32.7. Свойство (Property)

- В современных языках программирования появляется понятие «свойство», которое по существу объединяет методы `get` и `set`.

- Мы не должны давать прямой доступ к членам данных класса, однако работать с ними обычно требуется.
- Для этого реализуются методы, чтобы обезопасить целостность объекта, чтобы никто извне её не нарушил.
- Мы работаем как с членом, а реально вызываются методы, мы этого не видим.

32.7.1. Преимущества

- + Удобство доступа: можно работать с данными, как с обычными членами класса, при этом реально под капотом вызываются методы (инкапсуляция, скрытие реализации).
- + Защищённость целостности данных: внешний код не может напрямую изменить данные, только через контролируемые «свойства».
- + Унификация работы с атрибутами: свойства реализуют как чтение, так и запись данных с возможностью добавления дополнительной логики.

32.7.2. Недостатки

- Не во всех языках свойства реализованы как часть синтаксиса (например, в C++ это нужно делать вручную).
- Вручную реализованные свойства сложнее и громоздче, чем встроенные средства языков.

32.7.3. Пример

```
1 #include <iostream>
2 #include <memory>
3
4 using namespace std;
5
6 template <typename Owner, typename Type>
7 class Property
8 {
9     using Getter = Type (Owner::*)() const;
10    using Setter = void (Owner::*)(const Type&);
11
12    private:
13        Owner* owner;
14        Getter methodGet;
15        Setter methodSet;
16
17    public:
18        Property() = default;
19        Property(Owner* const ovr, Getter getmethod, Setter setmethod)
20            : owner(ovr), methodGet(getmethod), methodSet(setmethod)
21        {
22        }
23
24        void init(Owner* const ovr, Getter getmethod, Setter setmethod)
25        {
```

```

26     owner = ovr;
27     methodGet = getmethod;
28     methodSet = setmethod;
29 }
30
31 operator Type() { return (owner->*methodGet)(); } // Getter
32 void operator=(const Type& data) { (owner->*methodSet)(data); } //
Setter
33
34 // Property(const Property&) = delete;
35 // Property& operator=(const Property&) = delete;
36 };
37
38 class Object
39 {
40     private:
41         double value;
42
43     public:
44         Object(double v) : value(v) { Value.init(this, &Object::getValue,
&Object::setValue); }
45
46         double getValue() const { return value; }
47         void setValue(const double& v) { value = v; }
48
49         Property<Object, double> Value;
50 };
51
52 int main()
53 {
54     Object obj(5.);
55     cout << "value = " << obj.Value << endl;
56
57     obj.Value = 10.;
58     cout << "value = " << obj.Value << endl;
59
60     unique_ptr<Object> ptr = make_unique<Object>(15.);
61     cout << "value =" << ptr->Value << endl;
62
63     obj = *ptr;
64     obj.Value = ptr->Value;
65 }

```